



secmgr

Design system

The open source secret manager for teams.

Contents

About this system	4
Principles	5
Voice and copy	6
Color	7
Typography	12
Layout and spacing	15
Shape and elevation	17
Motion	19
Iconography	20
Logo	22
Accessibility	23
Using the system	25
Tokens	26
Components	29
Foundations	33
Icon	33
Logo	34
Actions	36
Button	36
CopyButton	37
IconButton	39
Kbd	40
Inputs	41
Checkbox	41
Field	42
Input	44
RadioGroup	46
SearchField	48
SegmentedControl	50
Select	52
Switch	54
Textarea	56
Navigation	58
Breadcrumbs	58
Sidebar	60
Tabs	63
Layout	65
AppShell	65
Card	67
CodeBlock	69
PageHeader	71

ProjectCard	72
SettingRow	74
StatTile	76
Table	78
Overlays	81
CommandMenu	81
Dialog	84
Menu	86
Popover	89
Sheet	91
Toast	93
Tooltip	95
Feedback	97
Badge	97
Callout	98
EmptyState	99
InsightRow	100
Progress	102
Skeleton	104
Spinner	105
Secrets	106
ChangesBar	106
EnvEditor	108
SecretInput	111
SecretKeyInput	113
SecretRow	115
SecretValue	121
VersionTimeline	123
Environments	125
EnvBadge	125
EnvSwitcher	126
Compare and import	128
CompareMatrix	128
DiffView	130
DropZone	133
ImportPreview	135
Activity	138
ActivityItem	138
Avatar	141

About this system

secmgr is a precision instrument that happens to be calm: monochrome, exact, quick to the value you came for, quiet everywhere else. It keeps a team's secrets for every project and environment, and it treats both the secrets and the people as if they matter: nothing glows unless it needs you, production always looks different from development, and every change can be seen before it is saved.

The open source secret manager for teams. This is the one slogan. Use it word for word in the hero, the footer, the banner, bios, meta descriptions and store listings, and never write a second one.

What is inside

PART	COUNT	WHERE
Color tokens	66, each with a light and a dark value	Color, Tokens
Type styles	17 across display, titles, body and mono	Typography
Spacing, size, radius, shadow and motion tokens	54	Tokens
Components	55	Components
Icons	109 Lucide icons	Iconography

Every value in this document comes from `tokens.json` and the component sources, and every contrast ratio is computed from the tokens, not asserted. The live, clickable previews of every component live in the design system package; this document is the reference you can read, print and link to.

Principles

1. **Quiet chrome, loud data.** Build the interface from neutrals (bg, surface, fill, border, the text ladder). Spend hue only on data: environment identity (env-*), status (success, warning, danger) and diffs (diff-*).
2. **The value is one keystroke away.** Every screen works from the keyboard. Copy is one click or c, reveal is r, the command menu is mod+k from anywhere.
3. **See before you save.** Edits stage into the ChangesBar and a diff review. Nothing overwrites silently; every save can be undone from its toast.
4. **Production is different.** Protected environments always carry their color and a lock, and changing them asks for the environment's name.
5. **Real names, exact numbers.** Copy states the count, the key and the environment: "Save 3 changes to staging", never "Save".

Voice and copy

- Write in the second person and the present tense, in sentence case, verb first. Name the action and its object: "Import .env", "Add secret", "Promote to production".
- Pair every action with its result: the button says "Copy value", the toast says "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s."
- Errors say what happened, what to do, and the number that matters: "3 lines could not be parsed. Fix the highlighted lines or skip them."
- Confirmations restate the object and the consequence: "Delete 4 secrets from production? Services reading them fail on their next deploy."
- Set technical names in mono exactly as they are: DATABASE_URL, lumen-api, staging. Environment names stay lowercase.
- Use digits for every number and n() for plurals. No exclamation marks, no emoji, no "oops", no apologies.

WHERE	EXACT COPY
Primary action in the changes bar	Save 3 changes
Copy toast	Copied STRIPE_SECRET_KEY. Clipboard clears in 30s.
Import summary	12 new · 3 changed · 8 unchanged · 1 invalid
Import button	Import 15 secrets into development
Protected save	Type production to confirm
Drift insight	FEATURE_NEW_CHECKOUT is missing in production
Danger insight	Staging uses a live Stripe key
Rotation badge	Rotation due
Empty project list	Start with one project
CLI success	Injected 24 secrets from lumen-api/staging
Network error	Could not reach the server. Your changes are kept; retrying in 5s.

Color

- The app ground is `bg`. The sidebar sits on it. The working area is an inset panel on `surface`. Menus, popovers, toasts and the command menu float on `surface-raised` with a shadow.
- Controls rest on `fill`, hover on `fill-hover`, press on `fill-active`. Rows hover on `fill`. Selected rows use `selection`.
- `text` is for primary text and icons, `text-secondary` for descriptions and inactive navigation, `text-tertiary` for timestamps, counts and placeholders. All three read at 4.5:1 or better on `bg`, `bg-subtle`, `surface`, `surface-raised` and `fill`. `text-disabled` is for disabled labels only.
- Use `primary` (ink in light, paper in dark) for exactly one action per view, with `on-primary` on it.
- `accent` is for links, info callouts and selection handles. It is not a second primary.
- Status always pairs its color with an icon and a word: success with `circle-check`, warning with `triangle-alert`, danger with `circle-alert`. Soft grounds (`success-soft` and friends) carry their own text token at 4.5:1.
- Environments own eight hues: `env-gray`, `env-blue`, `env-teal`, `env-green`, `env-amber`, `env-orange`, `env-rose`, `env-violet`. Each has a mark (`env-<hue>`, 3:1 on `surface` and `bg`), a chip ground (`env-<hue>-soft`) and a label (`env-<hue>-text`, 4.5:1 on its chip and on `surface`). Defaults: development blue, preview violet, staging amber, production rose.
- Diffs alias status: `diff-add` and `diff-add-text` for added, `diff-change` for changed, `diff-remove` for removed.
- The `.env` editor uses `code-key`, `code-punct`, `code-value`, `code-comment` and `code-ref`.
- The focus ring is `focus` (ink in light, paper in dark), a 1.5px outline that grows outward from light to full strength over `duration-focus`, never a filled border. Fields (`Input`, `SecretInput`, `Textarea`, `EnvEditor`, `Select`) wear it flush against their own border; buttons and links keep a 2px offset so the ring stays visible against `primary` fills, at 3:1 or better on every ground in both themes.

Every color token

Swatches show the light value, then the dark value. An alias names the token it follows.

Grounds, fills and lines

LIGHT	DARK	TOKEN	VALUES	USE
		<code>bg</code>	<code>#f9f9f8</code> <code>#09090a</code>	App ground behind the inset panel: sidebar, auth screens, marketing paper sections.
		<code>bg-subtle</code>	<code>#f2f2f0</code> <code>#131314</code>	Hover and selected rows on <code>bg</code> (sidebar items), table headers, code blocks inside <code>surface</code> .
		<code>surface</code>	<code>ffffff</code> <code>#101011</code>	The inset main panel, cards, sheets and dialogs.
		<code>surface-raised</code>	<code>ffffff</code> <code>#18181a</code>	Menus, popovers, the command menu and toasts, always with a <code>shadow-md</code> or larger.

LIGHT	DARK	TOKEN	VALUES	USE
		fill	#f0f0ee #1b1b1d	Resting fill of secondary controls, chips, segmented control track, skeletons.
		fill-hover	#e8e8e5 #232326	Hover fill for secondary controls and rows on surface.
		fill-active	#dfdfdb #2c2c30	Pressed fill, selected segmented item track in dark, drag handles.
		border	#e8e8e5 #222225	Hairlines: panel edges, row dividers, card outlines. Decorative, below 3:1 by design.
		border-strong	#d9d9d5 #303034	Input and button outlines at rest. Paired with a fill or label so the control never relies on the border alone.
		border-hover	#bdbdb7 #48484e	Input and button outlines on hover.
		selection	#d7e3fb #1f3a66	Text selection highlight and selected table rows.
		overlay	rgba(24, 24, 22, 0.32) rgba(0, 0, 0, 0.62)	Scrim behind dialogs and sheets.

Text

LIGHT	DARK	TOKEN	VALUES	USE
		text	#1a1a19 #eeced	Primary text and icons on every ground: bg, bg-subtle, surface, surface-raised, fill.
		text-secondary	#5c5c58 #a1a1a7	Secondary text: descriptions, metadata, inactive nav. On every ground.
		text-tertiary	#6b6b67 #8a8a91	Tertiary text: timestamps, placeholders, counts, = in .env lines. On every ground, 4.5:1 minimum.
		text-disabled	#a9a9a4 #56565c	Disabled labels only. Exempt from contrast by WCAG; always paired with a disabled cursor and no hover.

Actions and focus

LIGHT	DARK	TOKEN	VALUES	USE
		primary	#1a1a19 #eeced	The one primary action per view (ink button), checked checkboxes, selected segmented item text in light.
		primary-hover	#353532 #d2d2d5	Hover on primary fills.
		on-primary	#fafaf9 #0a0a0b	Text and icons on primary fills.
		accent	#2563d9 #5c9bff	Links, focus-adjacent emphasis, info callouts, selection handles. Text on every ground and on accent-soft.

LIGHT	DARK	TOKEN	VALUES	USE
		accent-soft	#eef3fd #15223a	Info callout ground, selected list item tint, drop zone active ground.
		on-accent	ffffff #0a0a0b	Text on an accent fill.
		focus	#1a1a19 #eeced	The focus ring: ink in light, paper in dark. 1.5px outline, 2px offset, grows out from flush and light to its offset and full strength over duration-focus. At least 3:1 on every ground in both themes.

Status

LIGHT	DARK	TOKEN	VALUES	USE
		success	#18794a #3fcf7a	Success icon and text (saved, in sync, identical). On every ground and on success-soft.
		success-soft	#e9f6ee #0f2a1b	Success callout and badge ground, added lines in diffs.
		warning	#8f5500 #f0b23e	Warning icon and text (drift, stale, rotation due). On every ground and on warning-soft.
		warning-soft	#fcf3e0 #2d220f	Warning callout and badge ground, changed values in diffs.
		danger	#c42b2b #ff6b61	Destructive actions, errors, removed lines. On every ground and on danger-soft.
		danger-soft	#fcecec #3a1614	Error callout and badge ground, removed lines in diffs.
		danger-hover	#a82222 #ff857c	Hover on danger buttons.
		on-danger	ffffff #1a0605	Text on a danger fill.

Environments

LIGHT	DARK	TOKEN	VALUES	USE
		env-gray	#85857f #8f8f96	Environment mark: dot, stripe, lock glyph. 3:1 on surface.
		env-gray-soft	#f0f0ee #1f1f22	Environment chip ground.
		env-gray-text	#52524e #c4c4ca	Environment chip label, on env-gray-soft and surface.
		env-blue	#3b7cf0 #5b9bff	Environment mark. Default for development.
		env-blue-soft	#ebf2fe #13213a	Environment chip ground.
		env-blue-text	#1d4fc4 #9cc2ff	Environment chip label.

LIGHT	DARK	TOKEN	VALUES	USE
		env-teal	#0f8f84 #2dd4bf	Environment mark.
		env-teal-soft	#e5f5f3 #0d2a27	Environment chip ground.
		env-teal-text	#0b6259 #7ee6d7	Environment chip label; also string values in the .env editor.
		env-green	#23924f #3fcf7a	Environment mark.
		env-green-soft	#e8f6ec #0f2a1b	Environment chip ground.
		env-green-text	#17693a #86e3aa	Environment chip label.
		env-amber	#b27800 #f5b83d	Environment mark. Default for staging.
		env-amber-soft	#fcf3dc #2e2310	Environment chip ground.
		env-amber-text	#7d5200 #f8cf7a	Environment chip label.
		env-orange	#dd6417 #ff8a3d	Environment mark.
		env-orange-soft	#fdeee4 #33200f	Environment chip ground.
		env-orange-text	#9c4108 #ffb584	Environment chip label.
		env-rose	#e0424b #ff6b72	Environment mark. Default for production.
		env-rose-soft	#fdecee #3a1519	Environment chip ground.
		env-rose-text	#ad1f2b #ffa3a8	Environment chip label.
		env-violet	#8457f0 #a78bfa	Environment mark. Default for preview.
		env-violet-soft	#f1ecfe #221a3d	Environment chip ground.
		env-violet-text	#5f30d0 #c9b8ff	Environment chip label; also \${REFERENCE} tokens in values.

Diffs

LIGHT	DARK	TOKEN	VALUES	USE
		diff-add (success-soft)	#e9f6ee #0f2a1b	Ground of an added key or line in compare, import and review.
		diff-add-text (success)	#18794a #3fcf7a	Marker and text of an added key or line.

LIGHT	DARK	TOKEN	VALUES	USE
		diff-change (warning-soft)	#fcf3e0 #2d220f	Ground of a changed value.
		diff-change-text (warning)	#8f5500 #f0b23e	Marker and text of a changed value.
		diff-remove (danger-soft)	#fcecec #3a1614	Ground of a removed key or line.
		diff-remove-text (danger)	#c42b2b #ff6b61	Marker and text of a removed key or line.

Code

LIGHT	DARK	TOKEN	VALUES	USE
		code-key (text)	#1a1a19 #eeced	Keys in the .env editor and raw views.
		code-punct (text-tertiary)	#6b6b67 #8a8a91	=, quotes and export in the .env editor.
		code-value (env-teal-text)	#0b6259 #7ee6d7	Values in the .env editor.
		code-comment (text-tertiary)	#6b6b67 #8a8a91	# comments in the .env editor, set italic.
		code-ref (env-violet-text)	#5f30d0 #c9b8ff	\${OTHER_KEY} references inside values.

Typography

- One family everywhere. **Geist** sets everything people read, from 11px overlines to 88px marketing headlines. **Geist Mono** sets anything a machine reads: keys, values, slugs, tokens, commands. Hierarchy comes from size, weight (300 to 600) and tracking, never from a second face. Both ship as variable woff2 files and render the same on every device.
- Titles: `title-lg` once per view for the page title, `title` for dialog and sheet titles, `heading` for card and group titles.
- Body: `body` by default, `body-medium` for buttons and navigation, `small` for descriptions and table metadata, `caption` for timestamps and hints, `overline` (uppercase, 0.06em tracking) for sidebar group labels and table group headers.
- Mono: `mono-medium` for secret keys in rows, `mono` for values and slugs, `mono-sm` for masked previews and token prefixes, `code` for code blocks and the editor.
- Display: `display-xl` (500) once on the homepage hero, with its second clause at 300 in `text-tertiary`; `display` (500) for marketing section openers; `display-light` (300) for statements and the manifesto; `display-sm` (500) for page-level empty states and auth titles.
- Tighten tracking as size grows (`title-lg` at -0.02em, `display` at -0.035em, `display-xl` at -0.045em); leave body text at 0.
- Use weight for emphasis, not italics or a second face: 300 for quiet contrast in large type, 400 for body, 500 for labels and buttons, 600 for titles.
- Set counts, times and anything in columns with `font-variant-numeric: tabular-nums` (the `sg-tabular` class).

Type scale

Display

`display-xl`

88px / 0.95 / 500 / -0.045em

The open source
secret manager
for teams.

display
56px / 1 / 500 / -0.035em

See drift before
production does.

display-sm
36px / 1.1 / 500 / -0.025em

Start with one project.

display-light
56px / 1.05 / 300 / -0.03em

Secret manager
for teams.

Titles

title-lg
24px / 32px / 600 / -0.02em

acme-api

title
18px / 26px / 600 / -0.015em

Environments

heading
14px / 20px / 600 / -0.005em

Danger zone

Body

body
14px / 20px / 400

Secrets are encrypted before they leave your browser.

body-medium
14px / 20px / 500

Import 12 secrets

small
13px / 18px / 400

Updated 4 minutes ago by Maya

small-medium
13px / 18px / 500

staging

caption
12px / 16px / 400

Last used 2 days ago

overline
11px / 16px / 500 / 0.06em

ENVIRONMENTS

Mono

mono
13px / 20px / 400

STRIPE_SECRET_KEY

mono-medium
13px / 20px / 500

DATABASE_URL

mono-sm

12px / 16px / 400

sk_live_4eC39...Pn7

code

13px / 22px / 400

secmgr run --env staging -- npm run dev

Layout and spacing

- The app is a `--sidebar` (240px) column on `bg` and an inset panel on `surface` with `--r-12` corners, inset 8px from the top, right and bottom. The panel's header row is `--header` (48px) with a hairline under it. Only the panel scrolls.
- Page content inside the panel is at most `--content` (1120px) wide with `space-24` side padding. Settings and onboarding columns are at most `--prose` (640px).
- Spacing uses a 4px grid with 2px and 6px half steps; token names are pixel values (`space-8`, `space-12`, `space-16`). Use `gap` in flex and grid layouts, never stacked margins.
- Controls are `control-sm` (28px) in toolbars, `control-md` (32px) by default, `control-lg` (40px) on auth and onboarding. Secret rows are `row` (44px), or `row-compact` (36px) at compact density.
- Under 768px the sidebar becomes a drawer and the panel goes full bleed. Under 640px rows stack and dialogs become bottom sheets.

Spacing scale

<code>space-0</code>	0px	
<code>space-2</code>	2px	
<code>space-4</code>	4px	
<code>space-6</code>	6px	
<code>space-8</code>	8px	
<code>space-10</code>	10px	
<code>space-12</code>	12px	
<code>space-16</code>	16px	
<code>space-20</code>	20px	
<code>space-24</code>	24px	
<code>space-32</code>	32px	
<code>space-40</code>	40px	
<code>space-48</code>	48px	
<code>space-64</code>	64px	
<code>space-96</code>	96px	

Sizes

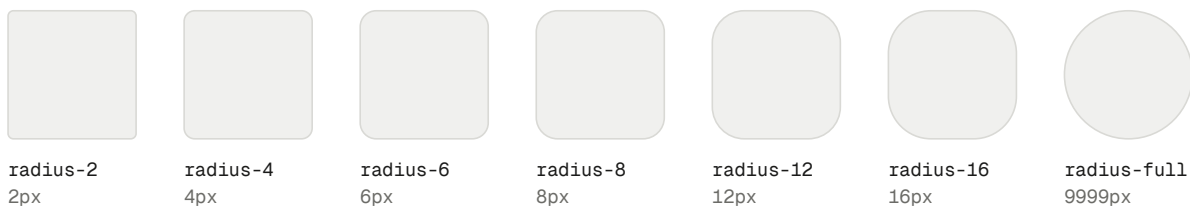
TOKEN	VALUE	USE
<code>control-xs</code>	24px	Icon buttons inside rows and chips.
<code>control-sm</code>	28px	Toolbar buttons, filters, compact inputs.
<code>control-md</code>	32px	Default buttons, inputs, selects.

TOKEN	VALUE	USE
control-lg	40px	Auth and onboarding buttons and inputs.
row-compact	36px	Secret rows at compact density.
row	44px	Secret rows, list rows, settings rows at comfortable density.
icon-sm	14px	Icons in badges, chips and kbd rows.
icon-md	16px	Default icon size.
icon-lg	20px	Empty states and callout icons.
header	48px	Top bar of the inset panel.
sidebar	240px	Sidebar width.
sheet	520px	Side sheet width (secret detail, import).
dialog-sm	400px	Confirmations.
dialog-md	520px	Forms.
dialog-lg	760px	Review changes, import preview.
content	1120px	Max width of page content inside the panel.
prose	640px	Max width of settings and onboarding columns.

Shape and elevation

- Corners are smoothed. Where the browser supports `corner-shape: superellipse(1.6)`, G2 continuous curvature) and its radius is multiplied by `corner-scale` (1.33) so it reads the same size as a round corner; elsewhere corners fall back to plain round corners at the base radius. Components always use the derived radii `--r-4`, `--r-6`, `--r-8`, `--r-12`, `--r-16`, never `--radius-*` directly.
- Radius grows with the object: chips, badges and checkboxes `--r-4`; buttons, inputs, rows and menu items `--r-6`; menus, cards, popovers, toasts and code blocks `--r-8`; the inset panel, dialogs, sheets and the command menu `--r-12`; marketing frames `--r-16`.
- Pills, avatars, dots and switches use `--radius-full` with `corner-shape: round` so they stay true circles.
- Draw hairlines with `border` as a 1px inset box shadow or border. Input and button outlines use `border-strong`, `border-hover` on hover.
- Only floating things get a shadow: `shadow-xs` for secondary buttons, `shadow-sm` for the inset panel and the changes bar, `shadow-md` for menus, tooltips and toasts, `shadow-lg` for dialogs and sheets, `shadow-xl` for the command menu. Every shadow starts with a 1px ring so edges hold in dark.

Radius



TOKEN	VALUE	USE
<code>radius-2</code>	2px	Diff markers, inline code, focus-adjacent details.
<code>radius-4</code>	4px	Badges, kbd, env chips, checkboxes.
<code>radius-6</code>	6px	Buttons, inputs, menu items, table rows, nav items.
<code>radius-8</code>	8px	Menus, popovers, cards, toasts, code blocks.
<code>radius-12</code>	12px	The inset main panel, dialogs, sheets, the command menu.
<code>radius-16</code>	16px	Marketing cards and product frames.
<code>radius-full</code>	9999px	Pills: marketing nav and CTA, avatars, switches, status dots.

Shadows

TOKEN	LIGHT	DARK	USE
shadow-xs	0 1px 0 rgba(20, 20, 18, 0.04)	0 1px 0 rgba(0, 0, 0, 0.4)	Secondary buttons and inputs at rest.
shadow-sm	0 0 0 1px rgba(20, 20, 18, 0.06), 0 1px 2px rgba(20, 20, 18, 0.06)	0 0 0 1px rgba(255, 255, 255, 0.06), 0 1px 2px rgba(0, 0, 0, 0.5)	Cards that float on bg, the inset panel, the changes bar.
shadow-md	0 0 0 1px rgba(20, 20, 18, 0.07), 0 4px 12px -2px rgba(20, 20, 18, 0.08), 0 12px 32px -8px rgba(20, 20, 18, 0.12)	0 0 0 1px rgba(255, 255, 255, 0.08), 0 8px 24px -6px rgba(0, 0, 0, 0.7)	Menus, popovers, tooltips, toasts.
shadow-lg	0 0 0 1px rgba(20, 20, 18, 0.06), 0 16px 48px -12px rgba(20, 20, 18, 0.24)	0 0 0 1px rgba(255, 255, 255, 0.08), 0 24px 64px -12px rgba(0, 0, 0, 0.8)	Dialogs and sheets.
shadow-xl	0 0 0 1px rgba(20, 20, 18, 0.06), 0 32px 80px -16px rgba(20, 20, 18, 0.32)	0 0 0 1px rgba(255, 255, 255, 0.09), 0 32px 96px -16px rgba(0, 0, 0, 0.85)	The command menu only.

Layers

TOKEN	VALUE	USE
z-sticky	10	Sticky table headers and the changes bar.
z-popover	50	Menus, selects, popovers.
z-sheet	60	Side sheets.
z-dialog	70	Dialogs.
z-command	75	The command menu.
z-toast	80	Toasts.
z-tooltip	90	Tooltips.

TOKEN	VALUE	USE
corner-shape	superellipse(1.6)	Corner curve for every rounded element where the browser supports corner-shape: G2 continuous curvature, the smoothed corner. Pills and circles opt out with corner-shape: round.
corner-scale	1.33	Radius multiplier applied with the smoothed corner, so a smoothed 8 reads the same size as a round 8. Components use the derived --r-* radii, never --radius-* directly.

Motion

- Things enter with ease-out and leave faster with ease-in. Things that move between two resting places use ease-in-out. ease-spring is reserved for the copy checkmark and the changes bar.
- duration-instant (80ms) for hover and press, duration-fast (140ms) for menus, tooltips and reveals, duration-base (200ms) for dialogs, sheets and toasts, duration-slow (320ms) for the changes bar. duration-reveal (700ms) exists for marketing only.
- Animate opacity and transform. Never animate layout, never animate a secret's value character by character, never loop anything in the app.
- Under prefers-reduced-motion: reduce every transition and animation collapses to 1ms; spinners stop turning and stay visible.

Durations

TOKEN	VALUE	USE
duration-instant	80ms	Hover fills, pressed states.
duration-fast	140ms	Menus and tooltips in, checkmarks, reveal of a secret value.
duration-base	200ms	Dialogs and sheets, row insertion, toasts.
duration-slow	320ms	Changes bar in and out, layout shifts.
duration-reveal	700ms	Marketing reveals only.
duration-focus	1400ms	The focus ring moving out from the control and darkening, with ease-out.

Curves

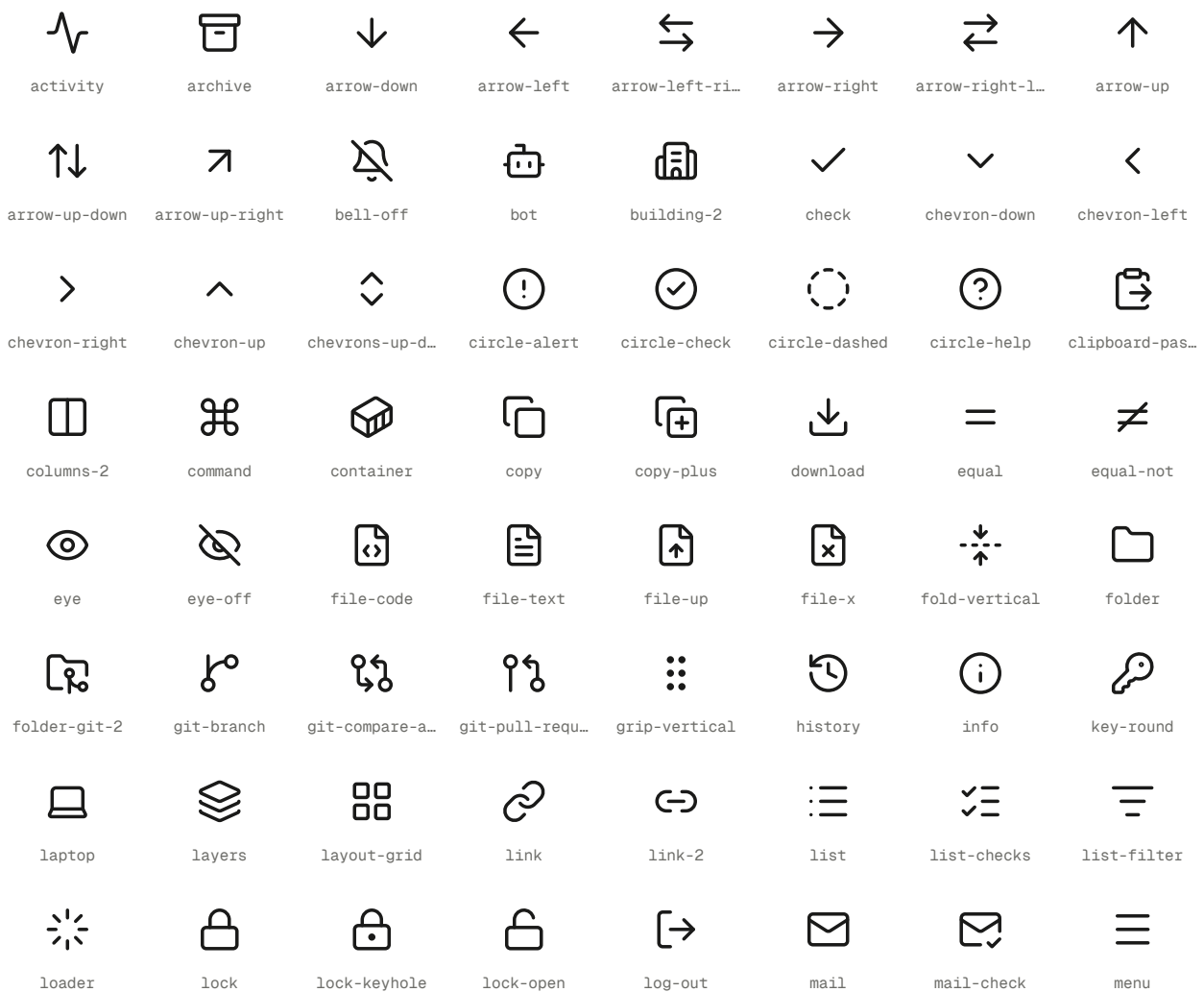


TOKEN	VALUE	USE
ease-out	<code>cubic-bezier(0.16, 1, 0.3, 1)</code>	Everything that enters.
ease-in	<code>cubic-bezier(0.5, 0, 0.75, 0)</code>	Everything that leaves.
ease-in-out	<code>cubic-bezier(0.65, 0, 0.35, 1)</code>	Things that move between two resting places.
ease-spring	<code>cubic-bezier(0.34, 1.56, 0.64, 1)</code>	The copy checkmark and the changes bar only.

Iconography

- Icons are Lucide, the set shadcn/ui uses, and nothing else: no other icon family, no emoji, no brand logos. Draw them at a 1.75 stroke in the 24 unit grid, in the current text color, with round caps and joins.
- Sizes: 14px in badges, chips and xs icon buttons; 16px by default; 20px in empty states and callouts.
- An icon never carries meaning alone: pair it with a word or give it a label.
- The icon files in the Icons group are stored in ink #85857f so they read on both themes; in the product they inherit `currentColor` through `Icon`.
- Everyday names: `key-round` (secrets), `layers` (environments), `folder-git-2` (projects), `git-compare-arrows` (compare), `upload` (import), `history` (versions), `lock` (protected), `eye` and `eye-off` (reveal), `copy`, `shield-check` (health), `terminal` (CLI).

All 109 icons

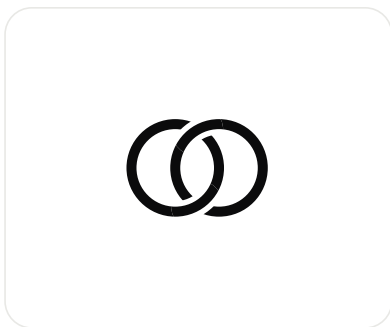


							
minus	monitor	moon	more-horizont...	panel-left	panel-right	panel-right-o...	pencil
							
plus	refresh-cw	rotate-ccw	rotate-cw	rows-3	rows-4	search	search-x
							
send	settings	shield-check	shield-x	sliders-horiz...	smartphone	sparkles	square-pen
							
square-termin...	star	sticky-note	sun	sun-moon	table-2	terminal	timer-reset
							
trash-2	triangle-alert	undo-2	unfold-vertic...	upload	user	user-minus	user-plus
							
users	wifi	wifi-off	wrap-text	x			

Logo

- The mark is two equal rings whose centres sit one radius apart, woven so each passes over the other once: two environments, one secret, held together. Use `secmgr-mark.svg` on light grounds and `secmgr-mark-light.svg` on dark ones; in code use `Mark` with `currentColor`.
- The wordmark sets "secmgr" in Geist Medium at $-0.035em$ beside the mark. Never capitalize it, never set it in another face.
- The app icon puts the light rings on an ink tile with smoothed `--r-12` corners.
- `secmgr-mark-lens.svg` is an alternate mark (rings with the shared lens filled), kept for review.

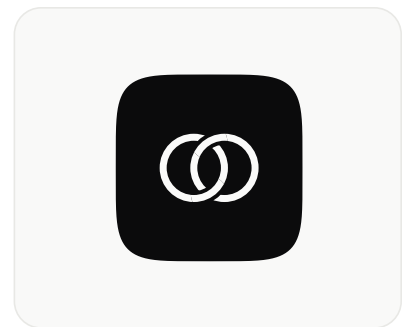
The mark, the wordmark and the app icon



Mark on light



Mark on dark



App icon

 **secmgr**

Wordmark: the mark and secmgr in Geist Medium at $-0.035em$, always lowercase.

Accessibility

- Contrast is computed from the tokens, not asserted: 180 pairs across both themes pass. Body text minimums are 4.69:1 in light (text-tertiary on fill) and 5.02:1 in dark; environment and status marks are at least 3.37:1; the focus ring is at least 3:1 on every ground.
- border and border-strong sit below 3:1 by design. Controls that rely on them also carry a fill, a label or a shadow, so no control is identified by its border alone.
- Every interactive component has visible hover, active, focus and disabled states. Menus, listboxes, tabs, the command menu and tables work with arrow keys, Home and End, Enter, Space and Escape, and return focus to their trigger. Dialogs and sheets trap focus.
- Icon buttons always have a label, read aloud and shown as a tooltip. Status is never color alone.
- Revealed secrets hide again after 10 seconds by default, and copied values clear from the clipboard after 30 seconds, both configurable in Appearance.

Computed contrast

Ratios against every ground, both themes. Text needs 4.5:1; marks, status and the focus ring need 3:1.

TOKEN	THEME	BG	BG-SUBTLE	SURFACE	SURFACE-RAISED	FILL
text	light	16.53:1	15.54:1	17.42:1	17.42:1	15.26:1
text-secondary	light	6.38:1	5.99:1	6.72:1	6.72:1	5.89:1
text-tertiary	light	5.08:1	4.77:1	5.35:1	5.35:1	4.69:1
env-blue	light	3.74:1	3.52:1	3.94:1	3.94:1	3.45:1
env-amber	light	3.57:1	3.36:1	3.76:1	3.76:1	3.30:1
env-rose	light	3.95:1	3.71:1	4.16:1	4.16:1	3.64:1
env-violet	light	4.33:1	4.07:1	4.56:1	4.56:1	4.00:1
env-teal	light	3.77:1	3.55:1	3.98:1	3.98:1	3.48:1
env-green	light	3.76:1	3.54:1	3.97:1	3.97:1	3.48:1
env-orange	light	3.37:1	3.17:1	3.55:1	3.55:1	3.11:1
env-gray	light	3.52:1	3.31:1	3.71:1	3.71:1	3.25:1
success	light	5.15:1	4.84:1	5.42:1	5.42:1	4.75:1
warning	light	5.75:1	5.40:1	6.06:1	6.06:1	5.31:1
danger	light	5.35:1	5.03:1	5.63:1	5.63:1	4.94:1
focus	light	16.53:1	15.54:1	17.42:1	17.42:1	15.26:1
text	dark	16.86:1	15.73:1	16.11:1	15.02:1	14.57:1
text-secondary	dark	7.74:1	7.23:1	7.40:1	6.90:1	6.69:1
text-tertiary	dark	5.81:1	5.42:1	5.55:1	5.17:1	5.02:1
env-blue	dark	7.18:1	6.70:1	6.86:1	6.40:1	6.21:1

TOKEN	THEME	BG	BG-SUBTLE	SURFACE	SURFACE-RAISED	FILL
env-amber	dark	11.19:1	10.44:1	10.69:1	9.97:1	9.67:1
env-rose	dark	7.20:1	6.72:1	6.88:1	6.42:1	6.22:1
env-violet	dark	7.31:1	6.82:1	6.99:1	6.52:1	6.32:1
env-teal	dark	10.69:1	9.98:1	10.22:1	9.52:1	9.24:1
env-green	dark	9.87:1	9.21:1	9.43:1	8.80:1	8.53:1
env-orange	dark	8.49:1	7.92:1	8.11:1	7.56:1	7.33:1
env-gray	dark	6.20:1	5.78:1	5.92:1	5.52:1	5.35:1
success	dark	9.87:1	9.21:1	9.43:1	8.80:1	8.53:1
warning	dark	10.56:1	9.85:1	10.09:1	9.41:1	9.13:1
danger	dark	7.13:1	6.66:1	6.82:1	6.36:1	6.16:1
focus	dark	16.86:1	15.73:1	16.11:1	15.02:1	14.57:1

Using the system

- Load `tokens.css`, the fonts, `components/bundle.css`, React 18 and ReactDOM 18, then `components/bundle.js`. Every component is on `window.Secmgr` (`Secmgr.Button`, `Secmgr.SecretRow`, `Secmgr.toast`).
- Set `data-theme="light"` or `data-theme="dark"` on the root, or leave it unset to follow the system. A section can set its own `data-theme` to scope a theme.
- Read a component's README before using it. Build screens from components; when a screen needs something the system lacks, add it to the system first.

Tokens

Monochrome chrome, colored data. Neutrals carry the interface; hue is reserved for environments, status and diffs. One primary per view.

Tokens are the only source of values. Components and screens read them as CSS custom properties (`var(--surface)`, `var(--space-12)`), and nothing states a raw color, size or duration.

Spacing

A 4px grid with a 2px and 6px half step. Token names are their pixel values.

TOKEN	VALUE	USE
space-0	0px	Reset.
space-2	2px	Icon to label nudge, badge inner block padding.
space-4	4px	Gap inside chips, between an icon and its count.
space-6	6px	Gap between icon and label in buttons and nav items.
space-8	8px	Default gap between inline controls; button side padding at sm.
space-10	10px	Button side padding at md; menu item side padding.
space-12	12px	Row side padding, card inner gap, field label to control.
space-16	16px	Panel and card padding, gap between form fields.
space-20	20px	Dialog body padding, section gap inside a card.
space-24	24px	Page side padding inside the inset panel; dialog padding.
space-32	32px	Gap between page sections.
space-40	40px	Empty state vertical padding.
space-48	48px	Large section rhythm on settings and onboarding.
space-64	64px	Marketing section padding at narrow widths.
space-96	96px	Marketing section padding at wide widths.

Size

Control and row heights, icon sizes and layout widths.

TOKEN	VALUE	USE
control-xs	24px	Icon buttons inside rows and chips.
control-sm	28px	Toolbar buttons, filters, compact inputs.
control-md	32px	Default buttons, inputs, selects.
control-lg	40px	Auth and onboarding buttons and inputs.
row-compact	36px	Secret rows at compact density.

TOKEN	VALUE	USE
row	44px	Secret rows, list rows, settings rows at comfortable density.
icon-sm	14px	Icons in badges, chips and kbd rows.
icon-md	16px	Default icon size.
icon-lg	20px	Empty states and callout icons.
header	48px	Top bar of the inset panel.
sidebar	240px	Sidebar width.
sheet	520px	Side sheet width (secret detail, import).
dialog-sm	400px	Confirmations.
dialog-md	520px	Forms.
dialog-lg	760px	Review changes, import preview.
content	1120px	Max width of page content inside the panel.
prose	640px	Max width of settings and onboarding columns.

Radius

Radius grows with the size of the object: chips 4, controls 6, menus and cards 8, panels and dialogs 12.

TOKEN	VALUE	USE
radius-2	2px	Diff markers, inline code, focus-adjacent details.
radius-4	4px	Badges, kbd, env chips, checkboxes.
radius-6	6px	Buttons, inputs, menu items, table rows, nav items.
radius-8	8px	Menus, popovers, cards, toasts, code blocks.
radius-12	12px	The inset main panel, dialogs, sheets, the command menu.
radius-16	16px	Marketing cards and product frames.
radius-full	9999px	Pills: marketing nav and CTA, avatars, switches, status dots.

Layers

Layering order, lowest to highest.

TOKEN	VALUE	USE
z-sticky	10	Sticky table headers and the changes bar.
z-popover	50	Menus, selects, popovers.
z-sheet	60	Side sheets.
z-dialog	70	Dialogs.
z-command	75	The command menu.
z-toast	80	Toasts.
z-tooltip	90	Tooltips.

Duration

Motion durations. Under `prefers-reduced-motion`: reduce every duration collapses to 0ms except opacity fades of 80ms.

TOKEN	VALUE	USE
<code>duration-instant</code>	80ms	Hover fills, pressed states.
<code>duration-fast</code>	140ms	Menus and tooltips in, checkmarks, reveal of a secret value.
<code>duration-base</code>	200ms	Dialogs and sheets, row insertion, toasts.
<code>duration-slow</code>	320ms	Changes bar in and out, layout shifts.
<code>duration-reveal</code>	700ms	Marketing reveals only.
<code>duration-focus</code>	1400ms	The focus ring moving out from the control and darkening, with ease-out.

Easing

Motion curves.

TOKEN	VALUE	USE
<code>ease-out</code>	<code>cubic-bezier(0.16, 1, 0.3, 1)</code>	Everything that enters.
<code>ease-in</code>	<code>cubic-bezier(0.5, 0, 0.75, 0)</code>	Everything that leaves.
<code>ease-in-out</code>	<code>cubic-bezier(0.65, 0, 0.35, 1)</code>	Things that move between two resting places.
<code>ease-spring</code>	<code>cubic-bezier(0.34, 1.56, 0.64, 1)</code>	The copy checkmark and the changes bar only.

Corner

Smoothed corners. Where `corner-shape` is supported, every radius is drawn as a superellipse and scaled by `corner-scale`; elsewhere corners stay round at the base radius.

TOKEN	VALUE	USE
<code>corner-shape</code>	<code>superellipse(1.6)</code>	Corner curve for every rounded element where the browser supports <code>corner-shape</code> : G2 continuous curvature, the smoothed corner. Pills and circles opt out with <code>corner-shape: round</code> .
<code>corner-scale</code>	1.33	Radius multiplier applied with the smoothed corner, so a smoothed 8 reads the same size as a round 8. Components use the derived <code>--r-*</code> radii, never <code>--radius-*</code> directly.

Components

55 components, grouped by the job they do. Each one is built only from tokens, works from the keyboard, and has hover, active, focus and disabled states.

Foundations

COMPONENT	WHAT IT IS FOR
Icon	Every pictogram in secmgr is a Lucide icon (the set shadcn/ui uses), drawn at a 1.75 stroke in the 24 unit grid, in the current text color. No other icon family, no emoji, no brand logos.
Logo	The secmgr identity in three parts: the Mark (two equal rings, centres one radius apart, woven over and under), the Wordmark (the mark and the word "secmgr"), and the AppIcon (the rings on a primary tile).

Actions

COMPONENT	WHAT IT IS FOR
Button	The control for every action a person takes. Quiet by default, with one primary per view for the thing the view exists to do.
CopyButton	The one click copy for values, tokens and CLI commands: the copy icon springs into a check for 1600ms, "Copied" is announced, and the app gets onCopied to toast the key and the clipboard timer.
IconButton	A square button that holds a single icon, for dense places such as secret rows, toolbars and headers. Its label is always there: read by screen readers and shown as a Tooltip.
Kbd	A key cap for keyboard shortcuts, shown in menus, tooltips, buttons and the shortcuts sheet.

Inputs

COMPONENT	WHAT IT IS FOR
Checkbox	A 16px box for choosing rows and toggling options that take effect when a form is saved.
Field	The frame around one control: a label, an optional description, the control, and a hint that an error replaces, with ids and ARIA wired for you.
Input	A single line text control for names, slugs, URLs, keys and values, with its ring, icon and affixes drawn on one wrapper so everything inside reads as one field.
RadioGroup	One choice from a short list that stays visible, as dots with labels or as bordered choice cards for decisions with consequences, such as how to handle import conflicts.
SearchField	The search box for secrets, projects, members and activity: an Input with a search icon, a "/" shortcut, a clear button, a live result count and a debounced onSearch.
SegmentedControl	2 to 5 mutually exclusive views or filters in one track, with a thumb that slides to the selected segment: "Table" and "Raw .env", "Matrix" and "Side by side", grid and list.
Select	A trigger styled like Input that opens a listbox in a popover, for choosing one environment, key, template or level from a known list.

COMPONENT	WHAT IT IS FOR
Switch	An on and off toggle for settings that take effect the moment they change, such as "Auto hide revealed values".
Textarea	A multi line text control for secret notes and raw .env text, sharing the Input ring so forms read as one family.

Navigation

COMPONENT	WHAT IT IS FOR
Breadcrumbs	The trail at the start of the panel header that says where you are: workspace, project, environment. Each level can carry a switcher, Vercel style.
Sidebar	The navigation column on the bg ground: the workspace switcher, the destinations, the projects with their environments, and the signed-in member.
Tabs	A row of tabs that switches views in place: underline for the views of a page (environments, project sections), pills for filters inside a view.

Layout

COMPONENT	WHAT IT IS FOR
AppShell	The frame of every signed-in screen: the Sidebar on the bg ground and the page on an inset surface panel with its own header row. Only the panel scrolls.
Card	A flat surface with a 1px border ring and --r-8 corners that groups one thing: a project, a health summary, a setting group, a snippet.
CodeBlock	A read only snippet for the CLI, a .env file or an export, with light highlighting from the code-* tokens and a copy button that swaps to a check.
PageHeader	The top of every page inside the panel: what this is, the numbers that matter, what you can do, and the views of it.
ProjectCard	A project in the projects list, as a card in the grid or a dense row in the list: its name, linked repo, environments with secret counts, health and the latest change. The whole card opens the project.
SettingRow	One setting on one row: its name and consequence on the left, its control on the right. SettingsGroup stacks rows inside a flush Card under a title.
StatTile	One headline number with its label, the change that matters, and an optional quiet trend line. Four of them open the project overview: "Secrets 89", "Environments 4", "Changes this week 17", "Issues 3".
Table	A dense, keyboard first data table for tokens, members, activity and secrets: a sticky header, hairline rows, optional selection, Linear style groups, and loading and empty states.

Overlays

COMPONENT	WHAT IT IS FOR
CommandMenu	The Cmd K palette: jump to a project, environment or secret, or run an action, without leaving the keyboard.
Dialog	A modal window for one focused task (add a secret, rename an environment) or one decision (ConfirmDialog). The page behind is dimmed, blurred and inert until you leave.
Menu	A dropdown list of actions opened from a trigger, such as the "More actions" button on a secret row or the workspace switcher. Every item works from the keyboard.

COMPONENT	WHAT IT IS FOR
Popover	A floating panel anchored to a trigger for one small task that needs interaction: filtering the secrets table, sharing a one time link, setting a rotation policy.
Sheet	A panel that slides in from the right edge for detail views and longer tasks: a secret's values across environments, the import preview, a member's access. Under 640px it rises from the bottom.
Toast	A short, temporary message that pairs an action with its result ("Copied STRIPE_SECRET_KEY. Clipboard clears in 30s.") and offers the way back ("Undo"). Call <code>toast()</code> from anywhere; render one <code>Toaster</code> .
Tooltip	A short label for one trigger: ink on paper, no arrow, shown after a hover delay or at once on keyboard focus.

Feedback

COMPONENT	WHAT IT IS FOR
Badge	A short label for a status, a count or a property, set in caption at medium weight on a 20px chip with <code>--r-4</code> corners.
Callout	A tinted note inside a page that states a fact about what you are looking at and, when there is one, the fix.
EmptyState	What an empty place is for and the one step that fills it, from a whole page ("Start with one project") down to a table with no matches.
InsightRow	A health finding with its evidence and one fix: a live key outside production, a key missing in an environment, a rotation that is overdue, values shared between environments.
Progress	A 4px bar for work people wait on: importing a .env, exporting, rotating keys, decrypting an environment, or how close a secret is to its rotation date.
Skeleton	A placeholder in the shape of the content that is on its way, so the page keeps its layout while secrets decrypt or a list loads.
Spinner	An indeterminate loading indicator in the Apple and Vercel style: 8 round capped spokes fading from full to 30 percent, ticking one spoke at a time through a full turn every 800ms, in the current text color.

Secrets

COMPONENT	WHAT IT IS FOR
ChangesBar	The floating bar that holds every staged edit until you save it: what changed, which environment it goes to, and the three ways out.
EnvEditor	The Raw .env mode of an environment: edit every secret as text, with the syntax colored, the lines numbered, and every problem marked before you stage it.
SecretInput	The field for writing a secret's value: mono, maskable, multiline when the value needs it, with <code>\$(KEY)</code> suggestions and a one click random value.
SecretKeyInput	The field for a secret's key. It fixes the key as you type, says what it changed, stops duplicates, and hands a pasted .env block to the import flow.
SecretRow	The secrets table, the screen <code>secmgr</code> exists to show: <code>SecretTable</code> holds <code>SecretTableHeader</code> and one <code>SecretRow</code> per key, works from the keyboard, groups by prefix, edits inline and shows every staged change before it is saved.
SecretValue	A secret's value as it appears in rows and detail views: hidden behind 12 fixed dots, revealed in mono on request, hidden again on its own.

COMPONENT	WHAT IT IS FOR
<code>VersionTimeline</code>	The version history of one secret, newest first: who changed it, what kind of change it was, the masked value with a reveal, a restore for older versions, and a side by side of any two versions.

Environments

COMPONENT	WHAT IT IS FOR
<code>EnvBadge</code>	The identity of an environment wherever it appears: a dot in the environment's color, its lowercase name, and a lock when it is protected.
<code>EnvSwitcher</code>	Moves between a project's environments. Tabs sit at the top of the environment panel with an underline in the active environment's own color; the select mode fits the same choice into a tight header.

Compare and import

COMPONENT	WHAT IT IS FOR
<code>CompareMatrix</code>	A grid of keys by environments that shows, without revealing a single value, which environments agree with the reference, which differ and which are missing a key.
<code>DiffView</code>	Two environments side by side, base on the left and compare on the right, with the keys that differ first and the identical keys folded into one row.
<code>DropZone</code>	Where a .env lands: a full page overlay while a file is dragged over the window, and an inline dashed box in empty states and the import sheet. It reads the file and hands over its text; nothing is saved until the person reviews it in <code>ImportPreview</code> .
<code>ImportPreview</code>	The review step of every import: each line of a pasted or dropped .env, classified against the target environment, with a choice per conflict and the exact count that will be written.

Activity

COMPONENT	WHAT IT IS FOR
<code>ActivityItem</code>	One audit log entry written as a sentence, "Priya Raman updated STRIPE_SECRET_KEY in staging", with its action icon and time, and <code>ActivityFeed</code> , which groups entries by day and joins their icons with a hairline.
<code>Avatar</code>	Who did it: a member's initials on a hue that never changes for their name, their photo when there is one, or a bot tile for a service token. <code>AvatarStack</code> overlaps several with a "+2" chip.

Icon

Lucide, one stroke width

Every pictogram in secmgr is a Lucide icon (the set shadcn/ui uses), drawn at a 1.75 stroke in the 24 unit grid, in the current text color. No other icon family, no emoji, no brand logos.

- **Sizes.** 14px in badges, chips and xs icon buttons; 16px by default; 20px in empty states and callouts. Never scale the stroke.
- **Color.** Icons inherit color. Beside text they take the text's token; alone in a toolbar they take text-secondary and lift to text on hover.
- **Meaning.** An icon never carries meaning alone. Pair it with a word, or give it a label (and use IconButton when it is clickable).
- **Adding one.** Add the Lucide name to an icons.txt file beside the component or screen that uses it, then rebuild. The build fails on any name that is not in Lucide.
- **Filled.** filled paints the shape in the current color for an on state that needs weight, such as a starred project. Keep the outline for everything else.

IconProps

Extends `Omit<React.SVGAttributes<SVGSVGElement>, "name">`.

PROP	TYPE	DESCRIPTION
name	string	Lucide icon name in kebab case, for example "key-round". Must be in the built icon set.
size?	14 16 20 24 number	Rendered size in px. Use 14 in chips and badges, 16 by default, 20 in empty states and callouts.
strokeWidth?	number	Stroke width in the 24 unit grid. Leave at 1.75.
filled?	boolean	Fills the shape with the current color. Use for on states that need weight, such as a starred project.
label?	string	Accessible name. Omit for decorative icons that sit next to a text label.

Logo

Two woven rings, the wordmark and the app tile

The secmgr identity in three parts: the Mark (two equal rings, centres one radius apart, woven over and under), the Wordmark (the mark and the word "secmgr"), and the AppIcon (the rings on a primary tile).

- **Anatomy.** The mark is drawn on a 32 unit grid: rings of radius 8.75 centred at x 11.625 and 20.375 on y 16, stroke 2, with a 1.1 gap where one ring passes under the other (a mask clipped to the upper and lower crossings). It draws in `currentColor`, so it takes text beside copy and on-primary on fills. Every instance gets its own mask ids, so a page can render any number of marks.
- **Wordmark.** "secmgr" in Geist 500 at the size you pass, tracked to -0.035em, always lowercase. The mark is 1.28em tall, sits 0.4em before the word, and centres on the x-height, the same proportions as `secmgr-wordmark.svg`. Use 16 in the sidebar and footers, 20 by default, 24 to 48 on marketing pages.
- **App icon.** The rings in on-primary on a primary tile, so it is ink on paper in light and paper on ink in dark. Corners are --r-12 scaled to the tile (a 48px tile has exactly --r-12) and smoothed where supported. Tiles of 24px and under draw a larger mark with a heavier ring so the weave stays legible; use 20 in the workspace switcher, 32 in lists, 48 and 96 in onboarding.
- **Behaviour.** Static. Never animate the rings apart, rotate them, or recolor one ring. Never place the mark on an environment color.
- **Copy.** The product is "secmgr", lowercase, everywhere, including the start of a sentence. Give `title="secmgr"` when the mark stands alone as a link or an image; omit it when the word is already beside it.
- **Composition.** In the `WorkspaceButton` a 20px `AppIcon` stands in for the workspace. On the homepage header use `Wordmark` at 20. Keep clear space of half the mark's height on every side.

MarkProps

Extends `Omit<React.SVGAttributes<SVGSVGElement>, "stroke">`.

PROP	TYPE	DESCRIPTION
<code>size?</code>	<code>number string</code>	Rendered width and height in px (or any CSS length). 16 in favicons and tight rows, 24 by default, 32 and up in headers and splash screens.
<code>title?</code>	<code>string</code>	Accessible name. Omit when the mark sits beside the word "secmgr" or is decorative.
<code>stroke?</code>	<code>number</code>	Ring stroke in the 32 unit grid. Leave at 2 to match the published logo.

WordmarkProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
size?	number	Font size of the word in px. The mark is 1.28 times this size and sits 0.4em before the word, with the x-height centred on the rings. 16 in the sidebar, 20 by default, 24 to 48 on marketing pages.
title?	string	Accessible name for the whole lockup. Defaults to "secmgr".

ApplconProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
size?	number	Tile size in px. 20 in the workspace switcher, 32 by default, 48 and 96 in onboarding and the homepage. Tiles of 24px and under draw a larger mark with a heavier ring so it stays legible.
title?	string	Accessible name. Omit when the tile sits beside the workspace or product name.

Button

Primary once per view, quiet everywhere else

The control for every action a person takes. Quiet by default, with one primary per view for the thing the view exists to do.

- **Anatomy.** Optional leading icon (16px, 14px at sm), label in body-medium (small-medium at sm), optional trailing icon, optional shortcut hint (Kbd, size sm). Heights come from control-sm, control-md, control-lg. Corners use --r-6, smoothed where supported.
- **Variants.** primary fills with primary and uses on-primary: one per view ("Save 3 changes", "Create project"). secondary sits on surface with a border-strong ring: most actions ("Import .env", "Compare"). ghost is transparent with text-secondary: toolbars, rows and menus. danger fills with danger: only the final step of an irreversible action ("Delete environment"), never the first click.
- **Behaviour.** Hover shifts the fill one step (primary-hover, fill, fill-hover). Press scales to 0.98 for duration-instant. Focus draws the focus ring at a 2px offset. loading swaps the leading icon for a Spinner, keeps the label and the width, and blocks clicks. disabled drops to fill with text-disabled.
- **Copy.** Verb first, sentence case, the object named: "Import .env", "Add secret", "Save 3 changes". Put the count in the label when the action applies to several things. Never "OK", "Submit" or "Click here".
- **Composition.** In a row of buttons the primary goes last. Pair a danger button with a secondary "Cancel". Use IconButton when there is no room for a label.

ButtonProps

Extends `Omit<React.ButtonHTMLAttributes<HTMLButtonElement>, "type">`.

PROP	TYPE	DESCRIPTION
variant?	"primary" "secondary" "ghost" "danger"	primary for the single most important action in a view, secondary for the rest, ghost in toolbars and rows, danger only for irreversible actions after a confirmation step.
size?	"sm" "md" "lg"	sm 28px in toolbars and rows, md 32px by default, lg 40px on auth and onboarding.
icon?	string	Leading Lucide icon name.
iconRight?	string	Trailing Lucide icon name, for disclosure (chevron-down) or navigation (arrow-right).
loading?	boolean	Shows a spinner in place of the leading icon, keeps the label and width, blocks clicks.
disabled?	boolean	
kbd?	string string[]	Shortcut hint shown at the end, for example ["mod", "s"].
fullWidth?	boolean	
href?	string	Renders an anchor instead of a button.
type?	"button" "submit" "reset"	
children?	React.ReactNode	

CopyButton

One click to copy, a check to confirm

The one click copy for values, tokens and CLI commands: the copy icon springs into a check for 1600ms, "Copied" is announced, and the app gets `onCopied` to toast the key and the clipboard timer.

- **Anatomy.** `icon` reuses the `IconButton` look (`control-xs`, `control-sm`, `control-md`, `ghost` or `secondary`) with a 16px copy icon (14px at `xs`). `text` reuses the small `Button` look with a 14px icon and a label that swaps from "Copy" to "Copied" inside a fixed width grid, so the button never changes size.
- **Behaviour.** Click, Enter or Space copies value (a string, or a function that returns one, sync or async, for values fetched on reveal). On success the copy icon shrinks away over `duration-instant` and the check scales and rotates in with `ease-spring` over `duration-base`, in text; after `timeout` (1600ms) it returns. A polite status outside the button announces "Copied". If the clipboard is blocked the icon turns danger and the status says "Copy failed. Select the value and copy it manually." Hover, press and focus follow `IconButton` and `Button`. `copied` forces the confirmed look when something else copied the value, such as the `c` shortcut.
- **Copy.** `label` is the action with its object: "Copy value", "Copy value of STRIPE_SECRET_KEY", "Copy command", "Copy token". Pair it with a toast from `onCopied`: "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s." Never toast a value itself.
- **Composition.** In secret rows at `xs` beside "Reveal value" and "More actions"; in CLI snippets and code blocks as text; beside a one time token reveal as `secondary md`. Only secrets the person can reveal get a copy button.

CopyButtonProps

Extends `Omit<React.ButtonHTMLAttributes<HTMLButtonElement>, "value" | "type">`.

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string number (() => string Promise<string>)</code>	The text to copy, or a function (sync or async) that returns it, for values fetched on demand.
<code>label?</code>	<code>string</code>	Accessible name and tooltip of the icon variant, written as the action. Defaults to "Copy value". Use "Copy command", "Copy token".
<code>copiedLabel?</code>	<code>string</code>	Announced and shown after copying. Defaults to "Copied".
<code>variant?</code>	<code>"icon" "text"</code>	<code>icon</code> looks like <code>IconButton</code> . <code>text</code> looks like a small <code>Button</code> whose label swaps from "Copy" to "Copied" without changing width.
<code>size?</code>	<code>"xs" "sm" "md"</code>	Icon: <code>xs</code> 24px in secret rows, <code>sm</code> 28px by default, <code>md</code> 32px. Text: <code>sm</code> 28px by default, <code>md</code> 32px.
<code>appearance?</code>	<code>"ghost" "secondary"</code>	<code>ghost</code> by default; <code>secondary</code> when it stands alone beside secondary buttons.
<code>timeout?</code>	<code>number</code>	How long the check stays, in ms. Defaults to 1600.
<code>copied?</code>	<code>boolean</code>	Forces the copied look, for when something else copied the value (the <code>c</code> shortcut) or in previews.

PROP	TYPE	DESCRIPTION
<code>onCopied?</code>	<code>(value: string) => void</code>	Called with the copied value after the clipboard write succeeds. Toast "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s." from here.
<code>disabled?</code>	<code>boolean</code>	Blocks copying, for values the person cannot reveal.
<code>children?</code>	<code>React.ReactNode</code>	Label of the text variant. Defaults to "Copy".

IconButton

Square, labelled, with a tooltip

A square button that holds a single icon, for dense places such as secret rows, toolbars and headers. Its label is always there: read by screen readers and shown as a Tooltip.

- **Anatomy.** One Lucide icon (16px, 14px at xs) centred in a square of `control-xs`, `control-sm` or `control-md`. ghost by default, secondary when it stands alone beside secondary buttons.
- **Behaviour.** Hover fills with `fill-hover` and lifts the icon to text. Press fills with `fill-active` and scales to 0.96. pressed marks toggles such as "Reveal value" with `aria-pressed` and a `fill-active` ground; an open menu trigger keeps the same ground through `aria-expanded`. Focus draws the focus ring.
- **Tooltip.** The label opens as a Tooltip after 450ms of hover, at once on keyboard focus, and instantly when you move along a toolbar (another tooltip closed less than 300ms ago). Pass `kbd` to show the shortcut after the label ("Copy value Cmd C"); chords are also exposed as `aria-keyshortcuts`. The tooltip stays closed while the button's menu is open. Use `tooltipSide` near screen edges. Set `tooltip={false}` only when the same words are visible beside the button.
- **Label.** `label` is required. It is the accessible name and the tooltip, written as the action: "Copy value", "Reveal value", "More actions". Never the icon's name.
- **Composition.** Inside a `SecretRow`, actions sit at xs, appear on row hover and focus, and stay visible on touch. As a Menu trigger, use `more-horizontal` and the label "More actions".

IconButtonProps

Extends `Omit<React.ButtonHTMLAttributes<HTMLButtonElement>, "type">`.

PROP	TYPE	DESCRIPTION
<code>icon</code>	<code>string</code>	Lucide icon name.
<code>label</code>	<code>string</code>	Required accessible name, written as the action: "Copy value", "Reveal value". Also the tooltip text.
<code>variant?</code>	<code>"ghost" "secondary"</code>	ghost in rows and toolbars, secondary when it stands alone beside secondary buttons.
<code>size?</code>	<code>"xs" "sm" "md"</code>	xs 24px inside rows and chips, sm 28px in toolbars, md 32px by default.
<code>pressed?</code>	<code>boolean</code>	Toggle state for buttons such as "Reveal value".
<code>loading?</code>	<code>boolean</code>	Shows a spinner in place of the icon and blocks clicks.
<code>disabled?</code>	<code>boolean</code>	
<code>kbd?</code>	<code>string string[]</code>	Shortcut shown in the tooltip after the label, for example "c" or ["mod", "c"]. Chords are also exposed as <code>aria-keyshortcuts</code> .
<code>tooltip?</code>	<code>boolean</code>	Shows the label as a tooltip on hover and keyboard focus. Set false only when the same words are visible beside the button.
<code>tooltipSide?</code>	<code>"top" "bottom" "left" "right"</code>	Side the tooltip opens on.
<code>type?</code>	<code>"button" "submit" "reset"</code>	

Kbd

Shortcuts, platform aware

A key cap for keyboard shortcuts, shown in menus, tooltips, buttons and the shortcuts sheet.

- **Anatomy.** One cap per key, mono-sm in text-secondary on surface with a border-strong ring and a 1px bottom lip. sm caps sit inside buttons and menu rows.
- **Behaviour.** Pass keys as a list; mod renders Cmd on macOS and Ctrl elsewhere, shift renders Shift, enter renders Enter. Sequences such as g then p are two caps with no plus sign.
- **Tones.** inverse inherits the parent's text color and sits on primary fills (primary buttons, tooltips).

KbdProps

PROP	TYPE	DESCRIPTION
keys?	string string[]	A shortcut such as ["mod", "k"] or ["g", "p"]. Rendered as one key cap per entry.
children?	React.ReactNode	A single key cap when keys is not given.
size?	"sm" "md"	
tone?	"default" "inverse"	inverse sits on primary fills, inside buttons and tooltips.
className?	string	

Checkbox

Selection and settings

A 16px box for choosing rows and toggling options that take effect when a form is saved.

- **Anatomy.** A `--r-4` box on surface with a border-hover ring; checked and mixed fill with `primary` and draw a 12px check or minus in `on-primary`. With `label`, a body label and an optional small description in `text-secondary` sit to the right and are clickable.
- **Behaviour.** Click or Space toggles. The mark scales in with `ease-spring over duration-fast`. Hover darkens the ring to `text-tertiary`. Focus draws the focus ring. `indeterminate` shows the mixed state of a select all box.
- **When to use.** Selection in tables and lists, and options inside forms that save together. For a setting that applies immediately, use `Switch`.
- **Copy.** Labels are statements that are true when checked: "Mark as sensitive", "Require two factor for production".

CheckboxProps

Extends `Omit<React.InputHTMLAttributes<HTMLInputElement>, "type" | "checked" | "defaultChecked" | "onChange">`.

PROP	TYPE	DESCRIPTION
<code>checked?</code>	<code>boolean</code>	
<code>defaultChecked?</code>	<code>boolean</code>	
<code>indeterminate?</code>	<code>boolean</code>	The mixed state of a "select all" box when only some rows are selected.
<code>onCheckedChange?</code>	<code>(checked: boolean) => void</code>	
<code>disabled?</code>	<code>boolean</code>	
<code>label?</code>	<code>React.ReactNode</code>	When given, renders a clickable label beside the box.
<code>description?</code>	<code>React.ReactNode</code>	Secondary line under the label.

Field

Label, hint or error, one control

The frame around one control: a label, an optional description, the control, and a hint that an error replaces, with ids and ARIA wired for you.

- **Anatomy.** Label in small-medium and text, with a quiet * in text-tertiary for required or "Optional" in caption for optional. An optional description under the label in caption, text-tertiary. The control. A foot row with the hint (caption, text-tertiary) or the error (caption, danger, 14px circle-alert) on the left and the counter slot (caption, tabular numbers) on the right. Rows sit space-6 apart.
- **Layout.** vertical stacks everything. horizontal puts the label and description in a 200px column with a space-24 gutter, aligned to the first line of a control-md control, for settings pages; below 640px of field width (a container query, so it works inside sheets and split panes) it stacks again.
- **Behaviour.** Field clones its single child with id, aria-describedby (description, then error or hint), aria-invalid and aria-required, and gives design system controls invalid and aria-labelledby so groups such as RadioGroup and SegmentedControl get their name. The label is a real <label>, so clicking it focuses inputs and toggles switches. An error fades in over duration-fast and replaces the hint so the row never grows twice. disabled dims the label and disables the control.
- **Copy.** Labels are short nouns in sentence case: "Key", "Value", "Reveal timeout". Hints say what is accepted or what happens: "Revealed values hide again after this long." Errors say what happened and what to do, with the number or name that matters: "DATABASE_URL already exists in staging. Edit it instead, or pick another key." Never "Invalid input".
- **Composition.** Wraps Input, Textarea, Select, RadioGroup, Switch, SegmentedControl or a native control. Stack fields space-20 apart in dialogs and forms. Show errors after the first blur or on submit, not on the first keystroke, except for format rules that convert as you type.

FieldProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "children">`.

PROP	TYPE	DESCRIPTION
label?	<code>React.ReactNode</code>	The field's name in small-medium, sentence case: "Key", "Reveal timeout".
hint?	<code>React.ReactNode</code>	Guidance under the control in caption, text-tertiary. Hidden while there is an error.
error?	<code>React.ReactNode</code>	What went wrong and what to do, in caption, danger, with a circle-alert icon. Replaces the hint and marks the control invalid.
description?	<code>React.ReactNode</code>	A line under the label, for settings rows in the horizontal layout.
required?	<code>boolean</code>	Adds a quiet asterisk after the label and <code>aria-required</code> on the control.
optional?	<code>boolean</code>	Adds "Optional" after the label. Use it instead of <code>required</code> when most fields in a form are required.
counter?	<code>React.ReactNode</code>	A slot at the right of the hint row, for a count such as "42 / 280" or "3 of 24 keys".

PROP	TYPE	DESCRIPTION
layout?	"vertical" "horizontal"	vertical stacks label over control. horizontal puts the label in a 200px column on the left for settings pages, and stacks again when the field is narrower than 640px.
disabled?	boolean	Dims the label and passes disabled to the control.
id?	string	Id for the control. Defaults to the child's id, or a generated one.
children	React.ReactElement	Exactly one control. Field clones it with id, aria-describedby, aria-invalid, aria-required, and for design system components invalid and aria-labelledby.

Input

One line, mono for keys, ring on focus

A single line text control for names, slugs, URLs, keys and values, with its ring, icon and affixes drawn on one wrapper so everything inside reads as one field.

- **Anatomy.** A wrapper on surface with a 1px border-strong ring, shadow-xs and --r-6 corners, at control-sm, control-md or control-lg. Inside, in order: an optional leading icon (16px, 14px at sm) in text-tertiary, an optional prefix flush with the value, the native input in body (small at sm), then one trailing slot (a Spinner, the clear IconButton, or a Kbd hint), then an optional suffix pinned right in text-tertiary with tabular numbers. Placeholder is text-tertiary. mono sets the value, placeholder and affixes in mono.
- **Behaviour.** Hover moves the ring to border-hover. Focus anywhere inside (:focus-within) keeps the border-hover ring and draws a 1.5px focus outline flush against it, and lifts the icon to text-secondary; the kbd hint hides while focused because the shortcut has done its job. invalid turns the ring danger (2px on focus) and sets aria-invalid. clearable shows an x while there is a value; clearing fires a real change event with an empty value and keeps focus in the field. loading shows a Spinner in the trailing slot for async checks. revealable on a password adds a "Reveal value" toggle. readOnly sits on bg-subtle, keeps the text selectable, and with selectOnFocus selects the whole value for copying. disabled drops to fill with a border ring and text-disabled. Clicking the padding or icon focuses the input.
- **Mono.** Use mono for anything a machine reads: keys (STRIPE_SECRET_KEY), values, slugs (lumen-api), token prefixes (smg_live_), paths. Mono turns off spellcheck and autocapitalize. Normalize keys as people type: uppercase, spaces and dashes to underscores.
- **Copy.** Placeholders show a real example in the expected shape ("maya@lumen.dev", "NEW_SECRET_KEY", "Search secrets"), never an instruction that disappears. Put the label in a Field, not the placeholder. Units go in suffix ("MB"), schemes in prefix ("https://").
- **Composition.** Wrap in Field for the label, hint and error; Field wires id, aria-describedby and invalid for you. Use SearchField for search boxes and Textarea for multi line values and raw .env text. Every prop other than className and style lands on the native input, so name, autoComplete, maxLength and handlers work as usual; the ref points at the input.

InputProps

Extends `Omit<React.InputHTMLAttributes<HTMLInputElement>, "size" | "prefix">`.

PROP	TYPE	DESCRIPTION
size?	"sm" "md" "lg"	sm 28px in toolbars and inline edits, md 32px by default, lg 40px on auth and onboarding.
icon?	string	Leading Lucide icon name, drawn in text-tertiary and lifted to text-secondary on focus.
prefix?	React.ReactNode	Text or node before the value, flush with it, for example "https://" or "\$".
suffix?	React.ReactNode	Text or node after the value, pinned to the right, for example "MB" or "12 of 24".

PROP	TYPE	DESCRIPTION
<code>invalid?</code>	<code>boolean</code>	Draws the danger ring and sets <code>aria-invalid</code> . Field sets it for you when it has an error.
<code>mono?</code>	<code>boolean</code>	Sets the value in Geist Mono for keys, values, slugs and tokens. Turns off spellcheck and autocapitalize.
<code>clearable?</code>	<code>boolean</code>	Shows an x button while there is a value. Clearing fires <code>onChange</code> with an empty value and keeps focus.
<code>clearLabel?</code>	<code>string</code>	Accessible name of the clear button. Defaults to "Clear".
<code>onClear?</code>	<code>() => void</code>	Called after the clear button empties the field.
<code>kbd?</code>	<code>string string[]</code>	Shortcut hint shown at the right while the field is empty and unfocused, for example <code>"/</code> .
<code>loading?</code>	<code>boolean</code>	Shows a Spinner at the right, for async checks such as slug availability.
<code>loadingLabel?</code>	<code>string</code>	Accessible name of the loading spinner. Defaults to "Loading".
<code>revealable?</code>	<code>boolean</code>	With <code>type="password"</code> , adds a "Reveal value" toggle that switches the field to plain text.
<code>selectOnFocus?</code>	<code>boolean</code>	Selects the whole value on focus, for read only values people copy (tokens, CLI commands).
<code>readOnly?</code>	<code>boolean</code>	Keeps the value selectable and copyable but not editable, on <code>bg-subtle</code> .
<code>disabled?</code>	<code>boolean</code>	Drops to fill with <code>text-disabled</code> and ignores input. Say why nearby.
<code>value?</code>	<code>string number</code>	The value. Controlled; pair with <code>onChange</code> or <code>onValueChange</code> .
<code>defaultValue?</code>	<code>string number</code>	The initial value when uncontrolled.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the new string on every change, after <code>onChange</code> .
<code>className?</code>	<code>string</code>	Class name for the wrapper that draws the ring.
<code>style?</code>	<code>React.CSSProperties</code>	Inline style for the wrapper, for example a fixed width.

RadioGroup

One choice, as dots or choice cards

One choice from a short list that stays visible, as dots with labels or as bordered choice cards for decisions with consequences, such as how to handle import conflicts.

- **Anatomy.** Each choice is a 16px round dot (`corner-shape: round`) on surface with a `border-hover` ring; the selected dot fills with `primary` and shows a 6px `on-primary` center. Beside it, the label in `body` with an optional 16px icon in `text-secondary`, and an optional description in `small, text-secondary`. `cards` wraps each choice in a `--r-8` card with `space-12` padding and a `border-strong` ring; the selected card takes a 1.5px `primary` ring. Card labels are `body-medium`.
- **Behaviour.** Clicking anywhere on a choice selects it. The group is one tab stop (roving `tabindex`): Tab lands on the selected choice, or the first enabled one; `ArrowDown` and `ArrowRight` move to and select the next choice, `ArrowUp` and `ArrowLeft` the previous, wrapping at the ends and skipping disabled ones; Home and End jump; Space selects the focused choice. The center dot scales in with `ease-spring over duration-fast`; press scales the dot to 0.92. Hover darkens the ring to `text-tertiary` (`cards` to `border-hover`). Focus draws the focus ring around the dot, or around the whole card. `invalid` rings the unselected dots and cards in danger. `disabled` choices sit on `fill` in `text-disabled`.
- **When to use.** 2 to 5 mutually exclusive options where people should see every option at once. Use `cards` when each option needs a sentence explaining its result. Use `Select` for longer lists, `SegmentedControl` for switching views, and `Switch` for on and off.
- **Copy.** Label each option with its outcome and the count: "Overwrite 3 secrets", "Skip conflicts", "Keep both as `_NEW`". Descriptions finish the thought: "Keep the 3 existing values in staging and import the 21 new keys only." Put the question in the `Field` label: "3 keys already exist in staging".
- **Composition.** Wrap in `Field`; it names the group through `aria-labelledby` and passes `invalid`. Pass `name` to submit with a native form.

RadioOption

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string</code>	The value passed to <code>onValueChange</code> . Unique within the group.
<code>label</code>	<code>React.ReactNode</code>	The choice, written as the outcome: "Overwrite 3 secrets", "Skip conflicts".
<code>description?</code>	<code>React.ReactNode</code>	A line under the label in <code>small, text-secondary</code> , saying what happens.
<code>icon?</code>	<code>string</code>	Leading Lucide icon name beside the label.
<code>disabled?</code>	<code>boolean</code>	Not selectable and skipped by the arrow keys.
<code>className?</code>	<code>string</code>	Class name for the option, for example a forced state class in previews.

RadioGroupProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "defaultValue" | "onChange">`.

PROP	TYPE	DESCRIPTION
<code>options</code>	<code>Array<RadioOption string></code>	Options as objects, or plain strings for simple lists.
<code>value?</code>	<code>string null</code>	The selected value, or null for none. Controlled.
<code>defaultValue?</code>	<code>string null</code>	The initially selected value when uncontrolled.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the new value when a choice is made.
<code>variant?</code>	<code>"list" "cards"</code>	<code>list</code> shows dots with labels. <code>cards</code> shows bordered choice cards for consequential choices such as import conflicts.
<code>orientation?</code>	<code>"vertical" "horizontal"</code>	<code>vertical</code> stacks the choices. <code>horizontal</code> puts them in a row, and cards wrap to one column when narrow. Arrow keys work in both directions either way.
<code>name?</code>	<code>string</code>	Adds a hidden input with this name so the value submits with a form.
<code>disabled?</code>	<code>boolean</code>	Disables every choice.
<code>invalid?</code>	<code>boolean</code>	Draws unselected dots and cards with a danger ring. Field sets it for you when it has an error.

SearchField

Slash to focus, Escape to clear, counts as you type

The search box for secrets, projects, members and activity: an Input with a search icon, a "/" shortcut, a clear button, a live result count and a debounced onSearch.

- **Anatomy.** The Input ring at control-sm, control-md or control-lg, a leading 16px search icon in text-tertiary, the query, then one trailing slot: the Kbd hint for the shortcut while empty, the x clear button while there is a query, or a Spinner while loading. With a query and a count, the right edge shows "12 of 24" in small, text-tertiary, tabular numbers.
- **Behaviour.** Pressing the hotkey ("/" by default) anywhere outside an editable field focuses the search and selects its text; the hint hides while focused. Typing calls onValueChange at once, for filtering in place, and onSearch after debounce ms (150 by default), or at once on Enter. Escape clears the query (and stops there, so a surrounding dialog stays open); a second Escape leaves the field. Clearing calls onSearch("") at once. The count is read politely to screen readers as "12 of 24 secrets match". Focus and hover follow Input.
- **Empty results.** Keep the field as typed and show the empty state below it with the query and the scope: "No secrets match 'stripe_live' in staging. Check the spelling or search all environments."
- **Copy.** Placeholders name what is searched and how many: "Search 21 secrets", "Search 5 projects", "Filter keys". Never "Search..." with an ellipsis.
- **Composition.** In toolbars at sm beside SegmentedControl and Select. One hotkey per page: when several fields share it, the first one on the page wins. Pass hotkey={false} for secondary search boxes.

SearchFieldProps

Extends `Omit<React.InputHTMLAttributes<HTMLInputElement>, "size" | "prefix" | "value" | "defaultValue">`.

PROP	TYPE	DESCRIPTION
value?	string	The query. Controlled; pair with onValueChange.
defaultValue?	string	The initial query when uncontrolled.
onValueChange?	(value: string) => void	Called with the query on every keystroke, for filtering in place.
onSearch?	(query: string) => void	Called with the query after typing pauses for debounce ms, at once on Enter, and with "" when cleared.
debounce?	number	Milliseconds to wait before onSearch. Defaults to 150; 0 calls it on every keystroke.
hotkey?	string false	A key that focuses the field from anywhere on the page, shown as a Kbd hint while empty. Defaults to "/". Pass false to turn it off.
count?	number	Number of matches. With a query, shows "12 of 24" (with total) or "12 results" at the right and announces it politely.
total?	number	Number of items searched, for "12 of 24".
noun?	string	Singular noun for the spoken and fallback count: "secret" gives "3 secrets found". Defaults to "result".

PROP	TYPE	DESCRIPTION
<code>resultLabel?</code>	<code>React.ReactNode</code>	Replaces the visible count with any text or node while there is a query.
<code>placeholder?</code>	<code>string</code>	Defaults to "Search". Name what is searched: "Search secrets", "Search 5 projects".
<code>size?</code>	<code>"sm" "md" "lg"</code>	sm 28px in toolbars, md 32px by default, lg 40px.
<code>loading?</code>	<code>boolean</code>	Shows a spinner in place of the clear button while results load.
<code>clearLabel?</code>	<code>string</code>	Accessible name of the clear button. Defaults to "Clear search".
<code>onClear?</code>	<code>() => void</code>	Called after the clear button or Escape empties the field.
<code>className?</code>	<code>string</code>	Class name for the wrapper that draws the ring.
<code>style?</code>	<code>React.CSSProperties</code>	Inline style for the wrapper, for example a fixed width in a toolbar.

SegmentedControl

Views and filters with a sliding thumb

2 to 5 mutually exclusive views or filters in one track, with a thumb that slides to the selected segment: "Table" and "Raw .env", "Matrix" and "Side by side", grid and list.

- **Anatomy.** A `fill` track with 2px padding and `--r-6` corners at `control-sm` or `control-md`. Segments are equal width (as wide as the widest, through a grid of `1fr` columns), each with an optional 16px icon (14px at `sm`), a label in `body-medium` (`small-medium` at `sm`) and an optional count in `text-tertiary` with tabular numbers. The thumb is a `--r-4` block on `surface` with `shadow-sm`; in dark it is `fill-active`. Icon only segments are square.
- **Behaviour.** Click selects. The group is one tab stop on the selected segment; `ArrowLeft` and `ArrowRight` (and `Up` and `Down`) move and select, wrapping and skipping disabled segments; `Home` and `End` jump; `Space` and `Enter` select the focused segment. The thumb slides with `transform` only over `duration-base` and `ease-out`, because every segment has the same width. Unselected labels sit in `text-secondary` and lift to `text` on hover; the selected label is `text`. Press dims the label. Focus draws the focus ring on the segment. Disabled segments are `text-disabled`; a disabled control drops the thumb to `fill-hover`.
- **When to use.** Switching how the same content is shown, or filtering a list with counts. The choice applies at once. Use `Tabs` for navigating between different content, `RadioGroup` when each option needs an explanation, `Select` past 5 options.
- **Copy.** 1 or 2 words per segment in sentence case, nouns for views ("Table", "Raw .env") and filters with their counts ("Missing 3"). Icon only segments need an `aria-label` written as the view: "Grid view", "List view".
- **Composition.** Sits in toolbars at `sm` beside `SearchField` and `Select`. In a `Field`, it takes its name from the label. Use `fullWidth` in narrow settings panels.

SegmentOption

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string</code>	The value passed to <code>onValueChange</code> . Unique within the control.
<code>label?</code>	<code>React.ReactNode</code>	Visible text. Leave it out for an icon only segment and give <code>aria-label</code> instead.
<code>icon?</code>	<code>string</code>	Leading Lucide icon name.
<code>count?</code>	<code>number string</code>	A count after the label in <code>text-tertiary</code> and tabular numbers: "Missing 3".
<code>disabled?</code>	<code>boolean</code>	Not selectable and skipped by the arrow keys.
<code>className?</code>	<code>string</code>	Class name for the segment, for example a forced state class in previews.

SegmentedControlProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "defaultValue" | "onChange">`.

PROP	TYPE	DESCRIPTION
<code>options</code>	<code>Array<SegmentOption string></code>	2 to 5 segments as objects, or plain strings for text only segments.
<code>value?</code>	<code>string</code>	The selected value. Controlled.
<code>defaultValue?</code>	<code>string</code>	The initially selected value when uncontrolled. Defaults to the first enabled option.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the new value on click or arrow keys.
<code>size?</code>	<code>"sm" "md"</code>	sm 28px in toolbars and table headers, md 32px by default.
<code>fullWidth?</code>	<code>boolean</code>	Stretches to the container width, sharing it equally.
<code>disabled?</code>	<code>boolean</code>	Disables every segment.

Select

Listbox in a popover, keyboard first, filters past 8

A trigger styled like Input that opens a listbox in a popover, for choosing one environment, key, template or level from a known list.

- **Anatomy.** The trigger reuses the Input ring (surface, border-strong, shadow-xs, --r-6, control-sm, control-md, control-lg) with an optional leading icon or the selected option's prefix (an EnvDot), the value (or the placeholder in text-tertiary), and a 14px chevrons-up-down in text-tertiary. The menu is a portal on surface-raised with shadow-md, --r-8 corners and space-4 padding; it is at least as wide as the trigger. Options are 32px rows with --r-6 corners: prefix, icon, label (medium when selected), optional description in caption, optional Kbd, and a reserved 16px column for the check so labels never shift. Groups get a caption heading and a border hairline between them. With searchable, a filter row with a search icon sits on top behind a border hairline.
- **Behaviour.** Click, Enter, Space or ArrowDown opens the menu on the selected option; ArrowUp opens on the last. Typing a letter on the closed trigger opens it on the first match (or seeds the filter). In the menu, ArrowUp and ArrowDown move the highlight (fill-hover), skipping disabled options; Home, End, PageUp and PageDown jump; Enter or Space chooses; typing jumps to the next option that starts with the typed letters; Escape closes and returns focus to the trigger; Tab closes and moves on from the trigger. The pointer highlights on move, not on scroll. The menu enters with sg-pop-in over duration-fast and ease-out, leaves over duration-instant with ease-in, flips above the trigger when there is no room, and stacks above any dialog or sheet it opens from. Focus ring and invalid follow Input; the ring only shows for keyboard focus.
- **Filtering.** Past 8 options the filter turns on by itself. It matches label, value, description and group. The empty state names the noun and the query: "No environments match 'prod-eu'".
- **Copy.** Placeholders say what to pick: "Select environment", "Pick an environment template". Environment names are lowercase slugs with their EnvDot. Descriptions give the number or the reason: "24 secrets", "Protected. You will confirm before saving.", "Current environment" on a disabled option.
- **Composition.** Wrap in Field for a label; Field passes id, aria-labelledby and invalid. Use SegmentedControl for 2 to 4 view options that should all stay visible, and RadioGroup when each choice needs a sentence of explanation. Pass name to submit with a native form.

SelectOption

PROP	TYPE	DESCRIPTION
value	string number	The value passed to onChange. Unique within the list.
label?	React.ReactNode	What people read. Defaults to the value.
textValue?	string	Text used for filtering and type to select when label is not a string.
icon?	string	Leading Lucide icon name.
description?	React.ReactNode	A second line in caption, text-tertiary: "24 secrets, protected".
disabled?	boolean	Skipped by the keyboard and not selectable.

PROP	TYPE	DESCRIPTION
group?	string	Group heading. Options with the same group should be adjacent.
prefix?	React.ReactNode	A node before the label, for example <code><EnvDot color="rose" /></code> . Also shown in the trigger when selected.
kbd?	string string[]	Shortcut hint at the right of the option, for example ["g", "s"].

SelectProps

Extends `Omit<React.ButtonHTMLAttributes<HTMLButtonElement>, "value" | "defaultValue" | "onChange" | "type" | "prefix">`.

PROP	TYPE	DESCRIPTION
options	Array<SelectOption string>	Options as objects, or plain strings for simple lists.
value?	string number null	The selected value, or null for none. Controlled.
defaultValue?	string number null	The initially selected value when uncontrolled.
onValueChange?	(value: string number) => void	Called with the new value when an option is chosen.
placeholder?	string	Shown in text-tertiary when nothing is selected: "Select environment".
size?	"sm" "md" "lg"	sm 28px, md 32px by default, lg 40px. Matches Input.
invalid?	boolean	Draws the danger ring. Field sets it for you when it has an error.
disabled?	boolean	Drops the trigger to fill with text-disabled; the menu cannot open.
renderValue?	(option: SelectOption) => React.ReactNode	Renders the selected option in the trigger, for example as an EnvBadge.
searchable?	boolean	Shows a filter input at the top of the menu. Defaults to on when there are more than 8 options.
searchPlaceholder?	string	Placeholder of the filter input. Defaults to "Filter <noun>s".
noun?	string	What the options are, singular or plural, for the filter placeholder and the empty text: "environment" gives "No environments match 'prod-eu'".
emptyText?	React.ReactNode ((query: string) => React.ReactNode)	Replaces the empty text shown when the filter matches nothing.
icon?	string	Leading Lucide icon in the trigger.
mono?	boolean	Sets the value and option labels in Geist Mono, for keys.
name?	string	Adds a hidden input with this name so the value submits with a form.
open?	boolean	Controls whether the menu is open.
defaultOpen?	boolean	Opens the menu on mount when uncontrolled.
onOpenChange?	(open: boolean) => void	Called when the menu opens or closes.
placement?	"bottom-start" "bottom-end" "top-start" "top-end"	Where the menu opens. It flips when there is no room. Defaults to "bottom-start".
menuClassName?	string	Class name for the menu surface.

Switch

Settings that apply the moment they change

An on and off toggle for settings that take effect the moment they change, such as "Auto hide revealed values".

- **Anatomy.** A round track (`--radius-full`, `corner-shape: round`) of 32 by 18 (md) or 26 by 16 (sm) with a 2px inset thumb. Off: border-hover track with a surface thumb with shadow-sm (text-secondary in dark so it reads on the dark track). On: primary track with an on-primary thumb. With `label`, a body label and an optional small description in text-secondary sit beside it and are clickable.
- **Behaviour.** Click, Space or Enter toggles. The thumb slides with ease-spring over duration-base; pressing stretches it to 1.2 times its width from the side it rests on. Hover shifts the track (text-disabled off, primary-hover on). Focus draws the round focus ring. loading shows a spinner in the thumb and ignores clicks while the change saves; update the description to say what is happening ("Saving for 5 members"). disabled drops the track to fill (text-disabled when on).
- **Layout.** `labelPosition="end"` puts the label after the switch, like Checkbox, for inline filters ("Show differences only"). `labelPosition="start"` spans the row with the label on the left and the switch on the right edge, for settings lists.
- **When to use.** Settings that apply immediately with no Save button. Inside a form that saves together, use Checkbox. Never use a switch for an action ("Delete") or for a choice between 2 named options; use SegmentedControl or RadioGroup.
- **Copy.** The label names the setting as a statement that is true when on: "Clear clipboard after copying", "Require two factor for production". The description states the consequence with its number, and may change with the state: "Revealed values hide again after 10 seconds."

SwitchProps

Extends `Omit<React.ButtonHTMLAttributes<HTMLButtonElement>, "value" | "type">`.

PROP	TYPE	DESCRIPTION
<code>checked?</code>	<code>boolean</code>	On or off. Controlled; pair with <code>onCheckedChange</code> .
<code>defaultChecked?</code>	<code>boolean</code>	The initial state when uncontrolled.
<code>onCheckedChange?</code>	<code>(checked: boolean) => void</code>	Called with the new state on click, Space or Enter.
<code>size?</code>	<code>"sm" "md"</code>	sm 26 by 16 in dense rows and filters, md 32 by 18 by default.
<code>disabled?</code>	<code>boolean</code>	Drops the track to fill and ignores clicks. Say why in the description.
<code>loading?</code>	<code>boolean</code>	Shows a spinner in the thumb while the change saves, and ignores clicks.
<code>label?</code>	<code>React.ReactNode</code>	When given, renders a clickable label beside the switch.
<code>description?</code>	<code>React.ReactNode</code>	Secondary line under the label in small, text-secondary.
<code>labelPosition?</code>	<code>"start" "end"</code>	end puts the label after the switch, like Checkbox. start puts the label first and the switch at the right edge, for settings rows.

PROP	TYPE	DESCRIPTION
<code>name?</code>	<code>string</code>	Adds a hidden input with this name while the switch is on, so it submits with a form.
<code>value?</code>	<code>string</code>	Value submitted with name while on. Defaults to "on".

Textarea

Notes and raw .env, grows as you type

A multi line text control for secret notes and raw .env text, sharing the Input ring so forms read as one family.

- **Anatomy.** A wrapper on surface with a 1px border-strong ring, shadow-xs and --r-6 corners. Inside, the native textarea in body (mono in Geist Mono) with space-6 by space-10 padding, and an optional footer with the counter in caption, text-tertiary, tabular numbers.
- **Behaviour.** Hover moves the ring to border-hover; focus keeps the border-hover ring and draws a 1.5px focus outline flush against it; invalid turns it danger. autoGrow fits the height to the content between minRows and maxRows on every change and when the width changes, then scrolls; the resize handle is off while it grows. Without autoGrow people can drag the height. The counter turns warning at 90 percent of the limit and danger past a soft limit. readOnly sits on bg-subtle; disabled drops to fill with text-disabled.
- **Raw .env.** Use mono, autoGrow, and wrap="off" so each line stays one key. Report parse problems in the Field error with the count: "3 lines could not be parsed. Fix the highlighted lines or skip them."
- **Copy.** Placeholders describe what belongs there in the product's voice: "Why this value exists and who owns it". The counter reads "142 / 280".
- **Composition.** Wrap in Field for the label, hint and error. Every prop other than className and style lands on the native textarea; the ref points at it.

TextareaProps

Extends `Omit<React.TextareaHTMLAttributes<HTMLTextAreaElement>, "rows">`.

PROP	TYPE	DESCRIPTION
autoGrow?	boolean	Grows with its content between minRows and maxRows, then scrolls. Turns off the resize handle.
minRows?	number	Visible rows when empty. Defaults to 3.
maxRows?	number	Rows before an auto growing textarea starts to scroll. Defaults to 12.
rows?	number	Fixed row count when autoGrow is off. Defaults to minRows.
mono?	boolean	Sets the text in Geist Mono, for raw .env text and values. Turns off spellcheck and autocapitalize.
invalid?	boolean	Draws the danger ring and sets aria-invalid. Field sets it for you when it has an error.
counter?	boolean number	true shows "142 / 280" against maxLength in the bottom right corner. A number sets a soft limit that turns the count danger when passed without blocking typing.
maxLength?	number	Hard limit enforced by the browser; also the counter's limit when counter is true.
readOnly?	boolean	Keeps the text selectable but not editable, on bg-subtle, with no resize handle.

PROP	TYPE	DESCRIPTION
<code>disabled?</code>	<code>boolean</code>	Drops to fill with text-disabled and ignores input.
<code>value?</code>	<code>string</code>	The text. Controlled; pair with <code>onChange</code> or <code>onValueChange</code> .
<code>defaultValue?</code>	<code>string</code>	The initial text when uncontrolled.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the new string on every change, after <code>onChange</code> .
<code>className?</code>	<code>string</code>	Class name for the wrapper that draws the ring.
<code>style?</code>	<code>React.CSSProperties</code>	Inline style for the wrapper.

Breadcrumbs

Workspace, project, environment; thin slashes

The trail at the start of the panel header that says where you are: workspace, project, environment. Each level can carry a switcher, Vercel style.

- **Anatomy.** Crumbs in body-medium (mono-medium for slugs with mono), 28px tall with --r-6 corners, an optional 16px icon or node (a 16px AppIcon for the workspace) or an 8px EnvDot for an environment. Crumbs are separated by a thin "/" in text-tertiary at weight 300, never chevrons. A crumb with switcher gets a chevrons-up-down IconButton at xs right after it.
- **States.** Earlier crumbs rest in text-secondary and lift to text on hover; press fills with fill; focus draws the focus ring. The last crumb is the current page: text, not a link, aria-current="page".
- **Behaviour.** Crumbs with href are links, crumbs with only onClick are buttons. The switcher opens a menu of siblings (projects, environments); pass a node to switcher to attach a Menu with its own trigger. Long labels truncate with an ellipsis at 240px and show the full name as a tooltip; the current page shrinks last.
- **Copy.** The workspace name as written ("Lumen Labs"), then exact slugs (lumen-api, staging). Switchers are labelled with the action: "Switch project", "Switch environment".
- **Composition.** Put Breadcrumbs first in the AppShell header, with page actions pushed to the right. Use at most three levels; deeper places belong in Tabs.

BreadcrumbItem

PROP	TYPE	DESCRIPTION
label	React.ReactNode	Visible text: the workspace name, or an exact slug such as lumen-api and staging.
icon?	string React.ReactNode	Leading Lucide icon name, or a node such as a 16px AppIcon for the workspace.
dot?	"gray" "blue" "teal" "green" "amber" "orange" "rose" "violet"	An environment hue for a leading EnvDot, for environment crumbs.
href?	string	Renders the crumb as a link. Ignored on the last crumb, which is the current page.
onClick?	(event: React.MouseEvent<HTMLElement>) => void	Renders the crumb as a button when there is no href. Ignored on the last crumb.
mono?	boolean	Sets the label in mono-medium, for project and environment slugs.
switcher?	boolean React.ReactNode	true adds a chevrons-up-down IconButton at xs beside the crumb (Vercel style) that calls onSwitch; a node renders as is, for example a Menu with its own trigger.
switcherLabel?	string	Accessible name of the default switcher, for example "Switch project".

PROP	TYPE	DESCRIPTION
onSwitch?	(event: React.MouseEvent<HTMLButtonElement>) => void	Called when the default switcher is pressed.
key?	string	Stable key when labels repeat.
className?	string	Extra class on the crumb, for forced preview states.

BreadcrumbsProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
items	BreadcrumbItem[]	Crumbs from the widest place to the current page, usually workspace, project, environment.
label?	string	Accessible name of the navigation landmark.

Sidebar

Workspace, destinations, projects and their environments

The navigation column on the background: the workspace switcher, the destinations, the projects with their environments, and the signed-in member.

- **Anatomy.** Sidebar stacks three slots: header (pinned), the scrolling nav list, and footer (pinned). `SidebarHeader` is `--header` tall with an 8px top inset, so the workspace row centres on the panel's header row; it holds a `WorkspaceButton` (20px `AppIcon`, the name in `body-medium`, chevrons-up-down in `text-tertiary`) and up to two `sm` `IconButton`s. `SidebarSection` starts 16px below the item above it with an overline label in `text-tertiary`. `NavItem` is 30px tall with `--r-6` corners, a 16px icon or an 8px `EnvDot`, the label in `small-medium`, then a shortcut, a trailing node and a tabular count in `caption text-tertiary`. `SidebarUser` shows initials on a round `env-blue-soft` tile with `env-blue-text`, the name and one detail line.
- **States.** Labels rest in `text-secondary` with icons in `text-tertiary`. Hover fills with `fill` and lifts the label to `text`. The current page (active) fills with `fill-hover`, sets text on label and icon, and carries `aria-current="page"`. Focus draws the focus ring. Disabled drops to `text-disabled` and ignores clicks.
- **Nesting.** A `NavItem` with children is a disclosure: a 12px chevron after the label turns 90 degrees when open, and the children sit 20px in beside a 1px border guide centred on the parent's icon. Put a project's environments under it, each with its dot, its secret count, and a 12px lock as trailing when protected.
- **Behaviour.** Arrow Up and Down move between visible items and section labels, Home and End jump to the ends. On a nested row, Arrow Right opens, Arrow Left closes, and Arrow Left on a child returns to its parent. `kbd` hints appear on hover and focus only. A section's action (a "plus" `IconButton` at `xs`) appears on hover and focus, and always on touch screens. Clicking a section label folds it; the chevron shows while it is closed or hovered.
- **Rail.** Collapsed (from `AppShell sidebarCollapsed` or the `collapsed` prop) the sidebar draws a 56px rail: icons centred in 32px rows, labels become the accessible name and a native tooltip, sections become a hairline, nested items hide, and a warning, danger or accent count becomes a 6px dot on the icon.
- **Copy.** Destinations are nouns in sentence case: "Projects", "Activity", "Health", "Tokens", "Settings". Projects and environments use their exact slugs: `lumen-api`, `staging`. Counts are digits only. Actions are verbs with objects: "Create project", "Add secret", "Help and shortcuts".
- **Composition.** Pass the whole Sidebar to `AppShell sidebar`. Wrap `WorkspaceButton` and `SidebarUser` in a `Menu` for the workspace switcher and the account menu. Use `countTone="warning"` only for things that need attention, such as "Health 3".

SidebarProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>header?</code>	<code>React.ReactNode</code>	Pinned above the scrolling list, usually a <code>SidebarHeader</code> with the <code>WorkspaceButton</code> .
<code>footer?</code>	<code>React.ReactNode</code>	Pinned below the scrolling list, usually a <code>SidebarUser</code> and a help <code>IconButton</code> .

PROP	TYPE	DESCRIPTION
children?	React.ReactNode	NavItems and SidebarSections. Arrow Up and Down move between visible items, Home and End jump to the ends.
collapsed?	boolean	Draws the 56px icon rail. Defaults to the AppShell state, so leave it unset inside a shell.
label?	string	Accessible name of the navigation landmark.

SidebarHeaderProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
children?	React.ReactNode	Usually a <code>WorkspaceButton</code> .
actions?	React.ReactNode	IconButtons at sm, for example "Search" and "Add secret". Hidden in the rail.

WorkspaceButtonProps

Extends `React.ButtonHTMLAttributes<HTMLButtonElement>`.

PROP	TYPE	DESCRIPTION
name	string	Workspace name as the members typed it: "Lumen Labs".
tile?	React.ReactNode	Replaces the default 20px <code>AppIcon</code> , for a workspace that uploaded its own tile.

SidebarSectionProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onToggle">`.

PROP	TYPE	DESCRIPTION
label	React.ReactNode	Overline label, for example "Projects".
children?	React.ReactNode	Items inside the section.
action?	React.ReactNode	A node at the right of the label, shown on hover and focus, typically an <code>IconButton</code> "plus" at xs labelled "Create project".
collapsible?	boolean	When false the label is static and the items always show.
open?	boolean	Controlled open state.
defaultOpen?	boolean	Initial open state when uncontrolled.
onOpenChange?	(open: boolean) => void	Called with the next open state when the label is clicked.

NavItemProps

Extends `Omit<React.HTMLAttributes<HTMLElement>, "children">`.

PROP	TYPE	DESCRIPTION
icon?	string	Leading Lucide icon name at 16px.
dot?	"gray" "blue" "teal" "green" "amber" "orange" "rose" "violet"	An environment hue for a leading <code>EnvDot</code> instead of an icon: "blue", "amber", "rose", "violet" and so on.

PROP	TYPE	DESCRIPTION
label	<code>React.ReactNode</code>	The destination, sentence case, or the exact slug for projects and environments.
count?	<code>number string</code>	A number after the label in tabular text-tertiary, for example secrets in an environment.
countTone?	<code>"neutral" "warning" "danger" "accent"</code>	warning, danger or accent draw the count as a soft chip, and as a 6px dot in the rail. Use warning for "Health 3".
kbd?	<code>string string[]</code>	Shortcut shown on hover and focus, for example ["mod", "k"].
active?	<code>boolean</code>	The current page: fill-hover ground, text label, <code>aria-current="page"</code> .
children?	<code>React.ReactNode</code>	Nested NavItems. The row becomes a disclosure: click, Enter or Arrow Right opens it, Arrow Left closes it or returns to the parent.
open?	<code>boolean</code>	Controlled open state of the nested items.
defaultOpen?	<code>boolean</code>	Initial open state of the nested items when uncontrolled.
onOpenChange?	<code>(open: boolean) => void</code>	Called with the next open state.
trailing?	<code>React.ReactNode</code>	A node before the count, for example a 12px lock icon on a protected environment, so counts stay in one column. Hidden in the rail.
href?	<code>string</code>	Renders an anchor instead of a button. Ignored when the item has children.
onClick?	<code>(event: React.MouseEvent<HTML<Element>)<=> void</code>	Called on click. Inside an AppShell drawer, clicking a leaf item also closes the drawer.
disabled?	<code>boolean</code>	Greys the item to text-disabled, ignores clicks and skips it in arrow key movement.

SidebarUserProps

Extends `React.ButtonHTMLAttributes<HTMLButtonElement>`.

PROP	TYPE	DESCRIPTION
name	<code>string</code>	Full name: "Maya Chen". Initials are derived from it.
detail?	<code>React.ReactNode</code>	Second line in caption text-tertiary: the email or the role.
avatar?	<code>React.ReactNode</code>	Replaces the initials tile, for example an Avatar at 24px.

Tabs

Underline for views, pills for filters

A row of tabs that switches views in place: underline for the views of a page (environments, project sections), pills for filters inside a view.

- **Anatomy.** A tab is a 40px hit area (36px at sm) holding a 28px inner pill (24px at sm) with `--r-6` corners: an optional 16px icon or 8px `EnvDot`, the label in `small-medium`, and an optional tabular count in `caption text-tertiary`. underline draws a 1px border hairline under the row and a 2px text indicator under the selected pill. pills fills the selected pill with `fill`.
- **States.** Labels rest in `text-secondary`. Hover lifts the label to `text` and, in underline, fills the pill with `fill`. Press fills with `fill-hover`. The selected tab uses `text` and its count moves to `text-secondary`. Focus draws the focus ring around the pill. Disabled uses `text-disabled` and is skipped by the keyboard.
- **Behaviour.** The indicator slides to the new tab with `ease-out` over `duration-base` (transform only, so it never reflows). Arrow Left and Right move focus and select at once (automatic activation), wrapping at the ends; Home and End jump to the first and last. Only the selected tab is in the Tab order. When the row is wider than its container it scrolls sideways without a scrollbar, fades 24px at the clipped edge, and keeps the selected tab in view.
- **Copy.** Views are nouns in sentence case: "Overview", "Secrets", "Compare", "Activity", "Settings". Environments use their slug and their color dot: `staging`. Filters state the condition: "Missing elsewhere", "Recently changed", "Rotation due". Counts are digits only.
- **Composition.** Put page tabs in the `PageHeader` tabs slot, which runs the hairline edge to edge. Put `TabPanels` inside `Tabs` to link them with `aria-controls`; only the selected panel mounts. `CodeBlock` uses `sm` underline tabs for install and export snippets. Use `SegmentedControl` for settings with two or three exclusive values, not tabs.

TabItem

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string</code>	Unique value reported by <code>onValueChange</code> .
<code>label</code>	<code>React.ReactNode</code>	Visible label: a sentence case noun ("Secrets") or an exact slug ("staging").
<code>icon?</code>	<code>string</code>	Leading Lucide icon name.
<code>count?</code>	<code>number string</code>	Tabular count after the label in <code>text-tertiary</code> , for example secrets in an environment.
<code>dot?</code>	<code>"gray" "blue" "teal" "green" "amber" "orange" "rose" "violet"</code>	An environment hue for a leading <code>EnvDot</code> ; takes the place of icon.
<code>disabled?</code>	<code>boolean</code>	Greys the tab and skips it with the arrow keys.
<code>className?</code>	<code>string</code>	Extra class on the tab, for forced preview states such as <code>is-hover</code> .

TabsProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "defaultValue">`.

PROP	TYPE	DESCRIPTION
<code>items</code>	<code>TabItem[]</code>	The tabs, in order.
<code>value?</code>	<code>string</code>	Controlled selected value.
<code>defaultValue?</code>	<code>string</code>	Initial value when uncontrolled. Defaults to the first enabled item.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the next value on click and on arrow keys (automatic activation).
<code>variant?</code>	<code>"underline" "pills"</code>	<code>underline</code> for page level views, with a 2px text indicator that slides to the selection and a border hairline under the row. <code>pills</code> for filters inside a view, with a fill ground behind the selection.
<code>size?</code>	<code>"sm" "md"</code>	<code>md</code> 40px row with small-medium labels; <code>sm</code> 36px row for card and code block headers.
<code>label?</code>	<code>string</code>	Accessible name of the tab list, for example "Environments".
<code>children?</code>	<code>React.ReactNode</code>	Optional <code>TabPanels</code> . When present, each tab points at its panel with <code>aria-controls</code> .

TabPanelProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string</code>	The value of the tab this panel belongs to.
<code>children?</code>	<code>React.ReactNode</code>	Panel content, mounted only while its tab is selected.

AppShell

Sidebar on the ground, the page on an inset panel

The frame of every signed-in screen: the Sidebar on the bg ground and the page on an inset surface panel with its own header row. Only the panel scrolls.

- **Anatomy.** A two column grid: the sidebar at `--sidebar` (240px) and the panel. The panel is inset 8px from the top, right and bottom, with `--r-12` corners and `shadow-sm`, so it reads as a sheet of paper resting on the ground (Linear's inset panel). Inside it, a `--header` (48px) row with a 1px border hairline under it holds the sidebar toggle, a 16px divider in `border-strong`, and your header content (usually Breadcrumbs, with actions pushed right). Below it the page scrolls on its own; the sidebar and the header never move.
- **Collapsed.** On wide screens the header starts with a `panel-left IconButton` ("Collapse sidebar", "Expand sidebar") that switches the sidebar to a 56px icon rail. Control it with `sidebarCollapsed`, or leave it uncontrolled with `defaultSidebarCollapsed`. Hide the toggle with `collapsible={false}`.
- **Small screens.** Under 768px of its own width the panel runs full bleed with no inset, radius or shadow, and the toggle becomes a menu `IconButton` ("Open navigation"). It opens the sidebar as a drawer from the left: bg ground, `--r-12` on the open edge, `shadow-lg`, over an `overlay scrim`. It slides in with `ease-out` over `duration-base` and out faster with `ease-in`. While open it is a modal dialog: focus moves into it and is trapped, Escape or a click on the scrim closes it and returns focus to the menu button, and choosing a destination closes it.
- **Behaviour.** The shell measures itself, not the window, so it also works inside previews and split views. It fills `100dvh` by default; pass a `style height` when it lives in a smaller box. The sidebar reads the collapsed and drawer state from the shell, so a `Sidebar` inside needs no props for it.
- **Copy.** The toggle labels state the action: "Collapse sidebar", "Expand sidebar", "Open navigation".
- **Composition.** `sidebar` is a `Sidebar`. `header` is `Breadcrumbs` (workspace, project, environment) plus at most two quiet actions. `children` starts with a `PageHeader`, then the page content with 32px side padding (16px under 768px).

AppShellProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>sidebar?</code>	<code>React.ReactNode</code>	The navigation column, usually a <code>Sidebar</code> . It reads the shell's collapsed and drawer state from context.
<code>header?</code>	<code>React.ReactNode</code>	Content of the <code>--header</code> row at the top of the panel, usually <code>Breadcrumbs</code> on the left and a few actions pushed right. The shell adds the sidebar toggle before it.
<code>children?</code>	<code>React.ReactNode</code>	The page. It scrolls inside the panel; give it its own padding (<code>PageHeader</code> brings its own).
<code>inset?</code>	<code>boolean</code>	Draws the panel inset 8px from the top, right and bottom with <code>--r-12</code> corners and <code>shadow-sm</code> . False runs it full bleed with a hairline on its left. Always full bleed below 768px.

PROP	TYPE	DESCRIPTION
sidebarCollapsed?	boolean	Controlled rail state on wide screens: true draws the 56px icon rail.
defaultSidebarCollapsed?	boolean	Initial rail state when uncontrolled.
onSidebarCollapsedChange?	(collapsed: boolean) => void	Called with the next rail state when the header toggle is pressed.
collapsible?	boolean	Shows the "Collapse sidebar" toggle at the start of the header on wide screens.
drawerOpen?	boolean	Controlled drawer state below 768px.
defaultDrawerOpen?	boolean	Initial drawer state when uncontrolled.
onDrawerOpenChange?	(open: boolean) => void	Called when the drawer opens (menu button) or closes (Escape, the scrim, or choosing a nav item).
sidebarLabel?	string	Accessible name of the sidebar region and of the drawer dialog.

Card

A hairline ring, flat until you can click it

A flat surface with a 1px border ring and --r-8 corners that groups one thing: a project, a health summary, a setting group, a snippet.

- **Anatomy.** Optional header (title in heading, one line of description in small text-secondary, actions top right), a body, and an optional footer: a strip on bg-subtle under a hairline, in small text-secondary, for recency and status ("Updated 4 min ago", "3 issues"). Padding is 16px (md), 12px (sm) or 24px (lg). none runs the body edge to edge for a Table, SettingRows or list rows; the header keeps 16px and gains a hairline under it.
- **Elevation.** None. Cards never carry a shadow at rest; hierarchy comes from the ring and the ground.
- **Interactive.** With interactive and href or onClick, the title becomes a link stretched over the whole card: hover draws a border-hover ring, shadow-sm and a 1px lift over duration-fast with ease-out; press settles back; keyboard focus draws the focus ring around the card. Buttons in actions, the footer and the body stay separately clickable above the link.
- **Copy.** Titles are the thing's name or slug (lumen-api, "Health"). Descriptions state facts with numbers ("3 things need attention in lumen-api."). Footers hold time and status, not calls to action.
- **Composition.** Grid project cards at a 260px minimum with 12px gaps. Put EnvBadges (outline, sm, with counts) in project cards. SettingsGroup wraps its rows in a padding="none" card. Do not nest cards; use hairlines inside instead.

CardProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "title" | "onClick">`.

PROP	TYPE	DESCRIPTION
title?	React.ReactNode	Heading in heading (14px, 600). On an interactive card the title is the link, stretched over the whole card.
description?	React.ReactNode	One line under the title in small text-secondary.
actions?	React.ReactNode	Buttons or icon buttons at the top right. They stay clickable on an interactive card.
footer?	React.ReactNode	A strip under a hairline on the bg-subtle ground, for a status line or secondary actions.
padding?	"none" "sm" "md" "lg"	Inner padding: sm 12px, md 16px, lg 24px, none for flush content such as a Table or SettingRows (the header keeps 16px and gains a hairline).
interactive?	boolean	Hover draws a border-hover ring, shadow-sm and a 1px lift. With href or onClick the whole card is one target with a focus ring.
href?	string	Makes an interactive card a link.
onClick?	(event: React.MouseEvent<HTMLElement>) => void	Makes an interactive card a button when there is no href.

PROP	TYPE	DESCRIPTION
<code>label?</code>	<code>string</code>	Accessible name for the card target when the title alone is not enough, or when there is no title.
<code>titleAs?</code>	<code>"h2" "h3" "h4" "p"</code>	Heading element for the title. h3 by default.
<code>children?</code>	<code>React.ReactNode</code>	Body content, padded by padding.

CodeBlock

Install, run and .env snippets with one copy

A read only snippet for the CLI, a .env file or an export, with light highlighting from the code-* tokens and a copy button that swaps to a check.

- **Anatomy.** A `bg-subtle` block with a border hairline and `--r-8` corners, set in code (13/22 Geist Mono) with 12px vertical and 16px horizontal padding. An optional 36px header on a hairline holds either a title (a `file-text` icon and the file name in `mono-sm`) or small underline Tabs (brew, npm, curl), with the copy `IconButton` at xs on the right. Without a header the copy button floats top right and appears on hover and focus (always on touch).
- **Highlighting.** Quiet and token based, never a rainbow. `shell`: a \$ prompt in `text-tertiary` that cannot be selected, the command in `text` at weight 500, arguments in `text`, flags in `text-secondary`, strings in `code-value`, `${REFS}` in `code-ref`, output lines in `text-secondary`. `dotenv`: keys in `code-key`, = and quotes in `code-punct`, values in `code-value`, references in `code-ref`, comments in *italic code-comment*. `json` and `yaml`: keys, punctuation and values the same way.
- **Behaviour.** Copy writes the snippet with `copyText`, swaps the icon to a success check labelled "Copied" for 1600ms and announces it to screen readers. In a shell snippet it copies the commands only: no prompts, no output. Tabs switch instantly and keep the copy button. `lineNumbers` adds a non selectable gutter; `highlightLines` marks lines with `fill-hover` and a 2px `text-tertiary` bar. Long lines scroll sideways unless `wrap` is set.
- **Copy.** Real, runnable commands with the real names: `secmgr run --env staging -- npm run dev`, then the output "Injected 24 secrets from lumen-api/staging". Mask secret values with an ellipsis (`sk_live_51N...`), never print a full live key.
- **Composition.** Put install tabs in onboarding and the project overview, a single command in an `EmptyState` hint, and an .env or export under the export sheet. Pair a copy with a toast: "Copied install command".

CodeTab

PROP	TYPE	DESCRIPTION
<code>label</code>	<code>string</code>	Tab label, usually the tool: "brew", "npm", "curl", or a format: ".env", "JSON", "YAML".
<code>code</code>	<code>string</code>	The code shown under this tab.
<code>language?</code>	<code>"shell" "dotenv" "json" "yaml" "text"</code>	Overrides the block's language for this tab.

CodeBlockProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>code?</code>	<code>string</code>	The code to show. Ignored when tabs is given.
<code>tabs?</code>	<code>CodeTab[]</code>	Alternative snippets under small underline tabs in the header, for install and export options.

PROP	TYPE	DESCRIPTION
language?	"shell" "dotenv" "json" "yaml" "text"	shell: a non selectable \$ prompt, the command in text, flags in text-secondary, strings in code-value, output lines in text-secondary. dotenv: keys, = in code-punct, values in code-value, # comments in italic, \${REFERENCES} in code-ref. json and yaml: keys, punctuation and values.
title?	React.ReactNode	File name in the hairline header, in mono-sm: ".env.production".
copy?	boolean	Shows the copy button: in the header when there is one, otherwise top right on hover. It copies only the commands of a shell snippet, never the prompts or the output.
prompt?	boolean	In shell, draws a \$ before each command. When the code already starts lines with "\$ ", only those lines are commands and the rest are output.
lineNumbers?	boolean	Shows 1 based line numbers in a non selectable gutter.
highlightLines?	number[]	1 based line numbers to mark with a fill-hover ground and a 2px bar.
wrap?	boolean	Wraps long lines instead of scrolling sideways.
tab?	string	Controlled selected tab label.
defaultTab?	string	Initial tab label when uncontrolled.
onTabChange?	(label: string) => void	Called with the next tab label.
maxHeight?	number string	Maximum height before the code scrolls.
onCopy?	(text: string) => void	Called with the copied text after a successful copy, for a toast such as "Copied install command".

PageHeader

Title, context, actions and tabs on one hairline

The top of every page inside the panel: what this is, the numbers that matter, what you can do, and the views of it.

- **Anatomy.** An optional leading 32px tile (bg-subtle, border hairline, --r-8, a 16px icon in text-secondary) or any node. The title in title-lg with meta badges beside it. One line of description in small text-secondary, capped at --prose. actions on the right, top aligned with the title. tabs sit under the header on a border hairline that runs the full width; the first tab's label lines up with the title.
- **Spacing.** 24px above, 32px at the sides (16px under 640px), 12px between the header and its tabs. flush removes the side padding when the column is already padded.
- **Behaviour.** Static. Under 640px of its own width (a container query, so it responds to the panel, not the window) the actions wrap below the title and the side padding drops to 16px. Long titles wrap, never truncate.
- **Copy.** Titles are nouns or exact slugs: "Access tokens", lumen-api, "Secrets". The description states counts and recency: "24 secrets in lumen-api/staging. Updated 4 min ago by Priya Raman." Actions are verbs with objects: "Add secret", "Import .env", "Create token".
- **Composition.** Put it first inside AppShell children. One primary action, placed last. Use EnvBadge in meta for the environment in view, and Badge for status ("Rotation due"). Tabs are Tabs with variant="underline".

PageHeaderProps

Extends `Omit<React.HTMLAttributes<HTMLElement>, "title">`.

PROP	TYPE	DESCRIPTION
title	<code>React.ReactNode</code>	Page title in title-lg: a noun ("Access tokens") or the exact slug of the thing (lumen-api).
description?	<code>React.ReactNode</code>	One line in small text-secondary that says what is here or states the numbers.
icon?	<code>string React.ReactNode</code>	A Lucide icon name drawn in a 32px bg-subtle tile with a hairline, or any leading node (an AppIcon, an Avatar).
meta?	<code>React.ReactNode</code>	Badges beside the title: EnvBadge, Badge ("Protected", "Rotation due").
actions?	<code>React.ReactNode</code>	Buttons at the right, primary last. They move under the title when the header is narrower than 640px.
tabs?	<code>React.ReactNode</code>	A Tabs with variant="underline", drawn under the header with its hairline across the full width.
flush?	<code>boolean</code>	Drops the 32px side padding when the header sits inside an already padded column.
titleAs?	<code>"h1" "h2" "h3"</code>	Heading element for the title. h1 by default.

ProjectCard

A project at a glance: environments, health, the latest change

A project in the projects list, as a card in the grid or a dense row in the list: its name, linked repo, environments with secret counts, health and the latest change. The whole card opens the project.

- **Anatomy, grid.** surface with a border hairline and `--r-8` corners, 16px padding. The name in heading; the repo under it with a 14px `folder-git-2` in `text-tertiary` and `lumen-labs/lumen-api` in `mono-sm text-secondary`. Environments as `EnvBadge dot sm` with their counts, wrapping. A border hairline, then the health line in `small-medium` ("3 issues" in `warning` with `triangle-alert`, or "Healthy" in `success` with `circle-check`) and the latest change in `small text-secondary` with its time in `text-tertiary` after a middle dot. The star and the menu slot sit top right.
- **Anatomy, row.** One 56px line on a transparent ground with `--r-6` corners: name and repo, environments as 6px `EnvDots` with counts (the name is in the tooltip), health, the latest change, then star and menu. Columns keep fixed tracks so a list of rows aligns. Below 640px of card width the row stacks like a small card.
- **Behaviour.** The name is the link (or a button with `onOpen`); its hit area stretches over the whole card, while the star and the menu stay separate targets above it. Grid cards lift 1px with a `border-hover` ring and `shadow-sm` on hover and settle on press; rows take `bg-subtle` on hover. Keyboard focus on the link draws the focus ring around the whole card. The star appears on hover and focus, stays visible when starred, and fills with `text` (pressed). On touch the star is always visible.
- **States.** Default, hover, focus, starred, custom health (for example a danger Badge "Live key in staging"), long names truncate with an ellipsis, loading skeleton for both variants.
- **Copy.** "3 issues", "Healthy", "Priya changed 3 secrets in staging", "4 min ago", "Star lumen-api", "Unstar lumen-api", "More actions for lumen-api". Keys and tokens inside the activity sentence go in `code`, which renders in `mono-sm`.
- **Composition.** Grid of cards at `minmax(280px, 1fr)`; rows inside a list with a border hairline. Put a Menu with Rename, Compare environments and Delete project in menu.

ProjectCardEnv

Extends `Env`.

PROP	TYPE	DESCRIPTION
<code>count?</code>	<code>number</code>	Number of secrets in this environment.

ProjectCardProps

Extends `React.HTMLAttributes<HTMLElement>`.

PROP	TYPE	DESCRIPTION
<code>variant?</code>	<code>"grid" "row"</code>	<code>grid</code> is a card in the projects grid; <code>row</code> is a dense line in the list view.
<code>name</code>	<code>string</code>	Project slug, for example <code>lumen-api</code> .

PROP	TYPE	DESCRIPTION
<code>repo?</code>	<code>string</code>	Linked repository as owner/name, for example <code>lumen-labs/lumen-api</code> .
<code>envs?</code>	<code>ProjectCardEnv[]</code>	Environments in their project order, each with a secret count.
<code>issues?</code>	<code>number</code>	Open health issues. <code>0</code> shows "Healthy"; more shows "2 issues" in warning.
<code>health?</code>	<code>React.ReactNode</code>	Replaces the health line, for example a danger "Live key in staging".
<code>activity?</code>	<code>React.ReactNode</code>	The latest change as a sentence: "Priya changed 3 secrets in staging".
<code>time?</code>	<code>string</code>	When activity happened, relative: "4 min ago".
<code>starred?</code>	<code>boolean</code>	Controlled star state.
<code>defaultStarred?</code>	<code>boolean</code>	Initial star state when uncontrolled.
<code>onStarredChange?</code>	<code>(starred: boolean) => void</code>	
<code>menu?</code>	<code>React.ReactNode</code>	Slot after the star, usually a Menu trigger with Rename, Archive and Delete.
<code>href?</code>	<code>string</code>	Link to the project; the whole card becomes the link.
<code>onOpen?</code>	<code>(event: React.MouseEvent) => void</code>	Called when the card is activated, when there is no href.
<code>loading?</code>	<code>boolean</code>	Shows a skeleton of the card while projects load.

SettingRow

Label and consequence left, control right

One setting on one row: its name and consequence on the left, its control on the right.

SettingsGroup stacks rows inside a flush Card under a title.

- **Anatomy.** SettingRow: label in body-medium text, description in small text-secondary (inline code in mono-sm), control right aligned, 16px padding, 24px between text and control. Rows are separated by a 1px border hairline. SettingsGroup: title in heading, optional description in small text-secondary, 12px above a Card with padding="none".
- **Danger.** danger on a row sets its label in danger; danger on a group tints the title and the card ring toward danger. Keep the button itself secondary: the destructive danger button belongs to the confirmation dialog that follows ("Delete lumen-api? Type lumen-api to confirm.").
- **Behaviour.** Static layout. When the row is narrower than 520px (a container query, so it responds to the column, not the window) the control moves under the text and aligns left. Settings that apply immediately use a Switch or a segmented control; settings saved with a form use a Checkbox.
- **Copy.** Labels name the setting in sentence case: "Theme", "Density", "Hide revealed values after". Descriptions state the effect and the number: "A revealed value masks itself again after this long." Danger descriptions restate the object and the consequence: "Permanently deletes lumen-api, its 4 environments and 89 secrets. Services reading them will fail on their next deploy."
- **Composition.** Settings pages stack groups 32px apart in a column no wider than --dialog-lg. The danger zone is always the last group.

SettingRowProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>label</code>	<code>React.ReactNode</code>	The setting's name in body-medium, sentence case: "Theme", "Hide revealed values after".
<code>description?</code>	<code>React.ReactNode</code>	What changes, in small text-secondary, with the number that matters.
<code>children?</code>	<code>React.ReactNode</code>	The control: a SegmentedControl, Select, Switch or Buttons.
<code>danger?</code>	<code>boolean</code>	Sets the label in danger for irreversible actions such as "Delete project".
<code>htmlFor?</code>	<code>string</code>	Id of the control, which turns the label into a <code><label></code> for it.

SettingsGroupProps

Extends `Omit<React.HTMLAttributes<HTMLElement>, "title">`.

PROP	TYPE	DESCRIPTION
<code>title?</code>	<code>React.ReactNode</code>	Group title in heading: "Appearance", "Danger zone".
<code>description?</code>	<code>React.ReactNode</code>	One line under the title in small text-secondary.
<code>children?</code>	<code>React.ReactNode</code>	SettingRows.

PROP	TYPE	DESCRIPTION
danger?	boolean	Tints the title and the card ring with danger, for the danger zone.

StatTile

One number, its change, and a quiet trend

One headline number with its label, the change that matters, and an optional quiet trend line. Four of them open the project overview: "Secrets 89", "Environments 4", "Changes this week 17", "Issues 3".

- **Anatomy.** A surface tile with a border hairline and `--r-8` corners, 12px by 16px padding. The label in `small text-secondary` with an optional 14px icon in `text-tertiary`; the value in `title-lg` with tabular figures; the delta in `caption`. An optional 96 by 32 sparkline sits bottom right: the area in `fill`, a 1.5px `text-tertiary` line with round joins, and a 7px `text` dot with a 2px surface ring on the last point.
- **Delta.** Name the period and sign the change: "+3 this week", "+5 on last week", "1 new today". Tone follows meaning, not direction: success when the change is good ("2 fewer than last month" tokens), warning or danger when it needs attention, `text-tertiary` otherwise. Long deltas truncate.
- **Sparkline.** Twelve points, oldest first, one series, no axes. Hovering reads the nearest point in place of the delta ("Sep 26 · 5 changes") and moves the dot there with a `border-strong` crosshair. The SVG carries a summary for screen readers ("Sep 19 to Sep 30: from 1 to 2 changes").
- **Interactive.** With `href` or `onClick` the whole tile is the link or button: hover lifts 1px with a `border-hover` ring and `shadow-sm` and shows an `arrow-up-right` hint; focus draws the focus ring. Use it for tiles that lead somewhere, such as Issues to the Health page.
- **States.** Default, interactive hover and focus, no delta, no sparkline, loading skeleton.
- **Composition.** A row of 2 to 4 tiles at `minmax(200px, 1fr)` with 12px gaps above the overview's environments and insights. Keep one idea per tile; do not put two numbers in one value.

StatTileProps

Extends `Omit<React.HTMLAttributes<HTMLElement>, "onClick">`.

PROP	TYPE	DESCRIPTION
<code>label</code>	<code>React.ReactNode</code>	Sentence case, no colon: "Secrets", "Environments", "Changes this week", "Issues".
<code>value</code>	<code>React.ReactNode</code>	The number, already formatted: 89, "1,284". Rendered in <code>title-lg</code> with tabular figures.
<code>delta?</code>	<code>React.ReactNode</code>	The change against a named period: "+3 this week", "2 new since Monday".
<code>deltaTone?</code>	<code>"neutral" "success" "warning" "danger"</code>	Color of delta: success when the change is good, warning or danger when it is not, neutral otherwise.
<code>series?</code>	<code>number[]</code>	Values for the sparkline, oldest first, for example changes per day over 12 days.
<code>seriesLabels?</code>	<code>string[]</code>	A label per point ("Sep 19"), read on hover in place of the delta line.
<code>unit?</code>	<code>string</code>	Unit after a hovered value and in the sparkline summary: "changes".
<code>icon?</code>	<code>string</code>	Lucide icon before the label.

PROP	TYPE	DESCRIPTION
<code>href?</code>	<code>string</code>	Makes the whole tile a link, with a hover lift and an <code>arrow-up-right</code> hint.
<code>onClick?</code>	<code>(event: React.MouseEvent) => void</code>	Makes the whole tile a button.
<code>loading?</code>	<code>boolean</code>	Shows a skeleton.

Table

Dense, keyboard first, with selection and groups

A dense, keyboard first data table for tokens, members, activity and secrets: a sticky header, hairline rows, optional selection, Linear style groups, and loading and empty states.

- **Anatomy.** A header row on `bg-subtle` with labels in `small-medium text-tertiary`, then rows of `--row` (44px) or `--row-compact` (36px) separated by 1px border hairlines. Cells have 12px side padding (16px at the table edges), small text in `text-secondary`, and the first column in `text-mono` columns use `mono`; `align="right"` columns use tabular numerals. `selectable` adds a 32px column of 16px Checkboxes and a select all box that shows the mixed state. Group headers are `row-compact` tall on `bg-subtle` with a chevron, an optional icon, the label in `small-medium` and a tabular count in `text-tertiary`. `framed` adds a border ring and `--r-8` corners.
- **States.** Row hover fills with `fill`; selected rows sit on `accent-soft`; the focused row draws a 1.5px focus ring inset by 2px so it is never clipped by the scroll edge. Loading draws skeleton bars in `fill-hover` that pulse and match the columns, and sets `aria-busy`. With no rows the empty node (an inline `EmptyState`) fills the body.
- **Behaviour.** One row is in the Tab order at a time. Arrow Down and Up (or `j` and `k`) move between rows and group headers, Home and End jump to the ends, Enter opens the row (`onRowClick`), Space or `x` toggles its selection. On a group header, Enter or Space folds it, Arrow Left folds and Arrow Right unfolds. Clicking a row opens it, but clicks on buttons, links and checkboxes inside it do not. The header, and each group header beneath it, stays pinned while the table scrolls; give the table a height with `maxHeight` or a flex parent. Narrow screens scroll the table sideways once the columns reach their minimum widths.
- **Copy.** Headers are short nouns in sentence case: "Name", "Scope", "Permission", "Last used", "Expires". Dates follow the product format ("4 min ago", "yesterday at 18:32", "Sep 12"). The selection summary counts: "2 tokens selected".
- **Composition.** Put scope in an `EnvBadge` (sm), row actions in an `xs IconButton` "more-horizontal" labelled with the row ("Actions for ci-deploy") in a 48px last column. Inside a `Card`, use `padding="none"` and leave `framed` off.

TableColumn

PROP	TYPE	DESCRIPTION
<code>key</code>	<code>string</code>	Unique key; also the row field shown when there is no <code>render</code> .
<code>header</code>	<code>React.ReactNode</code>	Header label in sentence case: "Name", "Last used".
<code>width?</code>	<code>number string</code>	Track width: a number in px, any CSS track ("minmax(160px, 2fr)", or unset for a flexible <code>minmax(0, 1fr)</code> column.
<code>minWidth?</code>	<code>number</code>	Minimum width in px for a flexible column; also counts toward the width at which the table starts to scroll sideways.
<code>align?</code>	<code>"left" "right" "center"</code>	<code>right</code> for numbers and dates that should line up (tabular numerals), <code>center</code> for icons.
<code>render?</code>	<code>(row: Row, index: number) => React.ReactNode</code>	Custom cell content. Strings and numbers returned here are truncated with an ellipsis.

PROP	TYPE	DESCRIPTION
mono?	boolean	Sets the cell in mono, for keys, token names and values.
className?	string	Extra class on every cell of the column.

TableGroup

PROP	TYPE	DESCRIPTION
key	string	Unique key of the group.
label	React.ReactNode	Group label, for example "Active" or an environment slug.
icon?	string React.ReactNode	Leading Lucide icon name or node, such as an EnvDot.
rows	Row[]	Rows in the group.
count?	number	Overrides the count shown after the label (defaults to rows.length).
defaultCollapsed?	boolean	Starts folded.

TableProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onSelect">`.

PROP	TYPE	DESCRIPTION
columns	TableColumn<Row>[]	Column definitions, left to right.
rows?	Row[]	Rows to show. Ignored when groups is given.
groups?	TableGroup<Row>[]	Rows split under collapsible group headers (label and count, Linear style) that stick under the table header.
rowKey?	string ((row: Row) => string)	Field name or function that returns a unique key for a row. Defaults to "id".
rowLabel?	(row: Row) => string	Readable name of a row for its selection checkbox: "Select ci-deploy". Defaults to the row key.
selectable?	boolean	Adds a 32px checkbox column and a select all box in the header that shows the mixed state when some rows are selected.
selected?	string[]	Controlled list of selected row keys.
defaultSelected?	string[]	Initial selection when uncontrolled.
onSelectedChange?	(keys: string[]) => void	Called with the next list of selected keys, in row order.
density?	"comfortable" "compact"	comfortable rows are --row (44px), compact rows are --row-compact (36px).
onRowClick?	(row: Row, event: React.SyntheticEvent) => void	Called when a row is clicked or Enter is pressed on it. Clicks on buttons, links and inputs inside the row are ignored.
empty?	React.ReactNode	Shown in place of the rows when there are none, typically an inline EmptyState.
loading?	boolean	Draws skeleton rows that match the columns and sets aria-busy.
loadingRows?	number	Number of skeleton rows.

PROP	TYPE	DESCRIPTION
<code>stickyHeader?</code>	<code>boolean</code>	Keeps the header (and group headers) pinned while the table scrolls. Give the table a height (<code>maxHeight</code> or a flex parent) for it to scroll.
<code>maxHeight?</code>	<code>number string</code>	Maximum height before the table scrolls inside itself.
<code>framed?</code>	<code>boolean</code>	Draws a border ring with <code>--r-8</code> corners, for a table that stands alone on a page.
<code>label?</code>	<code>string</code>	Accessible name of the grid, for example "Access tokens".

CommandMenu

Cmd K: jump anywhere, run anything

The Cmd K palette: jump to a project, environment or secret, or run an action, without leaving the keyboard.

- **Anatomy.** A 640px panel on surface-raised with `--r-12` corners and `shadow-x1`, placed 18 percent from the top over a lighter overlay than Dialog (the overlay token at 55 percent, no blur). A 48px input row with a search icon, page chips and a spinner while loading, above a border hairline. A scrolling list of groups: headings in `caption-text-tertiary`, 40px items with a 16px icon (or an `EnvDot`), the label in body (mono for keys, slugs and projects), a hint in `small-text-tertiary`, a `Kbd` at the right and a `chevron-right` for items that open a page. A footer with `Kbd` hints: "UpDown to navigate", "Enter to select", "esc to close". Under 640px it fills the width 8px from the top.
- **Behaviour.** Opens on Cmd K (Ctrl K elsewhere) through hotkey, with focus in the input, and returns focus where it was on close. Typing filters every group: a substring match ranks highest (more at the start of a word), then an in-order subsequence that starts a word, so "strpe" finds `STRIPE_SECRET_KEY`; hint and keywords match as plain substrings. Matched characters are set in 600 weight with a hairline underline. Groups with no matches hide and reorder by their best match. "Recent" shows only while the query is empty.
- **Keyboard.** ArrowDown and ArrowUp (and Ctrl N, Ctrl P) move with wrap; Home and End jump to the ends; Enter selects. The active item has `fill-hover` and is scrolled into view; the pointer moves it too. Escape goes back one page, then closes. Selecting an item with page pushes it: its title shows as a chip before the input, the placeholder changes, and Backspace on an empty input (or a click on the chip) goes back.
- **States.** Empty: No results for "kubernetes" in `body-medium` with a hint in `text-tertiary` ("Try a key such as `DATABASE_URL`, a project such as `lumen-api`, or an action such as `import`"). Loading: a spinner in the input, and Searching for "..." instead of the empty state.
- **Accessibility.** The panel is a modal dialog labelled "Command menu" with a focus trap. The input is a combobox controlling the listbox, with `aria-activedescendant` on the active option (`aria-selected`). Groups are groups labelled by their headings. A polite live region announces the result count.
- **Motion.** Opens with a fade and a 0.98 scale over `duration-fast` with `ease-out`; closes over `duration-instant` with `ease-in`.
- **Copy.** Actions are verbs with objects: "Add secret", "Import .env", "Compare environments", "Copy CLI command", "Switch environment". Hints give the context: "lumen-api/staging", "24 secrets", "secmgr run --env staging -- npm run dev".

CommandItem

PROP	TYPE	DESCRIPTION
<code>id?</code>	<code>string</code>	Stable key.
<code>icon?</code>	<code>string</code> <code>React.ReactNode</code>	Lucide icon name, or a node such as <code><EnvDot color="amber" /></code> .

PROP	TYPE	DESCRIPTION
label	<code>React.ReactNode</code>	What the item is or does: "lumen-api", "Add secret", STRIPE_SECRET_KEY. Matched characters are highlighted when it is a string.
textValue?	<code>string</code>	Text used for matching when <code>label</code> is not a string.
mono?	<code>boolean</code>	Sets the label in mono, for keys and slugs.
hint?	<code>React.ReactNode</code>	Context after the label in <code>text-tertiary</code> : "lumen-labs/lumen-api", "lumen-api/staging", "24 secrets". Also searched.
kbd?	<code>string string[]</code>	Shortcut at the right, for example "c" or ["mod", "i"].
keywords?	<code>string[]</code>	Extra words that match the item without being shown: ["new", "create"].
disabled?	<code>boolean</code>	Skipped by Enter and dimmed.
onSelect?	<code>(item: CommandItem) => void</code>	Runs on Enter or click. The menu then closes, unless the item has a page.
page?	<code>{ title?: string; placeholder?: string; items?: CommandItem[]; groups?: CommandGroup[] }</code>	A nested page pushed on select: its title shows as a chip in the input, Backspace on an empty input or Escape goes back.

CommandGroup

PROP	TYPE	DESCRIPTION
label?	<code>string</code>	Heading in caption <code>text-tertiary</code> : "Jump to", "Actions".
items	<code>CommandItem[]</code>	

CommandMenuProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
open?	<code>boolean</code>	Controlled open state.
defaultOpen?	<code>boolean</code>	Initial open state when uncontrolled.
onOpenChange?	<code>(open: boolean) => void</code>	Called with the next open state on the hotkey, selection, Escape and an overlay click.
placeholder?	<code>string</code>	Input placeholder on the root page.
groups?	<code>CommandGroup[]</code>	The searchable groups on the root page.
recent?	<code>CommandItem[]</code>	Items shown first under "Recent" while the query is empty.
loading?	<code>boolean</code>	Shows a spinner in the input, and "Searching for ..." instead of the empty state, while results load.
emptyHint?	<code>React.ReactNode</code>	Second line of the empty state under <code>No results for "...</code> ".
defaultQuery?	<code>string</code>	Query the menu opens with.
onQueryChange?	<code>(query: string) => void</code>	Called on every keystroke, for async search.
hotkey?	<code>string[] false</code>	Global shortcut that toggles the menu, ["mod", "k"] by default. Pass false to handle it yourself.

PROP	TYPE	DESCRIPTION
<code>initialFocus?</code>	<code>React.RefObject<HTMLElement> false</code>	Element to focus on open, the input by default. Pass false for a menu shown open at rest.
<code>container?</code>	<code>HTMLElement null</code>	Portal target. Defaults to <code>document.body</code> . Pass an element with <code>transform</code> set to confine the menu to a region, as the previews do.
<code>label?</code>	<code>string</code>	Accessible name of the dialog.

Dialog

Modal decisions that restate the consequence

A modal window for one focused task (add a secret, rename an environment) or one decision (ConfirmDialog). The page behind is dimmed, blurred and inert until you leave.

- **Anatomy.** An overlay in `overlay` with a `2px` backdrop blur, and a panel on surface with `--r-12` corners and `shadow-lg`, `dialog-sm` (400), `dialog-md` (520) or `dialog-lg` (760) wide. The header holds the title in `title`, the description in `body text-secondary`, and a `sm` close `IconButton` ("Close", Esc). The body scrolls on its own when the dialog is taller than the viewport. The footer is a strip on `bg-subtle` above a `border` hairline, buttons at the right with the primary last. Under 480px the dialog sits at the bottom of the screen and footer buttons stack full width, primary on top.
- **Behaviour.** Opens at `z-dialog` in a portal and locks page scroll. Focus moves to the element marked `data-autofocus`, else the first field in the body, else the first button in the footer, and is trapped: Tab and Shift Tab cycle inside. Escape, the close button and an overlay click close it (`closeOnOverlay={false}` for forms that would lose typed input), and focus returns to the element that opened it. `role="dialog"`, `aria-modal`, `aria-labelledby` by the title and `aria-describedby` by the description.
- **Motion.** The overlay fades in and the panel enters with `sg-pop-in`, both over `duration-base` with `ease-out`. Leaving fades and scales to `0.98` over `duration-fast` with `ease-in`.
- **ConfirmDialog.** Restates the object and the consequence in the title and description, then offers the way back and the action. `tone="danger"` uses a danger confirm button and focuses Cancel first; default uses `primary` and focuses the action. `requireText` asks the person to type the environment name ("Type production to confirm") into a mono input that takes focus; the confirm button stays disabled until the text matches exactly, and Enter confirms. When `onConfirm` returns a promise the confirm button shows a spinner, Cancel and Escape are blocked, and the dialog closes when it resolves or stays open when it rejects. It has `role="alertdialog"`.
- **Copy.** Titles are questions that name the count, the object and the environment: "Delete 4 secrets from production?", "Discard 3 changes?". Descriptions state what happens: "Services reading them will fail on their next deploy." The confirm label repeats the action: "Delete 4 secrets", "Discard 3 changes". The way back is "Cancel" or "Keep editing". Never "OK", "Yes" or "Are you sure?".
- **Composition.** List the affected keys in the body as mono chips when there are a few. Protected environments always use `requireText`. A task with more than one step, or a detail view people read alongside the page, belongs in a Sheet.

DialogProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "title">`.

PROP	TYPE	DESCRIPTION
<code>open?</code>	<code>boolean</code>	Controlled open state.
<code>defaultOpen?</code>	<code>boolean</code>	Initial open state when uncontrolled.
<code>onOpenChange?</code>	<code>(open: boolean) => void</code>	Called with the next open state on trigger click, the close button, Escape and an overlay click.

PROP	TYPE	DESCRIPTION
trigger?	<code>React.ReactElement</code>	Optional element that opens the dialog on click. Focus returns to whatever was focused before opening.
title?	<code>React.ReactNode</code>	Heading in <code>title</code> type, labelling the dialog: "Add secret to staging", "Delete 4 secrets from production?".
description?	<code>React.ReactNode</code>	One or two sentences under the title in <code>text-secondary</code> , describing the dialog: the consequence or what happens next.
size?	<code>"sm" "md" "lg"</code>	Width from the <code>dialog-sm</code> (400), <code>dialog-md</code> (520) or <code>dialog-lg</code> (760) tokens.
footer?	<code>React.ReactNode</code>	Buttons for the footer strip, primary last: <code><><Button>Cancel</Button><Button variant="primary">Add secret</Button></></code> .
closeOnOverlay?	<code>boolean</code>	Closes when the overlay is clicked. Turn it off for forms that would lose typed input.
hideClose?	<code>boolean</code>	Hides the close button in the header. Escape still closes.
initialFocus?	<code>React.RefObject<HTMLElement> false</code>	Element to focus on open. By default: an element with <code>data-autofocus</code> , else the first focusable element in the body, then in the footer. Pass <code>false</code> to leave focus where it is, for a dialog shown open at rest.
container?	<code>HTMLElement null</code>	Portal target. Defaults to <code>document.body</code> . Pass an element with <code>transform</code> set to confine the dialog to a region, as the previews do; the page is then not scroll locked.
children?	<code>React.ReactNode</code>	The body. It scrolls when taller than the viewport; the header and footer stay put.

ConfirmDialogProps

Extends `Omit<DialogProps, "footer" | "children">`.

PROP	TYPE	DESCRIPTION
confirmLabel?	<code>React.ReactNode</code>	The action, verb first with the object and count: "Delete 4 secrets", "Discard 3 changes". Never "OK".
cancelLabel?	<code>React.ReactNode</code>	The way back: "Cancel" or "Keep editing".
tone?	<code>"default" "danger"</code>	<code>danger</code> makes the confirm button <code>danger</code> and focuses Cancel first. <code>default</code> uses <code>primary</code> and focuses the confirm button.
requireText?	<code>string</code>	Text the person must type exactly before the confirm button enables, for protected environments: "production". Shows "Type production to confirm" above a mono input, which takes focus.
loading?	<code>boolean</code>	Shows a spinner in the confirm button, disables Cancel and blocks closing. When given, the parent closes the dialog itself.
onConfirm?	<code>() => void Promise<unknown></code>	Runs on confirm (button or Enter in the input). Return a promise to show loading until it settles: the dialog closes when it resolves and stays open when it rejects. Otherwise the dialog closes at once.
onCancel?	<code>() => void</code>	Runs when the person cancels, presses Escape or clicks the overlay.
children?	<code>React.ReactNode</code>	Extra content above the typed confirmation, such as the list of keys affected.

Menu

Row actions, submenus, full keyboard

A dropdown list of actions opened from a trigger, such as the "More actions" button on a secret row or the workspace switcher. Every item works from the keyboard.

- **Anatomy.** A panel on surface-raised with shadow-md, --r-8 corners, 4px padding, at least 200px and at most 320px wide. Items are 32px tall with --r-6 corners: a 16px icon in text-secondary (or an EnvDot), the label in body, an optional caption description in text-tertiary, and a Kbd (sm) at the right. MenuLabel is an overline heading in text-tertiary. MenuSeparator is a full bleed 1px border line. MenuSub items end in a chevron-right. MenuCheckboxItem draws a check in the icon slot; MenuRadioItem draws a 6px dot at the right.
- **States.** The highlighted item fills with fill-hover and lifts its icon to text; pressed fills with fill-active. tone="danger" items use danger text and highlight with danger-soft. Disabled items drop to text-disabled and are skipped by the keyboard. An open submenu keeps its trigger highlighted.
- **Opening.** Click the trigger to open with the panel focused and nothing highlighted. Enter, Space or ArrowDown on the trigger opens with the first item highlighted; ArrowUp opens with the last. The trigger gets aria-haspopup="menu", aria-expanded and aria-controls; the panel has role="menu" and is labelled by the trigger.
- **Keyboard.** ArrowDown and ArrowUp move with wrap, Home and End jump to the ends, typing a few letters jumps to the next matching item, Enter and Space select. Escape closes and returns focus to the trigger. Tab closes and moves on from the trigger. In a submenu, ArrowRight, Enter or Space opens it with the first item highlighted, and ArrowLeft or Escape closes only the submenu and returns to its item.
- **Pointer.** The highlight follows the pointer. A submenu opens after the pointer rests 100ms on its item; moving diagonally toward it keeps it open (a 300ms safe triangle). A click outside closes the menu.
- **Selection.** Selecting an item runs onSelect and closes the whole menu; call event.preventDefault() to keep it open. Checkbox items stay open by default so several columns can be toggled. Focus returns to the trigger before any dialog the item opens, so the dialog returns focus there too.
- **Motion.** Enters with sg-pop-in over duration-fast with ease-out, its transform origin on the trigger's side (mirrored for menus above the trigger and for submenus). Leaves with a fade over duration-instant with ease-in.
- **Copy.** Verb first, sentence case, the object named when it is not obvious: "Copy value", "Copy as export statement", "Show history", "Share one time link". Destructive items name the environment: "Delete from staging". Environment names stay lowercase.
- **Composition.** Order items by frequency, group them with separators, and put danger items last after a separator. Use align="end" for row actions at the right edge. A destructive item opens a ConfirmDialog rather than acting at once.

MenuProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "children">`.

PROP	TYPE	DESCRIPTION
------	------	-------------

PROP	TYPE	DESCRIPTION
trigger?	<code>React.ReactElement</code>	The element that opens the menu: usually an <code>IconButton</code> "More actions" or a ghost <code>Button</code> . It receives <code>aria-haspopup="menu"</code> , <code>aria-expanded</code> , <code>aria-controls</code> and an <code>id</code> the menu is labelled by.
open?	<code>boolean</code>	Controlled open state.
defaultOpen?	<code>boolean</code>	Initial open state when uncontrolled.
onOpenChange?	<code>(open: boolean) => void</code>	Called with the next open state on trigger click, selection, Escape, Tab and outside click.
side?	<code>"top" "bottom"</code>	Side of the trigger to open on. Flips when there is no room.
align?	<code>"start" "center" "end"</code>	Alignment along the side: end for row actions at the right edge.
offset?	<code>number</code>	Gap between the trigger and the menu in px.
autoFocus?	<code>boolean</code>	Moves focus into the menu on open: the first item when opened from the keyboard, the panel when opened by pointer. Set false for a menu shown open at rest.
children?	<code>React.ReactNode</code>	Items, labels, separators, groups and submenus.

MenuSelectEvent

PROP	TYPE	DESCRIPTION
defaultPrevented	<code>boolean</code>	
nativeEvent	<code>Event</code>	

MenuItemProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onSelect">`.

PROP	TYPE	DESCRIPTION
icon?	<code>string React.ReactNode</code>	Leading Lucide icon name, or a node such as <code><EnvDot color="amber" /></code> .
label?	<code>React.ReactNode</code>	The action, verb first: "Copy value", "Show history". Falls back to children.
description?	<code>React.ReactNode</code>	A second line in caption text-tertiary for consequences: "Expires after one view or 24 hours".
kbd?	<code>string string[]</code>	Shortcut shown at the right, for example "e" or ["mod", "c"] or ["g", "s"].
tone?	<code>"default" "danger"</code>	danger for destructive actions: danger text, danger-soft highlight. Put them last, after a separator.
disabled?	<code>boolean</code>	Skipped by the keyboard and not selectable.
onSelect?	<code>(event: MenuSelectEvent) => void</code>	Runs on click, Enter or Space.
closeOnSelect?	<code>boolean</code>	Closes the whole menu after <code>onSelect</code> unless it called <code>preventDefault()</code> .
textValue?	<code>string</code>	Text used by typeahead when <code>label</code> is not a string.
children?	<code>React.ReactNode</code>	

MenuCheckboxItemProps

Extends `Omit<MenuItemProps, "icon" | "tone">`.

PROP	TYPE	DESCRIPTION
<code>checked?</code>	<code>boolean</code>	Draws a check in the icon slot and sets <code>aria-checked</code> .
<code>onCheckedChange?</code>	<code>(checked: boolean) => void</code>	Called with the next checked state on select.

MenuRadioGroupProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>value?</code>	<code>string</code>	The selected item's value.
<code>onValueChange?</code>	<code>(value: string) => void</code>	Called with the chosen item's value.
<code>label?</code>	<code>string</code>	Accessible name of the group; pair it with a visible <code>MenuLabel</code> .
<code>children?</code>	<code>React.ReactNode</code>	

MenuRadioItemProps

Extends `Omit<MenuItemProps, "tone">`.

PROP	TYPE	DESCRIPTION
<code>value</code>	<code>string</code>	Value reported to the group.

MenuSubProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>icon?</code>	<code>string React.ReactNode</code>	Leading Lucide icon name or node.
<code>label</code>	<code>React.ReactNode</code>	Label of the submenu trigger: "Copy to environment".
<code>description?</code>	<code>React.ReactNode</code>	Second line under the label.
<code>disabled?</code>	<code>boolean</code>	
<code>defaultOpen?</code>	<code>boolean</code>	Opens the submenu at rest, for documentation.
<code>textValue?</code>	<code>string</code>	Text used by typeahead when <code>label</code> is not a string.
<code>children?</code>	<code>React.ReactNode</code>	The submenu's items.

Popover

One small task, anchored to its trigger

A floating panel anchored to a trigger for one small task that needs interaction: filtering the secrets table, sharing a one time link, setting a rotation policy.

- **Anatomy.** A panel on `surface-raised` with `shadow-md`, `--r-8` corners and 12px padding, sized by its content and kept 8px inside the viewport. It renders in a portal at `z-popover` (one step above the surrounding dialog or sheet when nested). No arrow.
- **Placement.** `side` is `bottom` and `align` is `start` by default, 6px from the anchor, flipping to the opposite side when there is no room. Anchor to the `trigger`, or pass `anchorRef` with a controlled `open` to anchor to anything (a table cell, a selection).
- **Behaviour.** The trigger toggles it on click and carries `aria-haspopup="dialog"`, `aria-expanded` and `aria-controls`. On open, focus moves to the first focusable element inside (or the panel). Escape and a click outside close it and return focus to the trigger. Tab past the last element closes it and continues to the element after the trigger; Shift Tab before the first returns to the trigger. Wrap a button in `PopoverClose` to close from inside.
- **Motion.** Enters with a fade, a 4px slide from the anchor and a 0.98 scale over `duration-fast` with `ease-out`, from a transform origin on the anchor's side. Leaves with a fade over `duration-instant` with `ease-in`.
- **Copy.** Give the panel a short heading in `heading` when it holds a form ("Share one time link"), state the consequence under it in `small text-secondary` ("Expires after one view or 24 hours."), and name buttons with the action ("Copy link", "Rotate now").
- **When not to use.** A list of actions is a `Menu`. A label for an icon is a `Tooltip`. Anything that needs more than one small form, or a confirmation, is a `Dialog`.

PopoverProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "children">`.

PROP	TYPE	DESCRIPTION
<code>trigger?</code>	<code>React.ReactElement</code>	The element that toggles the popover on click. It receives <code>aria-haspopup</code> , <code>aria-expanded</code> and <code>aria-controls</code> . Omit it and pass <code>anchorRef</code> with a controlled <code>open</code> to anchor anywhere.
<code>open?</code>	<code>boolean</code>	Controlled open state.
<code>defaultOpen?</code>	<code>boolean</code>	Initial open state when uncontrolled.
<code>onOpenChange?</code>	<code>(open: boolean) => void</code>	Called with the next open state on trigger click, outside click, Escape and <code>PopoverClose</code> .
<code>anchorRef?</code>	<code>React.RefObject<HTMLElement></code>	Element to anchor to when there is no <code>trigger</code> , or to anchor somewhere other than the trigger. Focus returns here on close when there is no trigger.
<code>side?</code>	<code>"top" "bottom" "left" "right"</code>	Side of the anchor to open on. Flips when there is no room.
<code>align?</code>	<code>"start" "center" "end"</code>	Alignment along the side.

PROP	TYPE	DESCRIPTION
<code>offset?</code>	<code>number</code>	Gap between the anchor and the panel in px.
<code>autoFocus?</code>	<code>boolean</code>	Moves focus to the first focusable element in the panel (or the panel itself) on open. Set false for a popover shown open at rest.
<code>children?</code>	<code>React.ReactNode</code>	Panel content. Keep it to one small task.

PopoverCloseProps

PROP	TYPE	DESCRIPTION
<code>children</code>	<code>React.ReactElement</code>	One element; its <code>onClick</code> also closes the surrounding popover.

Sheet

Detail views that slide in, modal or as a side peek

A panel that slides in from the right edge for detail views and longer tasks: a secret's values across environments, the import preview, a member's access. Under 640px it rises from the bottom.

- **Anatomy.** A panel on surface with `--r-12` corners, inset 8px from the top, right and bottom edges, sheet (520) wide and never wider than the screen. The header holds the title in `title`, a one line description in `small text-secondary`, an actions slot for `sm IconButton`s and a "More actions" Menu, and the close button ("Close", Esc), above a `border` hairline. The body scrolls. The footer is a sticky strip on `bg-subtle` with buttons at the right, primary last. Under 640px it becomes a bottom sheet up to 85 percent of the height with a grabber bar, and footer buttons share the width.
- **Modal.** By default the sheet dims the page with `overlay` and a 2px blur, traps focus, locks scroll, and closes on Escape, the close button or an overlay click. It sits at `z-sheet`, below dialogs, so a `ConfirmDialog` opened from its footer stacks on top.
- **Side peek.** `modal={false}` drops the overlay, the trap and the scroll lock: the page stays usable beside it, as in a Notion side peek. It carries `shadow-xl` to separate it from the page. Escape still closes it. Use it for reading a row's detail while scanning the table; keep the row highlighted.
- **Focus.** On open, focus moves to an element marked `data-autofocus`, else to the sheet itself so Tab starts at its first control. On close focus returns to the element that opened it. `role="dialog"`, `aria-modal` when modal, labelled by the title and described by the description.
- **Motion.** Slides in over `duration-base` with `ease-out` (from the right, or from the bottom under 640px) while the overlay fades in. Leaves faster over `duration-fast` with `ease-in`. Only `transform` and `opacity` animate.
- **Copy.** Title a detail sheet with the object's exact name in mono (`STRIPE_SECRET_KEY`), and a task sheet with the action and the target ("Import .env into staging"). The description gives the project, the count and the last change: "lumen-api, 4 environments, updated 4 min ago".
- **Composition.** Keep one primary action in the footer ("Edit values", "Import 12 secrets"). Destructive actions live in the header's Menu and open a `ConfirmDialog`. Use a `Dialog` instead when the task is one short decision.

SheetProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "title">`.

PROP	TYPE	DESCRIPTION
<code>open?</code>	<code>boolean</code>	Controlled open state.
<code>defaultOpen?</code>	<code>boolean</code>	Initial open state when uncontrolled.
<code>onOpenChange?</code>	<code>(open: boolean) => void</code>	Called with the next open state on trigger click, the close button, Escape and an overlay click.
<code>trigger?</code>	<code>React.ReactElement</code>	Optional element that toggles the sheet on click.

PROP	TYPE	DESCRIPTION
title?	<code>React.ReactNode</code>	Heading in <code>title</code> type that labels the sheet: a key such as <code>STRIPE_SECRET_KEY</code> , or a task such as "Import <code>.env</code> into staging".
description?	<code>React.ReactNode</code>	One line under the title in <code>small text-secondary</code> : project, counts, last change.
actions?	<code>React.ReactNode</code>	Header slot before the close button, for <code>sm</code> <code>IconButton</code> s and a "More actions" Menu.
footer?	<code>React.ReactNode</code>	Buttons for the sticky footer strip, primary last.
modal?	<code>boolean</code>	<code>true</code> dims the page with an overlay, traps focus and locks scroll. <code>false</code> is a side peek: no overlay, the page stays usable, focus moves in but is not trapped.
closeOnOverlay?	<code>boolean</code>	Closes when the overlay is clicked (modal only).
hideClose?	<code>boolean</code>	Hides the close button. Escape still closes.
initialFocus?	<code>React.RefObject<HTMLElement> false</code>	Element to focus on open. Defaults to an element with <code>data-autofocus</code> , else the sheet itself. Pass <code>false</code> to leave focus where it is, for a sheet shown open at rest.
container?	<code>HTMLElement null</code>	Portal target. Defaults to <code>document.body</code> . Pass an element with <code>transform</code> set to confine the sheet to a region, as the previews do.
children?	<code>React.ReactNode</code>	The scrolling body.

Toast

A result, the key and the way back

A short, temporary message that pairs an action with its result ("Copied STRIPE_SECRET_KEY. Clipboard clears in 30s.") and offers the way back ("Undo"). Call `toast()` from anywhere; render one `Toaster`.

- **Anatomy.** A 360px card on `surface-raised` with `--r-8` corners and `shadow-md`: a 16px icon, the title in `small-medium`, an optional description in `small text-secondary`, and an optional `sm secondary Button` for the action. Tones color the icon only (success, warning, danger); the card stays neutral. A 20px round close button sits on the top left corner and shows on hover and focus (always on touch).
- **Stack.** Newest in front at the bottom right, 16px from the edges (bottom centre under 640px). Up to 3 show at once: the ones behind are lifted 14px each, scaled down 5 percent each and matched to the front card's height, with their content hidden. Hovering or focusing the stack expands it into a column with 8px gaps. Older toasts wait behind the third and move up as others leave.
- **Timing.** 5s by default, 10s when there is an action, `Infinity` for toasts that must stay. Timers pause while the stack is hovered or focused and while the window is in the background, and resume with the time that was left. A loading toast never times out. `toast.promise` shows the loading state with a `Spinner` and turns the same toast into success or danger when the promise settles.
- **Motion.** Enters by sliding up from below with a fade over `duration-base` with `ease-out`; the stack moves with the same curve. Leaves with a fade (the front one also slides down) over `duration-fast` with `ease-in`. Only transform and opacity animate.
- **Accessibility.** The stack is a section with `aria-live="polite"`, labelled "Notifications alt+T"; Alt T moves focus to the newest toast's action. Close buttons are labelled "Dismiss notification". Never put the only copy of important information in a toast; it disappears.
- **Copy.** State the result with the count, the key and the environment, in sentence case: "Saved 3 changes to staging", "Imported 12 secrets into development", "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s." Errors say what happened and what happens next: "Could not reach the server" with "Your changes are kept; retrying in 5s." Actions are one verb: "Undo", "Retry now", "Rotate".
- **Composition.** Use a toast for the result of something the person just did. Use a `Callout` or an inline error for anything that needs a decision, and a `ConfirmDialog` before destructive actions (then a toast with "Undo" after).

ToastOptions

PROP	TYPE	DESCRIPTION
<code>id?</code>	<code>string</code>	Reuse an id to update a toast in place (the promise helper does this).
<code>title?</code>	<code>React.ReactNode</code>	The result, stating the count, the key and the environment: "Saved 3 changes to staging".
<code>description?</code>	<code>React.ReactNode</code>	A second line in <code>text-secondary</code> : the consequence or what happens next.
<code>tone?</code>	"neutral" "success" "warning" "danger"	success, warning and danger color the leading icon only; the card stays neutral.

PROP	TYPE	DESCRIPTION
<code>icon?</code>	<code>string</code>	Leading Lucide icon. Defaults to <code>circle-check</code> , <code>triangle-alert</code> or <code>circle-alert</code> by tone, none for neutral.
<code>action?</code>	<code>{ label: React.ReactNode; onClick?: () => void }</code>	One follow up action as a small secondary button: <code>{ label: "Undo", onClick }</code> . Clicking it also dismisses the toast.
<code>duration?</code>	<code>number</code>	Time on screen in ms: 5000 by default, 10000 with an action, Infinity to stay until dismissed. Paused while the stack is hovered or focused and while the window is in the background.
<code>loading?</code>	<code>boolean</code>	Shows a spinner in place of the icon and never times out. Set by <code>toast.promise</code> .
<code>dismissible?</code>	<code>boolean</code>	Set false to hide the close button.
<code>className?</code>	<code>string</code>	Extra class on the toast card.

ToasterProps

PROP	TYPE	DESCRIPTION
<code>visible?</code>	<code>number</code>	How many toasts show in the stack; older ones wait behind and resume when there is room.
<code>expand?</code>	<code>boolean</code>	Keeps the stack expanded instead of collapsing it behind the newest toast. Timers still run unless hovered.
<code>container?</code>	<code>HTMLElement null</code>	Portal target. Defaults to <code>document.body</code> . Pass an element with <code>transform</code> set to confine the stack to a region, as the preview does.
<code>hotkey?</code>	<code>string[] false</code>	Shortcut that moves focus to the newest toast, <code>["alt", "t"]</code> by default. Pass false to turn it off.
<code>label?</code>	<code>string</code>	Accessible name of the live region; the hotkey is appended.
<code>className?</code>	<code>string</code>	

Tooltip

Ink label, instant along a toolbar

A short label for one trigger: ink on paper, no arrow, shown after a hover delay or at once on keyboard focus.

- **Anatomy.** Text in caption on a primary fill with on-primary text, --r-6 corners, 4px by 8px padding, shadow-md, at most 240px wide and wrapping after that. An optional shortcut follows the text as Kbd caps in the inverse tone. No arrow. It renders in a portal at z-tooltip and never takes pointer events.
- **Placement.** side is top by default; align is center. It sits 6px from the trigger, flips to the opposite side when there is no room, and stays 8px inside the viewport.
- **Behaviour.** Opens 450ms after the pointer rests on the trigger, and at once on keyboard focus (focus that arrives by Tab, not by a click or a restored focus). When another tooltip closed less than 300ms ago the next one opens instantly and without animation, so moving along a toolbar reads as one label that follows the pointer. Only one tooltip is open at a time. Closes on pointer leave, pointer down, blur, Escape and any scroll. Stays closed while the trigger's menu is open (aria-expanded="true"). Touch never opens it.
- **Motion.** Enters with a fade and a 2px slide from its side over duration-fast with ease-out; leaves with a fade over duration-instant with ease-in.
- **Accessibility.** The tooltip has role="tooltip" and the trigger gets aria-describedby while it is open. When the trigger's accessible name already says the same words (IconButton), set describe={false} so it is not read twice.
- **Copy.** Name the action or the fact, in sentence case with no final period for single phrases: "Copy value", "Reveal value", "Show history". Use a full sentence only for facts: "Protected. Changes need a typed confirmation." Never put anything interactive inside; use a Popover for that.
- **Composition.** Wrap exactly one element that accepts a ref and pointer and focus handlers. Wrap disabled buttons in a span with tabIndex={0} if they need a tooltip. IconButton uses this component for its label.

TooltipProps

PROP	TYPE	DESCRIPTION
content?	React.ReactNode	The text to show, written as the action or the name: "Copy value", "Protected by Maya Chen". Empty content renders no tooltip.
kbd?	string string[]	Shortcut shown after the text as inverse Kbd caps, for example ["mod", "c"] or ["g", "p"].
side?	"top" "bottom" "left" "right"	Side of the trigger to open on. Flips to the opposite side when there is no room.
align?	"start" "center" "end"	Alignment along the side.
offset?	number	Gap between the trigger and the tooltip in px.
delay?	number	Hover delay in ms before opening. Skipped when another tooltip closed less than 300ms ago, and on keyboard focus.

PROP	TYPE	DESCRIPTION
<code>open?</code>	<code>boolean</code>	Controlled open state. Use it to show a tooltip at rest in documentation.
<code>defaultOpen?</code>	<code>boolean</code>	Initial open state when uncontrolled.
<code>onOpenChange?</code>	<code>(open: boolean) => void</code>	Called with the next open state on hover, focus, blur, Escape and scroll.
<code>disabled?</code>	<code>boolean</code>	Never opens while true.
<code>describe?</code>	<code>boolean</code>	Adds <code>aria-describedby</code> on the trigger while open. Turn it off when the trigger's accessible name already says the same thing, as <code>IconButton</code> does.
<code>className?</code>	<code>string</code>	Class for the floating tooltip.
<code>children</code>	<code>React.ReactElement</code>	Exactly one element that accepts a ref and pointer and focus handlers: a DOM element or a component that forwards them.

Badge

Status always pairs a color with a word

A short label for a status, a count or a property, set in caption at medium weight on a 20px chip with `--r-4` corners.

- **Tones.** `neutral` for counts and properties ("24 secrets", "Sensitive"). `accent` for new things. `success`, `warning` and `danger` for states: each pairs its `soft` ground with its text token, and always carries a word ("In sync", "Rotation due", "Missing in production"), plus an icon when there is room. Never color alone.
- **Variants.** `soft` by default. `outline` sits in dense tables where many badges would otherwise shout.
- **Dot.** `dot` adds a 6px circle for live states. It keeps `corner-shape: round`.
- **Environments** are not badges: use `EnvBadge`, which carries the environment's own color.

BadgeProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
<code>tone?</code>	<code>"neutral" "accent" "success" "warning" "danger"</code>	
<code>variant?</code>	<code>"soft" "outline"</code>	<code>soft</code> fills with the tone's soft ground; <code>outline</code> sits quietly in dense tables.
<code>icon?</code>	<code>string</code>	Leading Lucide icon at 14px.
<code>dot?</code>	<code>boolean</code>	A 6px dot in the tone color, for live states such as "In sync".
<code>children?</code>	<code>React.ReactNode</code>	

Callout

A tinted note that states the risk and offers the fix

A tinted note inside a page that states a fact about what you are looking at and, when there is one, the fix.

- **Anatomy.** A 16px icon in the tone color, a body-medium title in text, detail in small text-secondary (inline code in mono-sm text), an optional sm action on the right, and an optional xs close button. 12px padding, --r-8 corners, the tone's soft ground with a faint ring mixed from the tone color.
- **Tones.** info on accent-soft with an info icon, for guidance. success on success-soft with circle-check, after a completed step. warning on warning-soft with triangle-alert, for something due. danger on danger-soft with circle-alert, for a real risk. neutral on bg-subtle with a border ring, for standing rules such as a protected environment (pass icon="lock"). Color never works alone: every callout has an icon and words.
- **Behaviour.** Static. onDismiss adds a close button labelled by dismissLabel. Under 440px of its own width the action moves under the text, aligned with it.
- **Copy.** The title states the fact with the exact name: "Staging uses a live Stripe key", "production is protected", "JWT_SIGNING_KEY is due for rotation". The detail gives the evidence and the consequence with numbers: "Last rotated 214 days ago. This project rotates every 90 days." The action is a verb and an object: "Rotate key".
- **Composition.** Place callouts above the content they describe, one per issue, danger first. For transient confirmations use a toast; for issues across a project use Health.

CalloutProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "title">`.

PROP	TYPE	DESCRIPTION
tone?	"info" "success" "warning" "danger" "neutral"	info (accent) for guidance, success after a completed step, warning for something due, danger for a real risk, neutral for a standing rule such as a protected environment.
icon?	string React.ReactNode false	Lucide icon name or node. Defaults per tone: info, circle-check, triangle-alert, circle-alert, info. false hides it.
title?	React.ReactNode	One line in body-medium that states the fact: "Staging uses a live Stripe key".
children?	React.ReactNode	Detail in small text-secondary; inline code is set in mono.
action?	React.ReactNode	One sm button that fixes it, for example "Rotate key". Moves under the text when the callout is narrower than 440px.
onDismiss?	() => void	Shows an xs close IconButton and calls this when pressed.
dismissLabel?	string	Accessible name of the close button.

EmptyState

What fills this place, and the one step to do it

What an empty place is for and the one step that fills it, from a whole page ("Start with one project") down to a table with no matches.

- **Anatomy.** A 40px tile (`--r-12`, `bg-subtle`, border hairline) with a 20px icon in `text-secondary`; the title; one or two sentences of description in `text-secondary` capped at `--dialog-sm`; actions; and an optional left aligned hint (a `CodeBlock`) capped at `--dialog-md`. Everything is centred.
- **Variants.** `page`: 48px above and below, the title in `display-sm` (Geist 36px at 500, tracked to `-0.025em`), body text in `body`, 24px before the actions and 32px before the hint. Use it when a whole page or panel is empty. `inline`: 32px padding, the title in `heading`, the description in `small`, one `sm` button. Use it inside tables (`Table empty`), cards and menus.
- **Behaviour.** Static. The action does the thing that ends the empty state; after it, the content replaces the empty state in place.
- **Copy.** Titles say what to do or what happened, in sentence case: "Start with one project", "No secrets match "stripe"", "Nothing needs attention". Descriptions say what fills it and state names and numbers. Buttons name the action and the object: "Create project", "Import .env", "Clear search". Never "No data" or "Nothing here".
- **Composition.** One primary action at most, last in the row. Offer the CLI path in hint for developer tasks (`secmgr link lumen-api`). A search with no results always offers "Clear search".

EmptyStateProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "title">`.

PROP	TYPE	DESCRIPTION
<code>icon?</code>	<code>string</code> <code>React.ReactNode</code>	Lucide icon name (20px) or a node, drawn in a 40px <code>bg-subtle</code> tile with a hairline and <code>--r-12</code> corners.
<code>title?</code>	<code>React.ReactNode</code>	<code>page</code> : the <code>display-sm</code> title (Geist 36px, 500) of a whole page ("Start with one project"). <code>inline</code> : a heading title inside a table or card ("No secrets match "stripe").
<code>description?</code>	<code>React.ReactNode</code>	One or two sentences in <code>text-secondary</code> that say what fills this place.
<code>actions?</code>	<code>React.ReactNode</code>	One primary action and at most one more, or a single <code>sm</code> button <code>inline</code> ("Clear search").
<code>hint?</code>	<code>React.ReactNode</code>	A left aligned slot under the actions, for a <code>CodeBlock</code> with the CLI way to do the same thing.
<code>variant?</code>	<code>"page"</code> <code>"inline"</code>	<code>page</code> has 48px vertical padding and a <code>display-sm</code> title; <code>inline</code> is compact for tables, cards and menus.
<code>titleAs?</code>	<code>"h1"</code> <code>"h2"</code> <code>"h3"</code> <code>"p"</code>	Element for the title: <code>h2</code> for <code>page</code> , <code>p</code> for <code>inline</code> by default.

InsightRow

A health finding, its evidence and one fix

A health finding with its evidence and one fix: a live key outside production, a key missing in an environment, a rotation that is overdue, values shared between environments.

- **Anatomy.** A surface row with a border hairline and `--r-8` corners. A 28px tile with `--r-6` corners on the severity's soft ground: `danger-soft` with `circle-alert` in `danger`, `warning-soft` with `triangle-alert` in `warning`, `fill` with `info` in `text-secondary`. The title in `body-medium` text, the description in `small text-secondary` (keys inside in `mono-sm`), then the evidence: affected keys as `mono-sm` chips on `fill` with `--r-4` corners and the environments as `EnvBadges` at `sm`. At the right: the fix as a secondary `Button` at `sm`, `bell-off` "Snooze for 7 days" and `x` "Dismiss" as `IconButton`s, and an optional menu.
- **Compact.** One 36px line for the project overview: the 16px severity icon in its color, the title in `small-medium` (truncates), the affected environments as 6px `EnvDots`, and the fix as a ghost `Button` in a fixed 136px column so a list of rows aligns. Hover fills `bg-subtle`.
- **Severity.** `danger` when a secret is exposed or could charge, email or delete real things (a live key outside production). `warning` for drift and overdue rotation. `info` for patterns worth a look (identical values, empty values). The icon always carries a text label for screen readers ("Critical", "Warning", "Info"); never rely on color alone.
- **Behaviour.** The fix does the smallest safe thing and routes through staging: "Rotate key" opens rotation, "Add to production" stages the key, "Review" opens the compare view filtered to the keys. Snooze hides the insight for 7 days; Dismiss hides it until the evidence changes. Below 560px the buttons move under the text.
- **Copy.** Titles state the problem with the exact key, number and environment: "Staging uses a live Stripe key", "JWT_SIGNING_KEY has not been rotated in 214 days", "FEATURE_NEW_CHECKOUT is missing in production", "3 values are identical in staging and production". Descriptions say what it means and what to do. Actions are verbs: "Rotate key", "Add to production", "Review".
- **Composition.** A stack with 8px gaps on the Health page, sorted by severity; the compact variant under a "Health" heading on the project overview, beside `StatTiles`.

InsightAction

PROP	TYPE	DESCRIPTION
<code>label</code>	<code>string</code>	Verb first: "Rotate key", "Add to production", "Review".
<code>icon?</code>	<code>string</code>	Leading Lucide icon name.
<code>iconRight?</code>	<code>string</code>	Trailing Lucide icon name, for example "arrow-right" when it navigates.
<code>onClick?</code>	<code>(event: React.MouseEvent) => void</code>	
<code>href?</code>	<code>string</code>	Renders the action as a link.

InsightRowProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>severity?</code>	<code>"danger" "warning" "info"</code>	danger (circle-alert) for exposure, warning (triangle-alert) for drift and overdue rotation, info for things worth a look.
<code>title</code>	<code>React.ReactNode</code>	One sentence that names the problem: "Staging uses a live Stripe key".
<code>description?</code>	<code>React.ReactNode</code>	What it means and what to do, one or two sentences. Keys inside go in code.
<code>keys?</code>	<code>string[]</code>	Affected keys, shown as mono chips.
<code>envs?</code>	<code>Array<Env string></code>	Affected environments, shown as EnvBadges (compact: EnvDots).
<code>action?</code>	<code>InsightAction React.ReactNode</code>	The fix: an InsightAction rendered as a secondary button (ghost in compact), or any node.
<code>onDismiss?</code>	<code>(event: React.MouseEvent) => void</code>	Shows the x "Dismiss" button.
<code>onSnooze?</code>	<code>(event: React.MouseEvent) => void</code>	Shows the bell-off "Snooze for 7 days" button.
<code>menu?</code>	<code>React.ReactNode</code>	Slot after the buttons, for example a Menu with snooze durations.
<code>variant?</code>	<code>"default" "compact"</code>	default is the full row on the Health page; compact is one line for the project overview.

Progress

4px, determinate or not, tones for outcomes

A 4px bar for work people wait on: importing a .env, exporting, rotating keys, decrypting an environment, or how close a secret is to its rotation date.

- **Anatomy.** A fill track with `--radius-full` and `corner-shape: round`, and a fill in text (neutral) or the tone token. Above it, an optional head row: the label in `small-medium`, text (with a `14px circle-check`, `triangle-alert` or `circle-alert` for tones) and the value text at the right in `small`, `text-secondary`, tabular numbers ("12 of 24", or the percent with `showValue`). Below it, an optional hint in caption, `text-tertiary` (danger for the danger tone).
- **Behaviour.** Determinate fills by sliding a full width bar with `transform` over `duration-slow` and `ease-out`, so the leading edge stays round at every value. Indeterminate slides a 40 percent bar across over 2 times `duration-reveal` with `ease-in-out`; with reduced motion it rests in the middle. The track is a `progressbar` with `aria-valuenow`, `aria-valuemax` and the value text; indeterminate omits the numbers.
- **Tones.** `neutral` while working. `success` when done ("Imported 24 secrets to staging"), `warning` near a limit ("81 of 90 days" before a rotation is due), `danger` when it stopped ("Import stopped at line 18 of 24"). Change the label with the tone; never rely on color alone.
- **Copy.** Present tense verb and object while working, past tense when done: "Importing .env to staging", "Imported 24 secrets to staging". Value text counts the real unit ("12 of 24"), not a percent, when there is one. Hints say what happens next or what to do: "3 lines could not be parsed. Fix the highlighted lines or skip them."
- **Composition.** In the import sheet footer, export dialogs, and the rotation row of a secret. Use `Spinner` for short waits inside buttons and `Skeleton` for page loads.

ProgressProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>value?</code>	<code>number</code>	Current value between 0 and max. Leave it out, or set <code>indeterminate</code> , when the length is unknown.
<code>max?</code>	<code>number</code>	Defaults to 100. Use the real total, for example 24 lines.
<code>indeterminate?</code>	<code>boolean</code>	Slides a 40 percent bar back and forth instead of filling.
<code>label?</code>	<code>React.ReactNode</code>	What is happening, in <code>small-medium</code> : "Importing .env to staging". Tones add their icon.
<code>valueLabel?</code>	<code>React.ReactNode</code>	Text at the right in tabular numbers, for example "12 of 24". Also the spoken value when it is a string.
<code>showValue?</code>	<code>boolean</code>	Shows the rounded percent at the right when there is no <code>valueLabel</code> .
<code>hint?</code>	<code>React.ReactNode</code>	A line under the bar in caption: what happens next, or what went wrong.

PROP	TYPE	DESCRIPTION
tone?	"neutral" "success" "warning" "danger"	neutral fills with text. success, warning and danger fill with their token and add circle-check, triangle-alert or circle-alert to the label.

Skeleton

The shape of what is loading, pulsing quietly

A placeholder in the shape of the content that is on its way, so the page keeps its layout while secrets decrypt or a list loads.

- **Anatomy.** Blocks on fill-hover with `--r-4` corners (`--r-6` for block, round for circle). text draws bars 10px tall on a 20px rhythm, the last of several at 62 percent width. `SkeletonRows` mimics a secret row at `row` (44px) or `row-compact` (36px): a 16px selection box, a 12px key bar in a column up to 240px, a run of 4px masked value dots, a 20px avatar and a time bar, with border hairlines between rows. Below 520px of width it keeps only the box, key and avatar.
- **Behaviour.** Pulses in opacity only (`sg-pulse`, down to 45 percent) over 2 times `duration-reveal` with `ease-in-out`, never a gradient sweep. Each row starts `duration-instant` after the one above, so the table breathes top to bottom. Reduced motion stops the pulse and leaves the static shape. `Skeleton` pieces are hidden from screen readers; `SkeletonRows` is a status region that announces its `label` once.
- **When to use.** Page and table loads that take longer than 400ms, with the expected number of rows when you know it (use the environment's secret count). Swap to real content in one step, never row by row. For a button or a short inline wait, use `Spinner`; for work with a known length, use `Progress`.
- **Copy.** `label` says what is loading and where: "Loading secrets in staging". A visible caption can name the wait: "Decrypting 24 secrets in staging".

SkeletonProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
<code>variant?</code>	<code>"text" "block" "circle"</code>	text draws lines on a 20px rhythm, block a rectangle (40px tall by default), circle a disc (24px by default).
<code>lines?</code>	<code>number</code>	Number of text lines. The last of several is 62 percent wide.
<code>width?</code>	<code>number string</code>	Width as px (number) or any CSS length. Text and block default to 100 percent.
<code>height?</code>	<code>number string</code>	Height as px (number) or any CSS length. For text, the bar height inside each line (10px by default).

SkeletonRowsProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>count?</code>	<code>number</code>	Number of rows. Match the expected count when you know it. Defaults to 5.
<code>density?</code>	<code>"comfortable" "compact"</code>	<code>comfortable</code> 44px rows or <code>compact</code> 36px rows, matching the table setting.
<code>label?</code>	<code>string</code>	Announced to screen readers while loading. Defaults to "Loading secrets".

Spinner

Indeterminate, inherits color

An indeterminate loading indicator in the Apple and Vercel style: 8 round capped spokes fading from full to 30 percent, ticking one spoke at a time through a full turn every 800ms, in the current text color.

- **Use** it inside buttons (through `Button loading`), beside a status line ("Decrypting 24 secrets"), or centred in a region that is loading for longer than 400ms. For page and table loads prefer `Skeleton` rows that match the final layout.
- **Always** say what is loading next to it, or give it a `label` for screen readers.
- **Reduced motion** stops the rotation; the spokes stay visible as a static loading mark.

SpinnerProps

PROP	TYPE	DESCRIPTION
<code>size?</code>	<code>number</code>	Size in px. 14 inside small buttons, 16 by default.
<code>label?</code>	<code>string</code>	Accessible name announced to screen readers.
<code>className?</code>	<code>string</code>	

ChangesBar

Every edit waits here until you save it

The floating bar that holds every staged edit until you save it: what changed, which environment it goes to, and the three ways out.

- **Anatomy.** A 44px bar on `surface-raised` with `--r-12` corners and `shadow-1g`, docked at the bottom centre of the panel on `z-sticky`, 16px above the edge. Left to right: the summary in `small text-secondary` with a 2px colored tick and a text medium count per kind ("2 edited · 1 new · 1 deleted", ticks in `diff-change-text`, `diff-add-text`, `diff-remove-text`), a hairline divider, the target `EnvBadge` (sm, with its lock when protected), then "Discard" (ghost), "Review" (secondary) and the primary "Save 4 changes" with the `mod+s` hint.
- **Behaviour.** The bar rises in with `ease-spring` over `duration-slow` when the first change is staged and sinks out with `ease-in` over `duration-fast` when the last one is saved or discarded; counts update in place. `mod+s` saves from anywhere on the page while it is open. `saving` swaps the Save label for a Spinner and "Saving" and disables Discard and Review. `error` replaces the summary with a danger "Could not save. 4 changes kept." and the primary action becomes "Retry"; nothing staged is lost.
- **Narrow.** Under 560px of panel width the bar spans the full width minus 16px, the summary collapses to the environment dot and "4 changes", and the Save button reads "Save" without the key hint.
- **Copy.** Always state the count: "Save 1 change", "Save 4 changes". Never "Save all" or "Submit".
- **Composition.** Place it as the last child of the scroll area that holds `SecretTable`; its rows carry the matching marks. Review opens the diff review sheet. Saving a protected environment opens the typed confirmation ("Type production to save 4 changes") before `onSave` completes.

StagedChanges

PROP	TYPE	DESCRIPTION
<code>edited?</code>	<code>number</code>	Existing keys with a new value or name.
<code>added?</code>	<code>number</code>	Keys that do not exist in the environment yet.
<code>deleted?</code>	<code>number</code>	Keys staged for deletion.

ChangesBarProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>changes?</code>	<code>StagedChanges</code>	Staged counts. The bar shows itself when their sum is above 0.
<code>env?</code>	<code>Env</code> <code>string</code>	Target environment, shown as an <code>EnvBadge</code> with its lock when protected.
<code>open?</code>	<code>boolean</code>	Overrides the automatic visibility.
<code>saving?</code>	<code>boolean</code>	Save in progress: the Save button shows a Spinner and "Saving"; Discard and Review are disabled.
<code>error?</code>	<code>boolean</code> <code>string</code>	Save failed: <code>true</code> shows "Could not save. 4 changes kept." with Retry; a string replaces the message.

PROP	TYPE	DESCRIPTION
<code>onSave?</code>	<code>() => void</code>	Called by "Save 4 changes" and by <code>mod+s</code> . For a protected environment, open the typed confirmation here.
<code>onDiscard?</code>	<code>() => void</code>	Called by the ghost Discard button. Hidden when omitted.
<code>onReview?</code>	<code>() => void</code>	Called by the secondary Review button (the diff review). Hidden when omitted.
<code>onRetry?</code>	<code>() => void</code>	Called by Retry in the error state. Defaults to <code>onSave</code> .
<code>shortcut?</code>	<code>boolean</code>	Listens for <code>mod+s</code> on the document while the bar is open and shows the key hint. Default <code>true</code> .
<code>position?</code>	<code>"sticky" "static"</code>	<code>sticky</code> docks to the bottom of the scroll area (default); <code>static</code> renders in flow, for previews and docs.

EnvEditor

Raw .env with highlighting, line marks and a live parse

The Raw .env mode of an environment: edit every secret as text, with the syntax colored, the lines numbered, and every problem marked before you stage it.

- **Anatomy.** A surface panel with a border-strong ring and --r-8 corners. A sticky gutter of line numbers in code text-tertiary, then the text in code (13/22 mono): keys in code-key medium, = and quotes in code-punct, values in code-value, \${REFERENCES} in code-ref with a dotted underline, comments in italic code-comment, export in code-punct. A status bar on bg-subtle reads "24 keys · 2 issues · Ln 12, Col 8" on the left and "Parsed as dotenv" on the right, with an xs "Hide values" button.
- **Behaviour.** A transparent textarea sits under a highlighted layer with the same metrics, so typing, selection, undo and paste stay native. The line with the caret gets a bg-subtle band and its number turns text; the whole editor takes the 1.5px focus outline flush against its border. Invalid lines (no =, a key with spaces or a leading digit, an unclosed quote) get a danger mark and number in the gutter and a wavy danger underline; duplicate keys get the same in warning. Hovering a marked line or its number shows the message ("Line 11: 3D_SECURE starts with a digit. Start keys with a letter or an underscore."). "2 issues" in the status bar is a button that moves the caret to the next one.
- **Values hidden.** valuesHidden shows every value as 12 dots and makes the editor read only, under a bg-subtle banner "Values are hidden. The editor is read only." with a "Reveal values to edit" button. Keys and comments stay readable so people can scan the file on a shared screen.
- **Parser.** parseDotenv(text) is exported for import previews and the prototype: it handles comments, blank lines, export KEY=, single, double and backtick quotes (multi-line values, \n escapes in double quotes), inline # comments after unquoted values, and reports entries with line numbers, errors with messages, and duplicates (the last value wins).
- **Copy.** Error messages say what happened and what to do: "No "=" on this line. Write it as OPENAI_API_KEY=value.", "The quote after JWT_SIGNING_KEY is never closed. Add a closing " to end the value.", "LOG_LEVEL is also set on line 4. This line wins."
- **Composition.** Sits under EnvSwitcher when the view toggle is on Raw .env. Edits stage into ChangesBar as edited, new and deleted keys when you leave the editor or press mod+s; invalid lines block staging until fixed or removed.

DotenvEntry

PROP	TYPE	DESCRIPTION
key	string	
value	string	The value with quotes removed; escaped \n in double quotes become newlines.
line	number	1-based line where the entry starts.
endLine	number	1-based line where it ends (after a multi-line quoted value).
quote	' ' " " ` ` null	The quote the value used, or null when unquoted.
exported	boolean	The line started with export .
comment	string null	Text of a trailing # comment, or null.

PROP	TYPE	DESCRIPTION
duplicate	boolean	The key appears more than once; the last one wins.

DotenvError

PROP	TYPE	DESCRIPTION
line	number	
code	"missing-equals" "invalid-key" "empty-key" "unclosed-quote"	
message	string	What happened and what to do: "No "=" on this line. Write it as OPENAI_API_KEY=value."
raw	string	The raw line.
key?	string	
value?	string	

DotenvLine

PROP	TYPE	DESCRIPTION
n	number	
text	string	
kind	"blank" "comment" "entry" "continuation" "invalid"	
tokens	Array<{ t: "space" "comment" "keyword" "key" "key-error" "punct" "quote" "value" "ref" "error"; s: string; key?: string }>	
issue	{ level: "error" "warning"; code: string; message: string } null	

DotenvParseResult

PROP	TYPE	DESCRIPTION
entries	DotenvEntry[]	Valid entries in file order, duplicates included.
errors	DotenvError[]	
duplicates	Array<{ key: string; lines: number[] }>	Keys set more than once, with every line that sets them.
lines	DotenvLine[]	
keys	number	Number of distinct valid keys.
issues	number	Number of lines with an error or a warning.

EnvEditorProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "defaultValue" | "onChange">`.

PROP	TYPE	DESCRIPTION
<code>value?</code>	<code>string</code>	Controlled text.
<code>defaultValue?</code>	<code>string</code>	Initial text when uncontrolled.
<code>onValueChange?</code>	<code>(text: string) => void</code>	Called with the full text on every edit.
<code>valuesHidden?</code>	<code>boolean</code>	Controlled values hidden mode: values show as 12 dots, the editor is read only, and "Reveal values to edit" appears.
<code>defaultValuesHidden?</code>	<code>boolean</code>	Initial values hidden mode when uncontrolled.
<code>onValuesHiddenChange?</code>	<code>(hidden: boolean) => void</code>	Called by "Reveal values to edit" (false) and by the "Hide values" button in the status bar (true).
<code>readOnly?</code>	<code>boolean</code>	Keeps the text selectable but not editable, for Viewers.
<code>env?</code>	<code>Env string</code>	Environment, used in the accessible name "Raw .env for staging".
<code>placeholder?</code>	<code>string</code>	
<code>minLines?</code>	<code>number</code>	Minimum visible lines. Default 8.
<code>maxHeight?</code>	<code>number string</code>	Height before the editor scrolls, as a CSS length or px number. Default 480.
<code>onParsed?</code>	<code>(result: DotenvParseResult) => void</code>	Called with the parse result whenever the text changes.

SecretInput

Mono value field with references

The field for writing a secret's value: mono, maskable, multiline when the value needs it, with `#{KEY}` suggestions and a one click random value.

- **Anatomy.** A surface field with a border-strong ring, --r-6 corners and shadow-xs, 32px tall (control-md) or 28px at sm. The value sits in mono; the placeholder in text-tertiary. Two xs icon buttons sit at the end in text-tertiary: Multiline (wrap-text, pressed when on) and Reveal (eye, eye-off, pressed when revealed).
- **Behaviour.** Hover lifts the ring to border-hover; focus keeps the border-hover ring and draws a 1.5px focus outline flush against it, grown out from flush and light to full strength over duration-focus. invalid turns it danger. Masked text is drawn as dots, so shoulder surfers and screen shares see nothing. Multiline swaps to a textarea that grows from 3 to maxRows rows without wrapping, and turns on by itself when the value contains a newline (the toggle then stays pressed and disabled). Typing `#{` opens a "Reference a key" list at the caret on surface-raised with shadow-md: ArrowUp and ArrowDown move, Enter or Tab inserts `#{KEY}`, Escape closes it without leaving edit mode.
- **Copy.** Placeholder "Value", or an example of the expected shape ("postgres://user:password@host:5432/db"). Button labels: "Multiline value", "Reveal value", "Hide value".
- **Composition.** SecretRow uses it in edit mode beside SecretKeyInput, at sm in compact tables. In the Add secret dialog, pair it with a Field label "Value" and the Sensitive checkbox. The tool buttons are skipped by Tab so the key, value and Save flow stays three stops.

SecretInputProps

Extends `Omit<React.InputHTMLAttributes<HTMLInputElement>, "value" | "defaultValue" | "size" | "onChange">`.

PROP	TYPE	DESCRIPTION
value?	string	Controlled value.
defaultValue?	string	Initial value when uncontrolled.
onValueChange?	(value: string) => void	Called with the next value on every edit and suggestion insert.
revealed?	boolean	Controlled mask state. false masks the characters with dots.
defaultRevealed?	boolean	Initial mask state when uncontrolled. Default false. SecretRow passes the row's reveal state so you edit what you saw.
onRevealedChange?	(revealed: boolean) => void	Called when the eye toggle changes the mask state.
multiline?	boolean	Controlled multiline preference. A value that contains newlines is always multiline.
defaultMultiline?	boolean	Initial multiline preference when uncontrolled.
onMultilineChange?	(multiline: boolean) => void	Called when the multiline toggle flips.
keys?	Array<string { key: string }>	Keys that <code>#{</code> can reference, usually every other key in the environment. Strings or objects with a key.
invalid?	boolean	Draws the danger ring and sets aria-invalid.

PROP	TYPE	DESCRIPTION
<code>disabled?</code>	boolean	
<code>readOnly?</code>	boolean	Keeps the value selectable on a <code>bg-subtle</code> ground.
<code>size?</code>	"sm" "md"	md 32px by default, sm 28px inside compact rows.
<code>placeholder?</code>	string	
<code>maxRows?</code>	number	Rows before a multiline value scrolls. Default 8.

SecretKeyInput

Normalises as you type, catches duplicates, takes a pasted .env

The field for a secret's key. It fixes the key as you type, says what it changed, stops duplicates, and hands a pasted .env block to the import flow.

- **Anatomy.** The same field shell as `SecretInput` (surface, border-strong ring, --r-6, control-md or control-sm) holding the key in mono-medium. Under it (inline) or floating over the next row (floating), one caption line: a sparkles hint in text-tertiary, or a circle-alert error in danger on a danger ring.
- **Behaviour.** Normalise on every keystroke and keep the caret where it was: letters become uppercase; spaces, dashes and dots become underscores; anything else is dropped. When the typed text changed, show "Converted to DATABASE_URL" (or "Dropped "\$". Keys use A to Z, 0 to 9 and _") for 2.4s after the last change. Validate as you type: a key that starts with a digit, or one that already exists in the environment, errors at once; an empty key errors only after blur or when `showErrors` is set (after a Save attempt). Pasting a multi-line `KEY=value` block cancels the paste and calls `onPasteEnv`, which opens the import sheet. Pasting one `KEY=value` line fills the key and calls `onPastePair` so the value field fills too.
- **Copy.** Placeholder "DATABASE_URL". Errors: "Enter a key, for example DATABASE_URL.", "Start the key with a letter or an underscore.", "DATABASE_URL already exists in staging." Always name the environment in the duplicate message.
- **Composition.** First field of the Add secret dialog (with inline messages) and of `SecretRow` edit mode (with floating messages so the row does not jump). Pass `original` when renaming so the key is not a duplicate of itself.

SecretKeyInputProps

Extends `Omit<React.InputHTMLAttributes<HTMLInputElement>, "value" | "defaultValue" | "size" | "onChange">`.

PROP	TYPE	DESCRIPTION
<code>value?</code>	<code>string</code>	Controlled key. Always normalised.
<code>defaultValue?</code>	<code>string</code>	Initial key when uncontrolled.
<code>onValueChange?</code>	<code>(key: string) => void</code>	Called with the normalised key on every edit.
<code>existingKeys?</code>	<code>string[]</code>	Keys already in the environment, for the duplicate check.
<code>env?</code>	<code>string</code>	Environment slug named in the duplicate message: "DATABASE_URL already exists in staging."
<code>original?</code>	<code>string</code>	The key being renamed. It does not count as a duplicate of itself.
<code>error?</code>	<code>string</code>	An external error that replaces the built in validation message.
<code>showErrors?</code>	<code>boolean</code>	Shows the "empty" error before the field has been blurred, for example after a Save attempt.
<code>onErrorChange?</code>	<code>(message: string null) => void</code>	Called with the visible error message, or null when the key is valid.
<code>onPasteEnv?</code>	<code>(text: string, parsed: DotenvParseResult) => void</code>	Called when a multi-line <code>KEY=value</code> block is pasted; the paste is cancelled so the app can open the import sheet.

PROP	TYPE	DESCRIPTION
onPastePair?	(pair: { key: string; value: string }) => void	Called when a single KEY=value line is pasted; the key fills this field and the app fills the value.
invalid?	boolean	Forces the danger ring without a message.
disabled?	boolean	
readOnly?	boolean	
size?	"sm" "md"	md 32px by default, sm 28px inside compact rows.
placeholder?	string	
messagePlacement?	"inline" "floating"	inline puts the hint or error under the field (forms and dialogs); floating lays it over the content below (table rows).
hintDuration?	number	Milliseconds the "Converted to DATABASE_URL" hint stays after the last change. Default 2400.

SecretRow

The secrets table: keyboard first, staged, inline edit

The secrets table, the screen `secmgr` exists to show: `SecretTable` holds `SecretTableHeader` and one `SecretRow` per key, works from the keyboard, groups by prefix, edits inline and shows every staged change before it is saved.

- **Anatomy.** A grid of five columns shared by the header and every row: a 16px checkbox, the key in `mono-medium` (truncates first to the badges, then itself), the value as a `SecretValue`, "Priya · Sep 21" in `caption text-tertiary` with an optional avatar slot, and `xs` icon buttons (reveal, copy, history, more). Rows are `--row` (44px) or `--row-compact` (36px) tall with `--r-6` corners and a 1px border hairline between them. Badges after the key: "Rotation due" (warning, history icon), "Missing in production" (danger, `circle-alert`), a sticky-note icon when the secret has a note, then any badges slot content such as "Live key".
- **Header.** Sticky at the top of the scroll area on the table's ground (`z-sticky`), in `caption medium text-tertiary`: select all (mixed when some are selected), "Key 21", "Value", "Updated". While rows are selected it turns into "3 secrets selected" with the bulk actions and Clear.
- **States.** Hover fills with `fill`. The focused row draws a 1.5px inset focus ring. Selected rows sit on `accent-soft`. Staged rows carry a 2px mark at the left edge: `diff-add-text` with an outline "New" badge, `diff-change-text` with "Edited", `diff-remove-text` for deleted, where the key is struck through, the rest dims to 48 percent and a ghost "Undo" replaces the actions. Edit mode sits on `bg-subtle` and swaps the key and value for `SecretKeyInput` (floating messages) and `SecretInput`, with "Cancel" (ghost) and "Save" (primary) at `sm`.
- **Behaviour.** Actions appear on hover and on focus, stay visible on touch, and a revealed sensitive value keeps its pressed eye visible until it hides again after 10s. Copy flips its icon to a success check for 1.6s; the app follows with the toast "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s." Clicking a row outside its controls calls `onOpen` (the detail sheet); double clicking the value edits it.
- **Keyboard.** The table is one Tab stop with roving focus. `j` or `ArrowDown` and `k` or `ArrowUp` move, Home and End jump. Enter or `e` edits, `r` reveals, `c` or `mod+c` copies, `x` or Space selects, `u` undoes a staged delete. In edit mode Enter saves, `mod+Enter` saves a multiline value, Escape cancels and returns focus to the row. Group headers toggle with Enter, Space, `ArrowLeft` and `ArrowRight`.
- **Groups.** `groups` collects keys that share a prefix (AWS_, STRIPE_, DATABASE_) under collapsible headers on `bg-subtle` with an `overline` label and a count, Linear style, and the rest under "Other".
- **Narrow.** Under 640px of table width the row stacks the key over the value, drops the Updated column, and keeps copy and the more menu at the end; reveal and history move into the menu.
- **Copy.** "Save", "Cancel", "Undo", "Reveal value", "Hide value", "Copy value", "Show history", "More actions", "Missing in production", "Rotation due", "3 secrets selected". Bulk actions name the count: "Delete 3 secrets", "Copy to production".
- **Composition.** Put the table inside the environment panel under `EnvSwitcher`, with `ChangesBar` floating at the bottom of the same scroll area. Pass a `Menu` as `menu` (or `renderMenu`) and an `xs` `Avatar` as `avatar`.

Secret

PROP	TYPE	DESCRIPTION
id?	string	Stable row id. Defaults to key; give new unsaved rows an id so they survive a rename.
key	string	Uppercase key, for example "DATABASE_URL". Empty for a new row being typed.
value?	string	The plain value. May contain newlines and \${KEY} references.
note?	string	A short note shown as a sticky-note icon after the key, with the text as its tooltip.
sensitive?	boolean	false for plain config (LOG_LEVEL, AWS_REGION): shown unmasked and never auto hidden. Default true.
updatedBy?	string { name: string }	Name of the member who last changed it, or an object with name. Rows show the first name.
updatedAt?	Date number string	When it last changed: a Date, a timestamp, an ISO string ("4 min ago", "yesterday at 18:32", "Sep 12"), or preformatted text shown as is.
rotationDue?	boolean { age: number; every?: number }	Rotation due: true, or { age: 214, every: 90 } to explain it in the badge tooltip.
missingIn?	string[]	Environment slugs where this key is missing: shows "Missing in production" in danger.
references?	string[]	Keys this value references, for the detail sheet and health checks.
status?	"unchanged" "added" "modified" "deleted"	Staged state: added (green mark and "New"), modified (amber mark and "Edited"), deleted (struck through, dimmed, Undo).

SecretRowProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onCopy">`.

PROP	TYPE	DESCRIPTION
secret	Secret	
env?	string	Environment slug, used in messages such as "DATABASE_URL already exists in staging." Inherited inside <code>SecretTable</code> .
selectable?	boolean	Shows the selection checkbox. Default true.
selected?	boolean	Controlled selection.
defaultSelected?	boolean	
onSelectedChange?	(selected: boolean) => void	Called when the checkbox, x or Space toggles selection.
editing?	boolean	Controlled edit mode. Leave undefined to let the row manage it.
defaultEditing?	boolean	
onEdit?	(secret: Secret) => void	Called when the row asks to enter edit mode (Enter, e, double click on the value).
onSave?	(next: { key: string; value: string }, secret: Secret) => void	Called with the validated key and value when Save is pressed (or Enter, or mod+Enter in a multiline value). Not called when nothing changed.

PROP	TYPE	DESCRIPTION
onCancel?	(secret: Secret) => void	Called when Cancel or Escape leaves edit mode.
onUndo?	(secret: Secret) => void	Called by the Undo button (or u) on a deleted row.
revealed?	boolean	Controlled reveal state of the value.
defaultRevealed?	boolean	Initial reveal state. Defaults to revealed for sensitive: false secrets.
onReveal?	(revealed: boolean, secret: Secret) => void	Called with the next reveal state, including the auto hide after revealTimeout.
revealTimeout?	number	Milliseconds a revealed value stays visible. Default 10000.
onOpen?	(secret: Secret) => void	Called when the row is clicked outside its controls; open the detail sheet here.
onCopy?	(secret: Secret) => void	Called after the value is copied (copy button, c or mod+c). Show the toast "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s."
onHistory?	(secret: Secret) => void	Shows the history action and is called by it.
onMore?	(secret: Secret, event: React.MouseEvent) => void	Shows a "More actions" button when no menu is given.
menu?	React.ReactNode	Slot for the more actions menu, for example a Menu with a more-horizontal xs IconButton trigger. Replaces the onMore button.
avatar?	React.ReactNode	Slot before "Priya · 4 min ago", for example an xs Avatar.
badges?	React.ReactNode	Extra badges after the built in ones, for example <Badge tone="danger">Live key</Badge>.
existingKeys?	string[]	Keys in the environment for the duplicate check. Inherited inside SecretTable.
keys?	string[]	Keys offered after \${ while editing. Defaults to existingKeys.
resolve?	(key: string) => string undefined null	Resolves references for the hover tooltip. Inherited inside SecretTable.
density?	"comfortable" "compact"	comfortable uses --row (44px), compact uses --row-compact (36px). Inherited inside SecretTable.
now?	Date number	Reference time for "4 min ago". Defaults to now.
selectedCount?	number	Number of selected rows. Above 0 the labels give way to "3 secrets selected", actions and Clear.
actions?	React.ReactNode	Bulk actions shown while rows are selected: "Copy to...", "Export", "Delete".
onClearSelection?	() => void	Shows a ghost "Clear" button in bulk mode.
count?	number	Number shown after "Key".
density?	"comfortable" "compact"	
minGroupSize?	number	Smallest prefix group. Default 2.
selectable?	boolean	
selected?	string[]	Controlled selection, as row ids.
defaultSelected?	string[]	

PROP	TYPE	DESCRIPTION
onSelectedChange?	(ids: string[]) => void	
editingId?	string null	Controlled id of the row in edit mode, or null.
defaultEditingId?	string null	
onEditingIdChange?	(id: string null) => void	
collapsedGroups?	string[]	Controlled collapsed group ids (the prefix, for example "STRIPE_").
defaultCollapsedGroups?	string[]	
onCollapsedGroupsChange?	(ids: string[]) => void	
onSave?	(secret: Secret, next: { key: string; value: string }) => void	
onCancel?	(secret: Secret) => void	
onUndo?	(secret: Secret) => void	
onOpen?	(secret: Secret) => void	
onReveal?	(secret: Secret, revealed: boolean) => void	
onCopy?	(secret: Secret) => void	
onHistory?	(secret: Secret) => void	
onMore?	(secret: Secret, event: React.MouseEvent) => void	
renderMenu?	(secret: Secret) => React.ReactNode	Returns the menu slot for a row.
renderAvatar?	(secret: Secret) => React.ReactNode	Returns the avatar slot for a row.
renderBadges?	(secret: Secret) => React.ReactNode	Returns extra badges for a row.
resolve?	(key: string) => string undefined null	Resolves \${KEY} references. Defaults to the values in secrets.
revealTimeout?	number	
now?	Date number	
header?	boolean	Renders the sticky header. Default true.
bulkActions?	React.ReactNode ((selected: Secret[]) => React.ReactNode)	Bulk actions for the header while rows are selected, or a function of the selected secrets.
empty?	React.ReactNode	Shown in the body when secrets is empty.
footer?	React.ReactNode	Shown under the rows, for example an "Add secret" ghost button.

SecretTableHeaderProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>selectable?</code>	<code>boolean</code>	
<code>checked?</code>	<code>boolean</code>	Every row is selected.
<code>indeterminate?</code>	<code>boolean</code>	Some rows are selected.
<code>onCheckedChange?</code>	<code>(checked: boolean) => void</code>	
<code>selectedCount?</code>	<code>number</code>	Number of selected rows. Above 0 the labels give way to "3 secrets selected", actions and Clear.
<code>actions?</code>	<code>React.ReactNode</code>	Bulk actions shown while rows are selected: "Copy to...", "Export", "Delete".
<code>onClearSelection?</code>	<code>() => void</code>	Shows a ghost "Clear" button in bulk mode.
<code>count?</code>	<code>number</code>	Number shown after "Key".
<code>density?</code>	<code>"comfortable" "compact"</code>	

SecretTableProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onCopy">`.

PROP	TYPE	DESCRIPTION
<code>secrets?</code>	<code>Secret[]</code>	The rows, in display order.
<code>children?</code>	<code>React.ReactNode</code>	Extra rows rendered after the data rows, for example a custom <code>SecretRow</code> .
<code>env?</code>	<code>string</code>	Environment slug for messages and the grid's accessible name ("Secrets in staging").
<code>density?</code>	<code>"comfortable" "compact"</code>	<code>comfortable</code> (44px rows) or <code>compact</code> (36px rows).
<code>groups?</code>	<code>boolean ((secret: Secret) => string null)</code>	<code>true</code> groups keys that share a prefix (<code>STRIPE_</code> , <code>AWS_</code>) under collapsible headers, with the rest under "Other". A function returns each secret's group label.
<code>minGroupSize?</code>	<code>number</code>	Smallest prefix group. Default 2.
<code>selectable?</code>	<code>boolean</code>	
<code>selected?</code>	<code>string[]</code>	Controlled selection, as row ids.
<code>defaultSelected?</code>	<code>string[]</code>	
<code>onSelectedChange?</code>	<code>(ids: string[]) => void</code>	
<code>editingId?</code>	<code>string null</code>	Controlled id of the row in edit mode, or null.
<code>defaultEditingId?</code>	<code>string null</code>	
<code>onEditingIdChange?</code>	<code>(id: string null) => void</code>	
<code>collapsedGroups?</code>	<code>string[]</code>	Controlled collapsed group ids (the prefix, for example <code>"STRIPE_"</code>).
<code>defaultCollapsedGroups?</code>	<code>string[]</code>	

PROP	TYPE	DESCRIPTION
onCollapsedGroupsChange?	(ids: string[]) => void	
onSave?	(secret: Secret, next: { key: string; value: string }) => void	
onCancel?	(secret: Secret) => void	
onUndo?	(secret: Secret) => void	
onOpen?	(secret: Secret) => void	
onReveal?	(secret: Secret, revealed: boolean) => void	
onCopy?	(secret: Secret) => void	
onHistory?	(secret: Secret) => void	
onMore?	(secret: Secret, event: React.MouseEvent) => void	
renderMenu?	(secret: Secret) => React.ReactNode	Returns the menu slot for a row.
renderAvatar?	(secret: Secret) => React.ReactNode	Returns the avatar slot for a row.
renderBadges?	(secret: Secret) => React.ReactNode	Returns extra badges for a row.
resolve?	(key: string) => string undefined null	Resolves \${KEY} references. Defaults to the values in secrets.
revealTimeout?	number	
now?	Date number	
header?	boolean	Renders the sticky header. Default true.
bulkActions?	React.ReactNode ((selected: Secret[]) => React.ReactNode)	Bulk actions for the header while rows are selected, or a function of the selected secrets.
empty?	React.ReactNode	Shown in the body when secrets is empty.
footer?	React.ReactNode	Shown under the rows, for example an "Add secret" ghost button.

SecretValue

Masked by default, revealed for 10 seconds

A secret's value as it appears in rows and detail views: hidden behind 12 fixed dots, revealed in mono on request, hidden again on its own.

- **Anatomy.** Hidden: 12 dots, 96px wide, in `text-tertiary`, the same for every value so rows never jump and length never leaks. Revealed: the value in mono on text, with a 1.5px countdown line in `border-hover` along the bottom that shrinks to nothing over `revealTimeout`. References such as `${API_BASE_URL}` render in `code-ref` with a dotted underline. Multiline values show the first line and a +3 lines chip (`caption`, `text-tertiary` on fill). An empty value reads "Empty" in italic small, `text-tertiary`, whether hidden or not.
- **Behaviour.** Reveal fades the value in over `duration-fast`. A revealed sensitive value hides itself after 10s by default (`revealTimeout`); revealing again restarts the countdown. `sensitive={false}` values (`LOG_LEVEL`, `AWS_REGION`, `S3_BUCKET`) show unmasked by default and never auto hide. With `resolve`, hovering or focusing a reference shows a tooltip on `primary` with the key and its value in this environment, or "Not set in this environment" for a broken reference. `copyable` copies with `copyText`, flips its icon to a success check for 1.6s and calls `onCopied`.
- **Layout.** Use `truncate` (default) in rows: one line, ellipsis, the chip for extra lines. Use `wrap` in detail sheets and diffs: every line, breaking anywhere, references focusable.
- **Copy.** Hidden state is announced as "STRIPE_SECRET_KEY is hidden" when `secretKey` is given. Toggle labels are "Reveal value" and "Hide value"; the copy label is "Copy value", then "Copied". The app follows a copy with the toast "Copied STRIPE_SECRET_KEY. Clipboard clears in 30s."
- **Composition.** `SecretRow` renders it in the value column and owns the reveal and copy buttons. Standalone, turn on `revealable` and `copyable` for the controls inline.

SecretValueProps

Extends `Omit<React.HTMLAttributes<HTMLSpanElement>, "children">`.

PROP	TYPE	DESCRIPTION
<code>value?</code>	<code>string</code>	The plain value. Newlines are allowed; <code>\${OTHER_KEY}</code> references are highlighted.
<code>revealed?</code>	<code>boolean</code>	Controlled reveal state.
<code>defaultRevealed?</code>	<code>boolean</code>	Initial reveal state when uncontrolled. Defaults to <code>!sensitive</code> .
<code>onRevealedChange?</code>	<code>(revealed: boolean) => void</code>	Called with the next reveal state, including <code>false</code> when the timeout hides the value.
<code>revealTimeout?</code>	<code>number</code>	Milliseconds before a revealed sensitive value hides itself, with a countdown line under it. <code>0</code> keeps it revealed. Default 10000.
<code>sensitive?</code>	<code>boolean</code>	<code>false</code> for plain config such as <code>LOG_LEVEL</code> : shown unmasked by default and never auto hidden. Default <code>true</code> .
<code>layout?</code>	<code>"truncate" "wrap"</code>	<code>truncate</code> keeps one line with an ellipsis and a "+3 lines" chip (rows); <code>wrap</code> shows every line and breaks anywhere (detail views).

PROP	TYPE	DESCRIPTION
resolve?	<code>(key: string) => string undefined null</code>	Resolves a referenced key to its value in the current environment. Hovering a <code>_\${KEY}</code> token then shows the value, or "Not set in this environment" when it returns undefined.
revealable?	<code>boolean</code>	Shows an xs eye toggle after the value.
copyable?	<code>boolean</code>	Shows an xs copy button after the value that flips to a check for 1.6s.
onCopied?	<code>(value: string) => void</code>	Called with the copied text after the clipboard write succeeds.
secretKey?	<code>string</code>	The key this value belongs to, used in the hidden state's accessible name ("STRIPE_SECRET_KEY is hidden").

VersionTimeline

Every version of one secret: reveal, restore, compare two

The version history of one secret, newest first: who changed it, what kind of change it was, the masked value with a reveal, a restore for older versions, and a side by side of any two versions.

- Anatomy.** An optional header with the key in `mono-medium`, the `EnvBadge` at `sm`, the version count and the hint "Select two versions to compare". A 20px rail with a 1px `border-strong` line joining 10px dots: hollow `border-hover` rings for older versions, a filled `primary` dot for the current one, an `accent` dot for versions picked to compare. Each version is a two line block with `--r-6` corners: "v7" in `mono-medium`, the neutral outline `Badge` "Current", the summary in `body` ("Rotated", "Edited value", "Imported from .env", "Deleted", "Created"), and the author and time in `small` at the right; below, the value (12 dots in `mono-text-tertiary`, or "Deleted" in *italic*) with its reveal button, "Restore v5" and the Compare checkbox.
- Compare.** Ticking Compare on two versions opens a panel on `bg-subtle` with a `border-hairline`: "Comparing v6 and v7", then the older version on a `diff-remove-text` marker and the newer on a `diff-add-text` marker. "Reveal values" shows both with the changed words on `diff-remove` and `diff-add`. A third pick replaces the older pick. The x clears the comparison. `onCompare(older, newer)` fires when the second version is picked.
- Behaviour.** Hover fills a version with `bg-subtle` and shows its reveal, restore and Compare controls; they stay visible on touch, and Compare stays visible for every version once one is picked. Picked versions sit on `accent-soft`. The current version has no restore. `onRestore` stages the old value as a new version (v8), so history is never rewritten. `revealTimeout` hides values again.
- Keyboard.** Tab reaches reveal, restore and Compare in each version; Space toggles Compare; the reveal button is a toggle with `aria-pressed`.
- Copy.** "v7", "Current", "Rotated", "Edited value", "Created", "Restore v5", "Compare", "Select two versions to compare", "Select one more version to compare", "Comparing v6 and v7", "Reveal values", "Clear comparison".
- Composition.** Opens in the secret's detail sheet from "Show history" (`history`) on a `SecretRow`. Restoring into a protected environment goes through the changes bar and its typed confirmation.

SecretVersion

PROP	TYPE	DESCRIPTION
<code>version</code>	<code>number</code>	Version number; shown as "v7".
<code>author</code>	<code>string</code>	Who made it: "Jonas Weber", or a token such as "ci-deploy".
<code>summary</code>	<code>string</code>	What changed: "Rotated", "Edited value", "Created", "Restored v5", "Imported from .env".
<code>value?</code>	<code>string</code>	The value at this version; undefined for a deletion.
<code>time?</code>	<code>string</code>	Display time: "4 min ago", "Sep 12".
<code>date?</code>	<code>Date number string</code>	When it was saved, used through <code>relativeTime</code> when time is not given.
<code>current?</code>	<code>boolean</code>	Marks the live version. Defaults to the first entry.

PROP	TYPE	DESCRIPTION
<code>authorType?</code>	<code>"member" "token"</code>	token draws the author's avatar as a service tile.
<code>avatar?</code>	<code>React.ReactNode</code>	Slot before the author, for example an Avatar at 16px.

VersionTimelineProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>versions</code>	<code>SecretVersion[]</code>	Versions, newest first.
<code>secretKey?</code>	<code>string</code>	The secret's key, shown in the header in mono.
<code>env?</code>	<code>Env string</code>	The environment, shown in the header as an EnvBadge.
<code>selected?</code>	<code>number[]</code>	Controlled compare selection: up to two version numbers.
<code>defaultSelected?</code>	<code>number[]</code>	Initial compare selection when uncontrolled.
<code>onSelectedChange?</code>	<code>(versions: number[]) => void</code>	
<code>onCompare?</code>	<code>(older: number, newer: number) => void</code>	Called with the older and newer version when a second version is picked.
<code>onRestore?</code>	<code>(version: number) => void</code>	Shows "Restore v5" on hover for older versions. Restoring stages a new version; it never rewrites history.
<code>avatars?</code>	<code>boolean</code>	Draws a 16px Avatar before every author that has no avatar of its own.
<code>valueRenderer?</code>	<code>(value: string undefined, context: { version: number; revealed: boolean }) => React.ReactNode</code>	Custom value cell. Return undefined to keep the masked SecretValue and the reveal button.
<code>revealTimeout?</code>	<code>number</code>	Hides revealed values after this many milliseconds.
<code>now?</code>	<code>Date number</code>	Reference time for date. Defaults to now.
<code>loading?</code>	<code>boolean</code>	Shows three skeleton versions.

EnvBadge

Color is identity; the lock means protected

The identity of an environment wherever it appears: a dot in the environment's color, its lowercase name, and a lock when it is protected.

- **Anatomy.** An 8px dot in `env-<hue>`, the name in small-medium, a 14px lock icon for protected environments, an optional count. `soft` sits on `env-<hue>-soft` with `env-<hue>-text`; `outline` sits on `surface` with a `border-strong` ring; `dot` drops the chip for dense lists.
- **Color.** Eight hues: gray, blue, teal, green, amber, orange, rose, violet. Defaults by name: development blue, preview violet, staging amber, production rose, everything else gray. People can recolor any environment; the color follows it everywhere (tabs, matrix headers, the changes bar, the command menu).
- **Protected.** A protected environment always shows the lock. Never show a lock for anything else.
- **Copy.** The name is the slug as typed: `production`, `qa-eu`. Never capitalize it.
- **Use** `EnvDot` alone in tabs, menu items and table headers where the name is already beside it.

Env

PROP	TYPE	DESCRIPTION
<code>name</code>	<code>string</code>	Lowercase slug: "development", "staging", "production", "preview".
<code>color?</code>	<code>EnvColor</code>	One of the eight environment hues. Defaults by name: development blue, staging amber, production rose, preview violet, others gray.
<code>protected?</code>	<code>boolean</code>	Protected environments require a typed confirmation to change and always show a lock.

EnvBadgeProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
<code>env?</code>	<code>Env</code> <code>string</code>	The environment object, or its name.
<code>name?</code>	<code>string</code>	Overrides the environment's name.
<code>color?</code>	<code>EnvColor</code>	Overrides the environment's color.
<code>protected?</code>	<code>boolean</code>	Overrides <code>env.protected</code> .
<code>variant?</code>	<code>"soft"</code> <code>"outline"</code> <code>"dot"</code>	<code>soft</code> chip by default; <code>outline</code> on busy grounds; <code>dot</code> for dense lists, name only with the dot.
<code>size?</code>	<code>"sm"</code> <code>"md"</code>	
<code>count?</code>	<code>number</code>	Optional count after the name, for example the number of secrets.

EnvSwitcher

Tabs in the environment's own color, or a compact select

Moves between a project's environments. Tabs sit at the top of the environment panel with an underline in the active environment's own color; the select mode fits the same choice into a tight header.

- **Anatomy.** Tabs mode: a 40px row (control-lg) on a 1px border baseline. Each tab is an EnvDot, the lowercase name in small-medium, a 12px lock for protected environments and the secret count in caption text-tertiary with tabular numbers, inside a 28px --r-6 hover pill. The active tab's name turns text and a 2px bar in env-<hue> slides under it. A "+" IconButton (sm) ends the row. Select mode: a ghost 28px trigger with the dot, name, lock and a chevrons-up-down in text-tertiary.
- **Behaviour.** Hover fills the pill with fill-hover. The underline slides and recolors over duration-base with ease-out when you switch; it does not animate on first paint. When the row is too narrow, trailing tabs move into a "3 more" button with their dots, which opens a listbox on surface-raised with shadow-md; the active environment always stays visible as a tab. The select opens the same listbox with a check on the current environment and an "Add environment" item.
- **Keyboard.** The tab list is one Tab stop. ArrowLeft and ArrowRight move and switch (activation="auto"), Home and End jump; with manual, Enter or Space switches. In the listbox, ArrowUp and ArrowDown move, Enter or Space picks, Escape closes and returns focus to the trigger, Tab closes.
- **Copy.** Names are the slugs as typed, never capitalized: development, staging, production, qa-eu. The add action reads "Add environment".
- **Composition.** Place tabs directly above SecretTable, with the panel labelled by the tab (idPrefix). Use the select in the command bar, the compare picker and narrow headers. Production keeps its lock everywhere it appears.

EnvSwitcherItem

Extends Env.

PROP	TYPE	DESCRIPTION
count?	number	Number of secrets, shown after the name in caption text-tertiary.

EnvSwitcherProps

Extends Omit<React.HTMLAttributes<HTMLDivElement>, "defaultValue" | "onChange">.

PROP	TYPE	DESCRIPTION
environments	Array<EnvSwitcherItem string>	The project's environments in display order, or their names.
value?	string	Controlled active environment name.
defaultValue?	string	Initial environment when uncontrolled. Defaults to the first.
onValueChange?	(name: string) => void	Called with the environment name the person switches to.

PROP	TYPE	DESCRIPTION
mode?	"tabs" "select"	tabs (default) for the environment panel header; select renders a ghost trigger (dot, name, lock, chevrons-up-down) with a listbox, for tight headers and toolbars.
onAdd?	() => void	Shows a "+" icon button after the tabs (and an "Add environment" item in the select) that calls this.
addLabel?	string	Label of the add action. Default "Add environment".
showCounts?	boolean	Shows secret counts after the names. Default true.
activation?	"auto" "manual"	auto switches as arrow keys move focus; manual waits for Enter or Space. Default auto.
idPrefix?	string	Prefix for tab ids. Tab <code>\${idPrefix}-tab-\${name}</code> controls the panel with id <code>\${idPrefix}-panel-\${name}</code> .

CompareMatrix

Keys by environments: same, differs, missing

A grid of keys by environments that shows, without revealing a single value, which environments agree with the reference, which differ and which are missing a key.

- **Anatomy.** A toolbar with "Only differences" and its count, a legend of the three states, and a `referenceSelect` slot. A table on surface with a border hairline and `--r-8` corners; the header row sits on `bg-subtle` and stays pinned while the body scrolls, and the key column stays pinned while the environments scroll sideways. Column headers carry an `EnvDot`, the lowercase name, a lock for protected environments, and "Reference" or the secret count in `caption text-tertiary`. Environment columns are separated by border hairlines.
- **Cells.** Same as the reference: a 14px check and the 4 character fingerprint ("a3f9") in `mono-sm text-tertiary`. Differs: `equal-not` in warning and a different fingerprint in `text`. The reference column shows its fingerprint in `text-secondary`. Missing: a dashed `border-strong` box with "Missing" in `danger`, and an "Add" ghost button that appears on row hover or focus. An empty value reads "Empty" in `italic text-tertiary`. Matching fingerprints across any two columns mean equal values.
- **Rows.** A row with any difference gets a 2px marker at the key's left edge (`diff-change-text`, or `diff-remove-text` when a key is missing somewhere) and its key in `mono-medium text`; identical rows stay quiet in `mono text-secondary`. Keys that share a prefix (`AWS_`, `DATABASE_`, `STRIPE_`) sit under a collapsible group row on `bg-subtle` with "3 keys", "2 with differences", and a per environment summary ("2 differ", "1 missing", or a check). Nested keys dim their prefix to `text-tertiary`. annotations put a badge after a key: the danger "Shared live key" on `STRIPE_SECRET_KEY` when staging and production share it; annotated rows stay visible under "Only differences".
- **Behaviour.** Clicking a cell calls `onCellClick(key, env)` to open the value; "Add" calls `onAdd(key, env)` to stage the missing key. Changing the reference recomputes every mark. Below 760px of container width environment columns narrow to 96px; below 640px the key column narrows to 144px and truncates.
- **Keyboard.** The table has one tab stop. Arrow keys move between cells and group toggles, Home and End jump within a row (with `mod` to the first and last cell), Enter and Space open a cell or toggle a group.
- **Copy.** "Only differences", "Same as production", "Differs", "Missing", "Add", "Empty", "Reference", "24 secrets", "3 keys", "2 with differences", "Shared live key", "No differences. 4 environments match on all 24 keys."
- **Composition.** The Compare page's matrix view, beside `DiffView` for a side by side of two columns. Put a `Select` or `SegmentedControl` of environments in `referenceSelect`.

CompareMatrixProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>envs</code>	<code>Array<Env string></code>	The environments to compare, 2 to 5, in column order.
<code>values</code>	<code>Record<string, Record<string, string>></code>	Values by environment name, then by key. A key absent from an environment's map is "Missing"; an empty string is "Empty".

PROP	TYPE	DESCRIPTION
<code>reference?</code>	<code>string</code>	The environment every other column is compared with. Defaults to the first protected environment, else the first column.
<code>referenceSelect?</code>	<code>React.ReactNode</code>	Slot at the end of the toolbar for choosing the reference, for example a <code>Select</code> or <code>SegmentedControl</code> of environments.
<code>onlyDifferences?</code>	<code>boolean</code>	Hides keys that are identical everywhere. Controlled.
<code>defaultOnlyDifferences?</code>	<code>boolean</code>	Initial state of "Only differences" when uncontrolled.
<code>onOnlyDifferencesChange?</code>	<code>(onlyDifferences: boolean) => void</code>	
<code>groupBy?</code>	<code>"prefix" "none"</code>	prefix groups keys that share a prefix (<code>STRIPE_</code> , <code>AWS_</code> , <code>DATABASE_</code>) under collapsible rows; none lists keys flat.
<code>collapsed?</code>	<code>string[]</code>	Collapsed group prefixes, for example <code>["AWS_"]</code> . Controlled.
<code>defaultCollapsed?</code>	<code>string[]</code>	Initially collapsed group prefixes when uncontrolled.
<code>onCollapsedChange?</code>	<code>(collapsed: string[]) => void</code>	
<code>annotations?</code>	<code>Record<string, React.ReactNode></code>	Extra content after a key, by key: a danger Badge "Shared live key" on <code>STRIPE_SECRET_KEY</code> .
<code>onCellClick?</code>	<code>(key: string, env: string) => void</code>	Called when a present cell is clicked or activated with Enter, to open that value.
<code>onAdd?</code>	<code>(key: string, env: string) => void</code>	Called by the "Add" button of a missing cell. Falls back to <code>onCellClick</code> .
<code>maxHeight?</code>	<code>number string</code>	Height of the scroll area; the header row and the key column stay pinned while it scrolls.
<code>loading?</code>	<code>boolean</code>	Shows skeleton rows while values load.

DiffView

Base on the left, compare on the right, identical keys folded

Two environments side by side, base on the left and compare on the right, with the keys that differ first and the identical keys folded into one row.

- **Anatomy.** A toolbar with the summary counts ("12 differ · 1 missing in production · 11 identical"), each led by a 2px mark in its diff color, then "Only differences" and "Reveal values". A column header on `bg-subtle` with "Key", the base `EnvBadge`, an `arrow-left-right` icon over the gutter, and the compare `EnvBadge`. Rows of at least 40px: a 2px status marker at the left edge (`diff-change-text` changed, `diff-remove-text` missing in compare, `diff-add-text` only in compare), the key in `mono-medium`, the base value, a 24px gutter, the compare value, and a reveal button. The card is `surface` with a border hairline and `--r-8` corners.
- **Values.** Masked values are 12 dots in `mono text-tertiary`. Revealed values wrap in `mono`; on changed rows the words that differ sit on `diff-change` in `diff-change-text`. A key missing on one side shows a dashed border-strong chip with "Missing" in danger. An empty value reads "Empty" in `italic text-tertiary`.
- **Relation gutter.** Changed rows show `equal-not` in `diff-change-text`, identical rows show `equal` in `text-tertiary`. On hover or focus the gutter of a changed row turns into the "Copy to production" `arrow-right` button.
- **Behaviour.** Differences come first, sorted by key; identical keys collapse into "Show 11 identical keys" (`unfold-vertical`) on `bg-subtle`, which expands in place and reads "Hide 11 identical keys". "Reveal values" reveals every row; the row eye reveals one row. `revealTimeout` hides everything again after the workspace timeout. `onlyDifferences` drops the folded row entirely. Row actions appear on hover and focus and stay visible on touch: "Copy to production" in the gutter, "Add to production" inside a missing cell, and "Use staging value" under the values in the stacked layout. Nothing is written directly: `onAction` stages the change for the changes bar.
- **Keyboard.** Rows use a roving tab stop. `j`, `k` and the arrow keys move between rows, `Home` and `End` jump, `r` reveals the focused row, `Enter` or `Space` on the folded row expands it, `Tab` moves into the row's buttons.
- **Narrow.** Below 640px of container width each row stacks: the key with its actions, then the base and compare values, each labelled by an `EnvDot` and the environment name. The column header becomes the two badges and the arrow.
- **Copy.** "12 differ", "1 missing in production", "1 only in production", "11 identical", "Show 11 identical keys", "Copy to production", "Add to production", "Use staging value", "Reveal values", "Hide values", "Comparing staging and production", "staging and production match on all 24 keys."
- **Composition.** Sits in the Compare page under the environment pickers, or in the review sheet of staged changes. Pass `valueRenderer` to swap the mask for `SecretValue`. Pair with `CompareMatrix` for more than two environments.

DiffRow

PROP	TYPE	DESCRIPTION
key	string	
status	"added" "removed" "changed" "same"	
base?	string	Value in the base environment; undefined when the key is missing there.
compare?	string	Value in the compare environment; undefined when the key is missing there.

DiffValueContext

PROP	TYPE	DESCRIPTION
key	string	
env	string	Name of the environment this value belongs to.
side	"base" "compare"	
revealed	boolean	Whether this row is revealed through the header toggle or its own reveal button.
status	DiffRow["status"]	

DiffViewProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onChange">`.

PROP	TYPE	DESCRIPTION
base	Env string	The environment on the left, the one you compare from, for example staging.
compare	Env string	The environment on the right, the one you compare against, for example production.
baseValues?	Record<string, string>	Values of base by key. Ignored when rows is given.
compareValues?	Record<string, string>	Values of compare by key. Ignored when rows is given.
rows?	DiffRow[]	Precomputed rows, for example from <code>diffEnvs(baseValues, compareValues)</code> .
revealed?	boolean	Controlled state of the header "Reveal values" toggle.
defaultRevealed?	boolean	Initial state of the header toggle when uncontrolled.
onRevealedChange?	(revealed: boolean) => void	
onlyDifferences?	boolean	Hides the identical keys, including the "Show 20 identical keys" row. Controlled.
defaultOnlyDifferences?	boolean	Initial state of the "Only differences" filter when uncontrolled.
onOnlyDifferencesChange?	(onlyDifferences: boolean) => void	
showIdentical?	boolean	Whether the folded identical keys are expanded. Controlled.

PROP	TYPE	DESCRIPTION
<code>defaultShowIdentical?</code>	<code>boolean</code>	Initial expansion of the identical keys when uncontrolled.
<code>onShowIdenticalChange?</code>	<code>(open: boolean) => void</code>	
<code>revealTimeout?</code>	<code>number</code>	Hides every revealed value after this many milliseconds, following the workspace reveal timeout (10000 by default in settings). Omit to keep values revealed.
<code>onAction?</code>	<code>(action: DiffAction, row: DiffRow) => void</code>	Called by the row actions: "Copy to production", "Use staging value", "Add to production". Row actions render only when this is given.
<code>valueRenderer?</code>	<code>(value: string, context: DiffValueContext) => React.ReactNode</code>	Custom value cell. Return undefined to fall back to the masked SecretValue and the word highlight.
<code>toolbar?</code>	<code>React.ReactNode</code>	Extra controls at the start of the tool group, for example an environment picker.
<code>loading?</code>	<code>boolean</code>	Shows skeleton rows and "Comparing staging and production" while values load.

DropZone

Drop or paste a `.env`; nothing is saved until you review

Where a `.env` lands: a full page overlay while a file is dragged over the window, and an inline dashed box in empty states and the import sheet. It reads the file and hands over its text; nothing is saved until the person reviews it in `ImportPreview`.

- **Anatomy, overlay.** A surface scrim at 92 percent opacity over the whole viewport, then a frame inset by 16px with a 2px dashed border-hover line and `--r-12` corners. Centred: a 48px surface tile with shadow-sm holding a 24px file-text, the title "Drop `.env` to import into" with the target `EnvBadge` in title, and "We parse it first. Nothing is saved until you review." in body text-secondary. While a file is over the page the frame turns accent on accent-soft with a 4px accent-soft ring.
- **Anatomy, inline.** A 1px dashed border-strong box with `--r-8` corners: a 40px fill tile with file-text, "Drag a `.env` file here, or paste its contents" in body-medium, the hint "Text files up to 1 MB. Nothing is saved until you review." in small text-tertiary, and a secondary "Choose file" button with upload.
- **States.** Idle, hover (border-hover), focus (the focus ring; the line changes to "Press Cmd V to paste a `.env`"), drag over (accent border and 1px ring, accent-soft ground, file-up, "Drop to import into staging"), reject (danger border, danger-soft ground, file-x, "Only text files up to 1 MB" with the reason: "huge.env is 1.05 MB.", "logo.png is not a text file.", "Drop one file at a time."), loading ("Reading lumen-api.env" with a Spinner), disabled (bg-subtle, text-disabled).
- **Behaviour.** Enter and leave events are counted so moving over child elements never flickers the state; the overlay appears on the first enter that carries files and hides when the count returns to zero. Obvious rejects (images, several files) show the reject state while dragging. A drop is checked with `checkEnvFile`: text by type or name (`.env`, `.env.local`, `.txt`, `.json`, `.yaml`), one file, at most `maxSize`. Accepted files are read and passed to `onFile(file, text)`; refused ones call `onReject(file, reason)` and hold the reject state for 2.4s. `onText` receives pasted text on the focused inline box, or anywhere outside inputs with `pasteAnywhere`.
- **Composition.** Mount one overlay per page for the current environment. Turn it off (disabled, or unmount) while a sheet with an inline zone is open, because the overlay sits above everything. Send the text to `ImportPreview` with `text` and the environment's current values.
- **Copy.** "Drop `.env` to import into staging", "We parse it first. Nothing is saved until you review.", "Drag a `.env` file here, or paste its contents", "Choose file", "Only text files up to 1 MB", "Press Cmd V to paste a `.env`", "Reading lumen-api.env".

DropZoneProps

Extends `Omit<React.HTMLAttributes<HTMLDivElement>, "onDrop">`.

PROP	TYPE	DESCRIPTION
<code>variant?</code>	<code>"overlay" "inline"</code>	overlay listens on the whole window and appears only while a file is dragged over it; inline is always visible.
<code>env?</code>	<code>Env string</code>	The environment the file will be imported into, named in the copy: "Drop <code>.env</code> to import into staging".

PROP	TYPE	DESCRIPTION
<code>onFile?</code>	<code>(file: File, text: string) => void</code>	Called with the accepted file and its text once it is read. Nothing is saved yet: open <code>ImportPreview</code> with the parsed lines.
<code>onText?</code>	<code>(text: string) => void</code>	Called with pasted text: on the focused inline zone, or anywhere on the page with <code>pasteAnywhere</code> .
<code>onReject?</code>	<code>(file: File null, reason: "type" "size" "multiple") => void</code>	Called when a drop or a chosen file is refused: not a text file, larger than <code>maxSize</code> , or more than one file.
<code>maxSize?</code>	<code>number</code>	Largest accepted file in bytes. Default 1 MB.
<code>state?</code>	<code>"idle" "drag-over" "reject"</code>	Forces a visual state for documentation and tests. The zone stops listening while forced.
<code>fixed?</code>	<code>boolean</code>	Overlay only. <code>true</code> (default) renders in a portal fixed to the viewport; <code>false</code> fills the nearest positioned ancestor.
<code>open?</code>	<code>boolean</code>	Overlay only. Forces the overlay to show or hide regardless of dragging.
<code>pasteAnywhere?</code>	<code>boolean</code>	Treats a paste of <code>KEY=value</code> lines anywhere on the page, outside inputs, as an import and calls <code>onText</code> .
<code>disabled?</code>	<code>boolean</code>	Ignores drops and pastes. The inline zone dims; the overlay stops listening.
<code>loading?</code>	<code>boolean</code>	Inline only. Shows a spinner while a dropped file is read or parsed.
<code>loadingLabel?</code>	<code>string</code>	Inline only. Text beside the spinner, for example "Reading lumen-api.env".
<code>rejectMessage?</code>	<code>string</code>	Overrides the reject title "Only text files up to 1 MB".

ImportPreview

Every parsed line: new, changed, unchanged, invalid, duplicate

The review step of every import: each line of a pasted or dropped .env, classified against the target environment, with a choice per conflict and the exact count that will be written.

- **Anatomy.** A header with "Import into" in heading, the target EnvBadge, the line count in text-tertiary, and "Reveal values". Below it the summary chips ("12 new", "3 changed", "8 unchanged", "1 invalid", "1 duplicate"): 24px, --r-4, a border-strong ring, a 6px dot in the status color and the count in text. A danger-soft alert when lines fail to parse. A select all bar on bg-subtle. Rows of at least 44px: checkbox, status Badge, key in mono-medium, value, and the decision control. A footer on bg-subtle holds the import button.
- **Statuses.** New is a success badge. Changed is warning and shows the current value, an arrow-right, then the new value; revealed, the old words sit on diff-remove with a strike and the new words on diff-add. Unchanged is a neutral outline badge, folded into "Show 8 unchanged keys". Invalid is danger on a danger-soft row, the key with a wavy danger underline, and the reason: "Line 14: keys cannot start with a digit". Duplicate is warning on the earlier line with "Also on line 24; the last one wins". Invalid and duplicate lines are never imported and have no checkbox.
- **Behaviour.** New and changed rows start selected. Clicking a row or its checkbox toggles it; the select all box shows the mixed state. Each changed row has an Overwrite and Skip toggle bound to the same selection: Skip unchecks the row, checking the row sets Overwrite. Chips filter the list to one status and press to primary; "Show all" clears. Values stay masked (12 dots, mono, text-tertiary) until "Reveal values". footer may be a function of the live summary so the button always states the count.
- **Keyboard.** Tab walks checkboxes, toggles and chips. In a decision toggle the arrow keys move and select, Home and End jump; it is a radiogroup with one tab stop. Space toggles a focused checkbox.
- **Copy.** "Import into", "25 lines", "12 new", "Show 8 unchanged keys", "All 15 secrets selected", "13 of 15 secrets selected", "Overwrite", "Skip", "1 line could not be parsed. Fix the highlighted line or skip it.", "Parsing .env", "No secrets found. Paste lines like KEY=value, one per line." Footer: "Cancel", "Import 15 secrets into development", "3 existing values will be overwritten".
- **Composition.** Opens in the import sheet after a paste anywhere on the page or a drop on DropZone. Feed it the parser's entries with `classifyImport(entries, existing)`. Importing into a protected environment then asks for its name.

ImportRow

PROP	TYPE	DESCRIPTION
id	string	Stable id, the line number as a string by default.
line	number	1 based line number in the pasted or dropped text.
key	string	
value	string	
status	"new" "changed" "unchanged" "invalid" "duplicate"	

PROP	TYPE	DESCRIPTION
<code>current?</code>	<code>string</code>	The value already in the target environment, for changed and unchanged rows.
<code>reason?</code>	<code>string</code>	Why an invalid line cannot be imported, completing "Line 14: ...".
<code>otherLine?</code>	<code>number</code>	For duplicate rows, the later line with the same key that wins.

ImportEntry

PROP	TYPE	DESCRIPTION
<code>line?</code>	<code>number</code>	
<code>key</code>	<code>string</code>	
<code>value</code>	<code>string</code>	
<code>error?</code>	<code>string</code>	A parser error for this line; marks it invalid.

ImportSummary

PROP	TYPE	DESCRIPTION
<code>selected</code>	<code>number</code>	Secrets that will be written: selected new rows plus changed rows set to Overwrite.
<code>selectable</code>	<code>number</code>	
<code>counts</code>	<code>Record<ImportRow["status"], number></code>	
<code>env</code>	<code>string</code>	
<code>selectedIds</code>	<code>string[]</code>	

ImportPreviewProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>env</code>	<code>Env string</code>	The environment the secrets are imported into.
<code>rows?</code>	<code>ImportRow[]</code>	Classified rows, for example from <code>classifyImport(entries, existing)</code> .
<code>text?</code>	<code>string</code>	Raw <code>.env</code> text, parsed with the shared <code>dotenv</code> parser and classified against existing when rows and entries are not given.
<code>entries?</code>	<code>ImportEntry[]</code>	Parsed entries to classify against existing when rows is not given.
<code>existing?</code>	<code>Record<string, string></code>	Current values of the target environment by key, used with entries.
<code>selected?</code>	<code>string[]</code>	Controlled ids of the rows that will be imported. Changed rows in this list are overwritten, changed rows outside it are skipped.
<code>defaultSelected?</code>	<code>string[]</code>	Initial selection when uncontrolled. Defaults to every new and changed row.

PROP	TYPE	DESCRIPTION
onSelectedChange?	(ids: string[]) => void	
revealed?	boolean	Controlled state of "Reveal values" in the header.
defaultRevealed?	boolean	
onRevealedChange?	(revealed: boolean) => void	
filter?	ImportRow["status"] null	The status chip that filters the list, or null for every line. Controlled.
defaultFilter?	ImportRow["status"] null	
onFilterChange?	(status: ImportRow["status"] null) => void	
showUnchanged?	boolean	Whether the folded unchanged rows are expanded. Controlled.
defaultShowUnchanged?	boolean	
onShowUnchangedChange?	(open: boolean) => void	
footer?	React.ReactNode ((summary: ImportSummary) => React.ReactNode)	Footer content, or a function of the live summary that returns it: (s) => <Button variant="primary">Import {s.selected} secrets into {s.env}</Button>.
valueRenderier?	(value: string, context: { key: string; side: "current" "new"; revealed: boolean }) => React.ReactNode	Custom value rendering. Return undefined to use the masked SecretValue.
loading?	boolean	Shows "Parsing .env" and skeleton rows.

ActivityItem

Who did what to which key, where and when

One audit log entry written as a sentence, "Priya Raman updated STRIPE_SECRET_KEY in staging", with its action icon and time, and ActivityFeed, which groups entries by day and joins their icons with a hairline.

- **Anatomy.** A 24px round fill tile with a 14px action icon in text-secondary (plus added, pencil updated, trash-2 deleted in danger, eye revealed and key-round read in text, lock, lock-open, rotate-cw, upload, copy, history). The sentence in body text-secondary: the actor in body-medium text (a service token such as ci-deploy in mono-medium), keys in mono text, counts in tabular text, the environment as an EnvBadge at sm. The time sits at the right in small text-tertiary, tabular. An optional avatar slot sits before the name.
- **Expandable change.** Entries with diff become a full width button with a chevron-right that turns 90 degrees. The panel below is bg-subtle with a border hairline and --r-6 corners: a diff-remove-text marker with the old version label ("v6") and value, and a diff-add-text marker with the new one ("v7"). Values stay masked (12 dots in mono text-tertiary); "Not set" and "Deleted" stand in for missing sides. Pass valueRenderer to use SecretValue, which logs a reveal of its own.
- **Feed.** Day headings in overline text-tertiary ("Today", "Yesterday", "Sep 28"), aligned with the sentences. Today's entries show relative times ("4 min ago"), earlier days show the clock ("18:32"). A 1px border-strong line runs from each icon to the next within a day. Empty: "No activity yet. Changes, reveals and reads show up here." Loading: three skeleton entries.
- **Behaviour.** Hover on an expandable entry fills bg-subtle; focus draws the focus ring inset; Enter or Space toggles the change, aria-expanded and aria-controls point at the panel. Below 480px the time moves under the sentence.
- **Copy.** Verbs by action: added to, updated in, deleted from, revealed in, read from, protected, removed protection from, rotated in, imported into, copied from staging to production, restored to v5 in. Reveals and token reads are always logged; say so where people can reveal values.
- **Composition.** The project overview shows the latest five entries without day groups; the Activity page shows the full ActivityFeed with filters for member, action and environment above it.

Props (ActivityItem): actor, actorType, action, keys, count, env, target, version, time, date, now, diff, avatar, icon, expanded, defaultExpanded, onExpandedChange, valueRenderer, connector, loading, children. Props (ActivityFeed): items, now, loading, empty, valueRenderer. Also exports dayLabel.

ActivityDiff

PROP	TYPE	DESCRIPTION
before?	string	Value before the change; undefined reads "Not set".
after?	string	Value after the change; undefined reads "Deleted".
beforeLabel?	string	Label of the old line, for example "v6". Default "Before".
afterLabel?	string	Label of the new line, for example "v7". Default "After".

ActivityItemProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>actor?</code>	<code>string</code>	A member's full name ("Priya Raman") or a service token's name ("ci-deploy").
<code>actorType?</code>	<code>"member" "token"</code>	token sets the actor in mono.
<code>action?</code>	<code>ActivityAction</code>	The action, which picks the verb and the icon.
<code>keys?</code>	<code>string string[]</code>	The key or keys acted on. One or two keys are named in mono; more collapse to a count.
<code>count?</code>	<code>number</code>	Number of secrets for bulk actions: "read 23 secrets", "imported 15 secrets".
<code>env?</code>	<code>Env string</code>	The environment, shown as an <code>EnvBadge</code> at sm.
<code>target?</code>	<code>Env string</code>	The destination environment of a copied action.
<code>version?</code>	<code>string</code>	The version a restored action went back to, for example "v5".
<code>time?</code>	<code>string</code>	Display time at the right: "4 min ago", "18:32".
<code>date?</code>	<code>Date number string</code>	When it happened; used for time through <code>relativeTime</code> when time is not given.
<code>now?</code>	<code>Date number</code>	Reference time for date. Defaults to now.
<code>diff?</code>	<code>ActivityDiff</code>	Makes the entry expandable with a masked before and after preview.
<code>avatar?</code>	<code>React.ReactNode true</code>	Slot before the actor's name. <code>true</code> draws an <code>Avatar</code> at 20px from actor (a bot tile for tokens); a node replaces it.
<code>icon?</code>	<code>string</code>	Overrides the action's icon with another Lucide name.
<code>expanded?</code>	<code>boolean</code>	Controlled expansion of the diff preview.
<code>defaultExpanded?</code>	<code>boolean</code>	
<code>onExpandedChange?</code>	<code>(expanded: boolean) => void</code>	
<code>valueRenderer?</code>	<code>(value: string, context: { side: "before" "after"; key?: string; env: string }) => React.ReactNode</code>	Custom rendering of the before and after values. Return undefined to keep the masked <code>SecretValue</code> .
<code>connector?</code>	<code>boolean</code>	Draws the hairline from this entry's icon down to the next one. <code>ActivityFeed</code> sets it.
<code>loading?</code>	<code>boolean</code>	Shows a skeleton line with a spinner in the icon tile.
<code>children?</code>	<code>React.ReactNode</code>	Replaces the generated sentence.

ActivityFeedItem

Extends `ActivityItemProps`.

PROP	TYPE	DESCRIPTION
<code>id?</code>	<code>string</code>	
<code>date?</code>	<code>Date number string</code>	Groups the entry under "Today", "Yesterday" or a date like "Sep 28".

ActivityFeedProps

Extends `React.HTMLAttributes<HTMLDivElement>`.

PROP	TYPE	DESCRIPTION
<code>items?</code>	<code>ActivityFeedItem[]</code>	Entries, newest first.
<code>now?</code>	<code>Date number</code>	Reference time for day groups and relative times. Defaults to now.
<code>loading?</code>	<code>boolean</code>	Shows three skeleton entries.
<code>empty?</code>	<code>React.ReactNode</code>	Replaces the empty state "No activity yet. Changes, reveals and reads show up here."
<code>avatars?</code>	<code>boolean</code>	Draws an Avatar before every actor that has no avatar of its own.
<code>valueRenderer?</code>	<code>ActivityItemProps["valueRenderer"]</code>	Default value renderer for every entry's diff preview.

Avatar

Initials on a stable hue, bots for tokens

Who did it: a member's initials on a hue that never changes for their name, their photo when there is one, or a bot tile for a service token. AvatarStack overlaps several with a "+2" chip.

- **Anatomy.** A round disc (`--radius-full`, `corner-shape: round`) of 16, 20, 24 or 32px. Initials are 2 letters (first and last word, or the first 2 letters of a single word or of an email's name) in 600 weight at 42 percent of the diameter; at 16px, 1 letter. The ground is `env-<hue>-soft` and the letters `env-<hue>-text`, with the hue picked from blue, teal, green, amber, orange, rose and violet by a stable FNV hash of the lowercased name, so Maya Chen has the same color on every screen. Photos cover the disc with a 1px border inset. `kind="service"` is a square tile with `--r-4` corners (`--r-6` at 32px) on fill with a border-strong ring and a bot icon in `text-secondary`.
- **Stack.** AvatarStack overlaps by 14 percent of the diameter, so initials stay whole, separates each disc with a 2px ring in the ground color (ground, surface by default; 1.5px at 16 and 20px), shows max avatars, then a `fill-active` chip with "+2" in tabular numbers whose tooltip lists the rest. The stack is one image to screen readers: "Maya Chen, Jonas Weber, Priya Raman and 2 more".
- **Behaviour.** Avatars are not interactive; the tooltip shows the name. A photo that fails to load falls back to initials without a flash of a broken image.
- **Copy.** Always pair an avatar with the name the first time it appears in a view (activity rows, member lists). Service tokens show their slug in mono: `ci-deploy`.
- **Composition.** 20px in activity rows and the "updated by" column, 24px in headers and share dialogs, 32px in member settings, 16px inside dense table cells. Hues come from the environment palette but mean nothing about environments; never place an avatar where it could be read as an environment color.

AvatarProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
<code>name</code>	<code>string</code>	Full name or email. Gives the initials ("Maya Chen" is MC, 1 letter at 16px) and the hue, which never changes for the same name.
<code>src?</code>	<code>string</code>	Photo URL. Falls back to initials if it fails to load.
<code>size?</code>	<code>16 20 24 32</code>	Diameter in px: 16 in dense rows, 20 in activity and lists, 24 by default, 32 in member settings.
<code>kind?</code>	<code>"person" "service"</code>	<code>service</code> draws a square <code>--r-4</code> tile with the bot icon, for tokens such as <code>ci-deploy</code> .
<code>alt?</code>	<code>string</code>	Accessible name. Defaults to name.
<code>title?</code>	<code>string null</code>	Tooltip. Defaults to name; pass <code>null</code> to turn it off.

AvatarStackPerson

PROP	TYPE	DESCRIPTION
name	string	Full name, or the token slug for a service.
src?	string	Photo URL. Falls back to initials if it fails to load.
kind?	"person" "service"	service draws the bot tile.

AvatarStackProps

Extends `React.HTMLAttributes<HTMLSpanElement>`.

PROP	TYPE	DESCRIPTION
people	Array<AvatarStackPerson string>	People and tokens in order of relevance, most recent first. Strings are names.
max?	number	Avatars shown before the "+N" chip. Defaults to 3.
size?	16 20 24 32	Diameter in px of every avatar and the chip. Defaults to 24.
ground?	"surface" "surface-raised" "bg" "bg-subtle"	The ground token under the stack, used for the 2px separating ring. Defaults to "surface".
label?	string	Accessible name. Defaults to "Maya Chen, Jonas Weber, Priya Raman and 2 more".